ISG 5311 - Genomic Data Analysis in Practice

Noah Reid

2023-09-14

Table of contents

Preface

This course is focused on imparting practical knowledge of how to conduct bioinformatic analyses, with a focus on teaching users to interact with computers using a command-line interface. The initial parts of the course will introduce you to the Linux operating system, the BASH shell, and how to access and schedule work on a high performance computing cluster. We will move from there to understanding sequencing data and exploring ways to manipulate and summarize it. Finally, we will explore a very common workflow, differential expression with bulk RNA-seq, as a model for how to organize and conduct an analysis project. In the last stage we will also introduce the R statistical computing language, and use differential expression analysis as a vehicle for learning about data visualization.

Materials necessary for the course

You will need a laptop or desktop computer running a Windows, MacOS or Linux operating system. Most machines produced in the last few years will be sufficient because heavy computation will be done on a powerful remote computer cluster.

You'll be working at your a computer a lot during this course, often simultaneously using multiple applications, so it will be helpful to have the right setup. Screen real estate is an important factor. If you're working at a desktop, the bigger the display(s) the better. If you are using a laptop, we recommend a desk space with a second display, mouse and keyboard, if possible. Something like this:

Figure 1: A useful, though maybe not aesthetically pleasing, work station

Part I

Intro to Linux and Bash

Part II

Learning Objectives

Learning Objectives:
Connect to a remote computer cluster
Navigate the Linux file system using the BASH shell
Manipulate files and directories
Modify file ownership and permissions
Use multiple strategies to get help and problem solve
Ask questions about the status of the system
Move files between computers
Work with file compression
Inspect files
Use regular expressions with `sed` and `grep`
Pipe inputs and outputs

Resources:

- https://www.gnu.org/savannah-checkouts/gnu/bash/manual/bash.html
- https://tldp.org/LDP/abs/html/index.html
- https://journals.plos.org/ploscompbiol/article?id=10.1371/journal.pcbi.1008645
- https://journals.plos.org/ploscompbiol/article?id=10.1371/journal.pcbi.1009207
- https://github.com/onceupon/Bash-Oneliner

1 Getting Connected

Learning Objectives:
Connect to the Xanadu cluster
Identify when a terminal window is connected to a remote computer

1.1 Objective

Our starting exercises, and indeed most of the work for the course, will take place on a remote computer cluster running a Linux operating system (the cluster is named "Xanadu"). So the first thing you're going to do is connect your local machine to that server. We'll get into many of the details to help you understand what's going on later, but to start we're going to have you follow some directions without a lot of explanation.

You can connect to the server using a computer running Windows, MacOS, or any variety of Linux. We'll cover how to do it with a machine running Windows or MacOS here, and assume that if you are running Linux as your primary operating system, you are savvy enough that you can piece this together on your own (or ask an instructor for help).

1.2 Getting ready to connect with MacOS

Start by opening the app "Terminal" on your computer. It should already be installed. MacOS is a unix-like operating system (as is Linux) and as such, you can interact with it through the command-line in a terminal without installing any special software (again, more on this later). When you open a terminal, you'll see a mostly empty window with a **prompt**. The prompt will be a single symbol, probably a `>` or a `$`. Here is an example of a terminal with a customized prompt (giving the username and current working directory):

1.3 Getting ready to connect with Windows

Connecting to a Linux-based cluster is a little more complicated on a machine running Windows than one running MacOS. There are a few different ways to do it, but our recommended

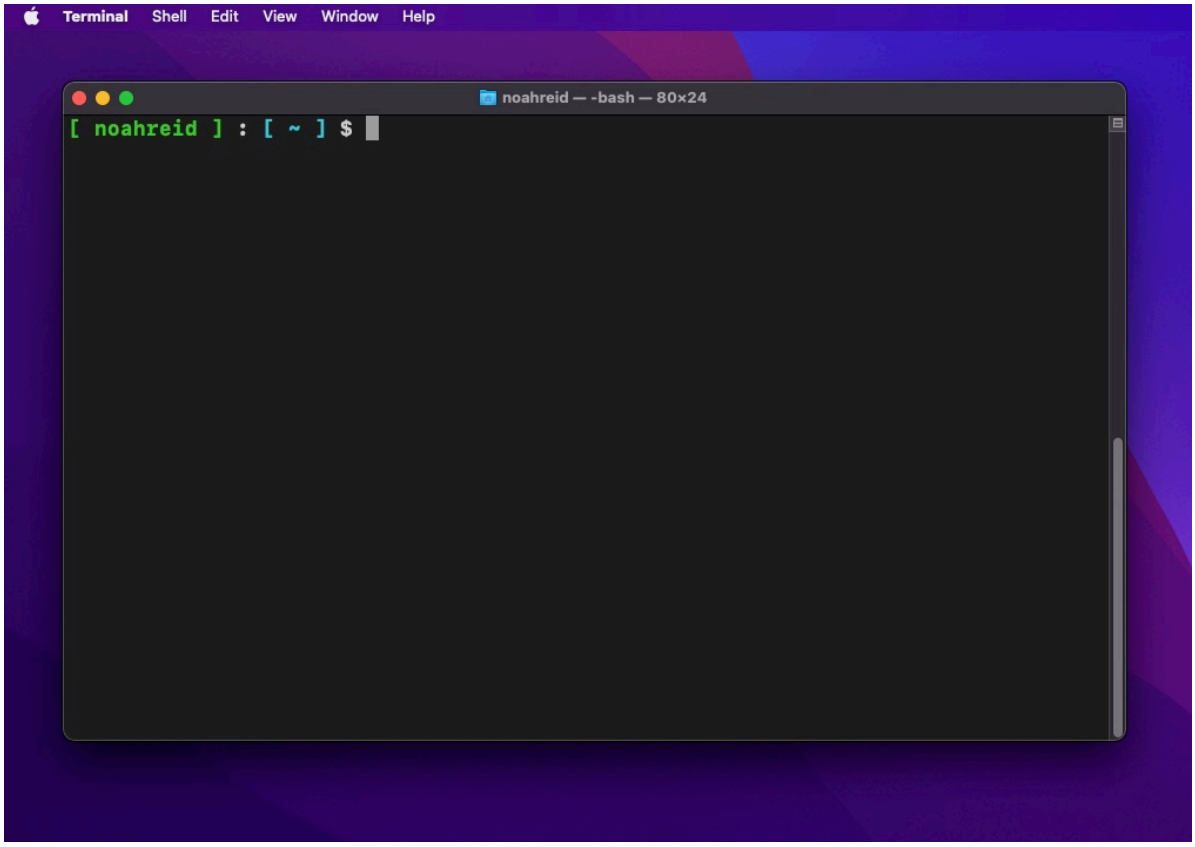

Figure 1.1: A Terminal Window

approach is to use Windows Subsystem for Linux (WSL). WSL essentially allows a Windows machine to run Linux programs, and interact with the Linux operating system through a command-line terminal. Connecting to the cluster this way will mean that most future instruction will be identical, or nearly so, for Mac and Windows users.

WSL first needs to be installed. Instructions for that are here. There are different flavors of Linux: Debian, Ubuntu, CentOS, which Xanadu runs on, and many others. the WSL default, Ubuntu, will be fine.

After you've installed WSL, you should open a Ubuntu terminal window. When you open a terminal, you'll see a mostly empty window with a **prompt**. The prompt will be a single symbol, probably a `>` or a `$`. Above is an example of a terminal with a customized prompt (giving the username and current working directory).

1.4 Connecting

You should now have a terminal window open. We're going to use SSH (*S*ecure *SH*ell protocol) to connect to Xanadu. You should already have set up your Xanadu account with a username and password (if not, see here). The username is distinct from your netID, and is usually first initial, last name (i.e. the username for John Smith will often be *jsmith*).

To login type the following command at the prompt:

```
ssh username@xanadu-submit-ext.cam.uchc.edu
```

You will be prompted for your password. When you type your password, the cursor will not move, and you won't see any characters on the screen. This is normal. Just hit enter when you've finished typing your password.

If you were successful and made no mistakes your terminal window is now connected to a remote computer (i.e. Xanadu) and you should see a screen like this:

Again, this particular terminal prompt has been customized (we'll show you how to do that later) so your prompt won't look the same, but you have now logged in to the Xanadu cluster.

1.4.1 Breaking it down

Because this is our first command issued in the course, I want to break down what we did. When you typed `ssh username@xanadu-submit-ext.cam.uchc.edu`, you used the fundamental command-line syntax `<command> <arguments>`. You told **your computer** to execute the program `ssh`, which allows you to securely gain command-line access to another computer, and provided the details ("arguments") `username@xanadu-submit-ext.cam.uchc.edu`, which are your username, and the server address of the computer you want to connect to.

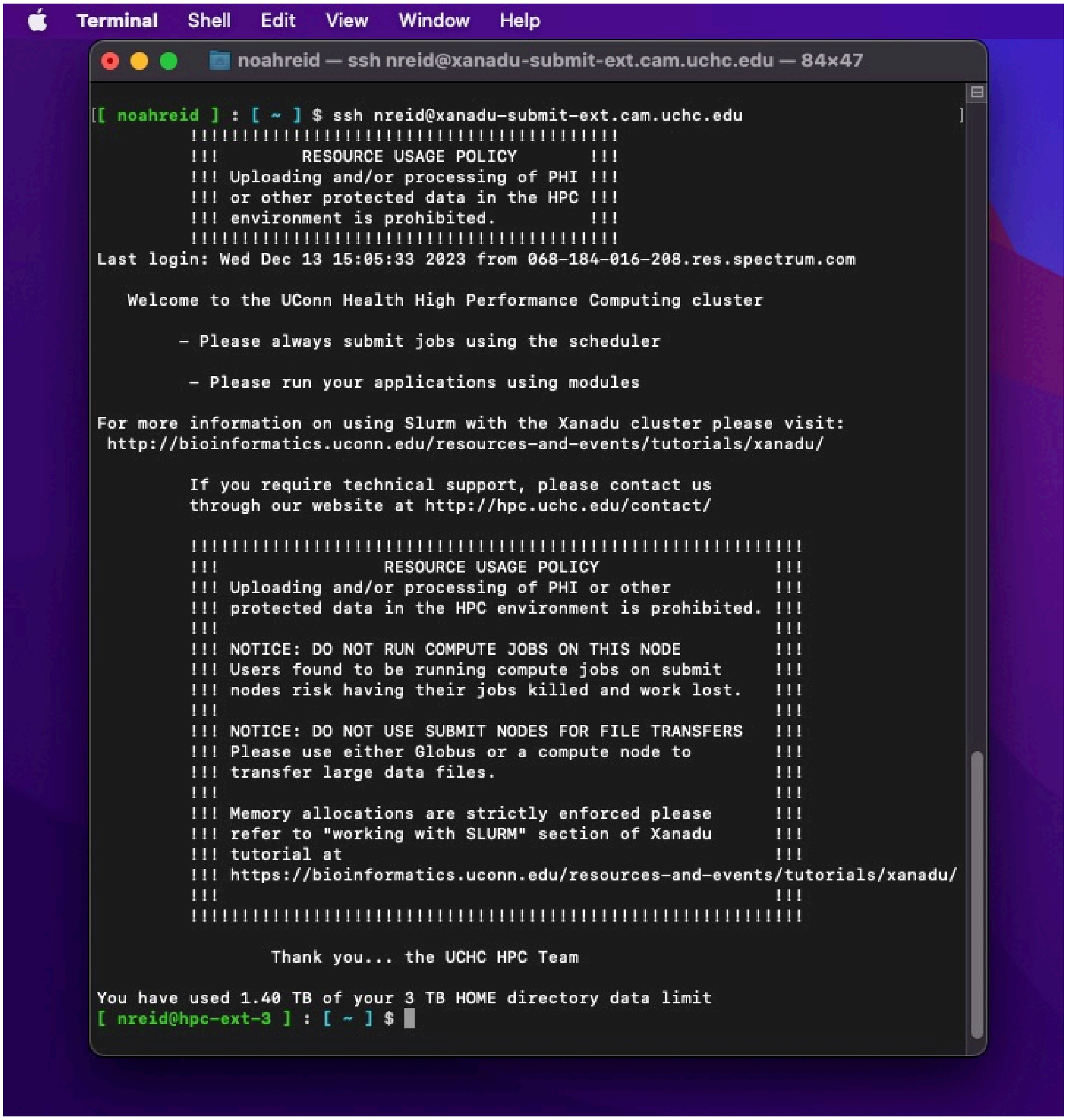

Figure 1.2: The usual Xanadu login message

I want to take a moment to emphasize something that might not be entirely clear yet, but which is very important. When you first opened your terminal window, any commands you wrote there were commands directed to and executed by **your computer**. After you ran `ssh` on your computer to connect to Xanadu, your terminal window effectively became a terminal window on the **Xanadu cluster**. Any subsequent commands you enter at the prompt will be received and executed by **Xanadu**. It is as if you were actually sitting at a terminal physically connected to one of Xanadu's nodes. Nothing you do at this terminal will impact your computer in any way, and programs running on Xanadu cannot directly access your local system or any of its files. Conversely, nothing you do outside this terminal window will impact Xanadu.

Unless you are told directly, all the work for the Linux and HPC sections of this course should be conducted on Xanadu, after using `ssh` to connect.

Now, without a customized prompt, it will not always be obvious *what computer* a given terminal is connected to. if you're working in a terminal window and you're not sure about this, you can always enter the command `hostname`. If the output is something like `hpc-ext-#`, `hpc-int-#`, or `xanadu-##`, then you're connected to Xanadu. If it's anything else you're not! Try that now. This is especially useful because in some cases you may want to have multiple terminal windows open at a time.

Ok, now that we're connected, this exercise is nearly over. You can disconnect by typing `exit`. You should see a message like this:

```
logout
Connection to xanadu-submit-ext.cam.uchc.edu closed.
```

Type `hostname` to verify that you are working locally. Now any commands you type in the terminal will be executed by your computer. Alternatively, you can simply close the terminal window, and that will kill your connection to Xanadu.

By now you've probably noticed the terms **remote** and **local** being used regularly. When we refer to doing something *locally*, we mean to do it on your personal computer, without connecting it to Xanadu, whereas *remote* means the opposite.

1.5 Linux Commands in This Section

Command	Description
cmnd	desc

2 Getting Started with Linux

Learning Objectives:
Navigate the Linux file system using the BASH shell
Manipulate files and directories
Modify file ownership and permissions
Use multiple strategies to get help and problem solve
Ask questions about the status of the system

2.1 The Linux operating system

As we have mentioned, this course will make heavy use of a high performance computing cluster running a **Linux operating system**. But what is Linux? Linux is an open-source computer operating system descended from an older operating system called Unix. There are lots of different flavors of Linux, referred to as **distributions**.

The vast majority of cutting-edge research software is built in Linux environments, and can be relatively easily compiled on most distributions. MacOS is built on a different descendant of Unix, BSD, which differs from Linux. Because of their shared ancestral design features, however, most software developed for use on Linux systems can be compiled and run on MacOS. To run Linux software on a Windows system, a good option is to install *Windows Subsystem for Linux*, which we asked you to do in the previous chapter if you are using Windows. Xanadu runs on a Linux distribution called CentOS. This is occasionally important to know when compiling software.

Linux distributions can have *graphical user interfaces*, but for scientific computing purposes, interacting with Linux through a **command-line interface** (sometimes abbreviated **CLI**) is nearly universal.

2.2 The shell and the operating system

While Linux is the **operating system** that runs the computer, command-line interaction with that operating system happens through another layer of software called a **shell**. A shell is essentially an application, like any other, that takes input from the user, passes it to the

operating system to be processed, and then returns any output, warnings or errors. There are several commonly used command-line shells, including Z shell (zsh), TC shell (tcsh) and Bourne-Again shell (bash). These shells share many features in common, but in scientific computing, **bash** is most commonly used and is what we'll use here. Newer MacOS computers default to zsh (this can be changed), but zsh is an extension of bash that is mostly compatible with it, so bash scripts should work in zsh.

Try this now: To find out what shell your local computer is running, open a terminal window and type `echo $SHELL` at the prompt. `echo` is a command that simply prints whatever input it receives and `$SHELL` is an environment variable (more about those later). The variable contains a character string, probably `/bin/bash` or `/bin/zsh`, depending on your operating system and if you have changed the defaults. If you log in to Xanadu (remember: `ssh username@xanadu-submit-ext.cam.uchc.edu`) and type the same command, it will say `/bin/bash`.

Your interactions with the command-line shell will nearly always take the form of `<command> <argument1> ... <argumentN>` You've seen this twice now when you ran `ssh` and `echo`. Using the command line, you will issue commands to navigate the file system, to manipulate files, and to execute programs.

2.3 Navigating the file system

Now that we have learned a little about the Linux operating system and how to connect to a remote computer, a natural place to begin *interacting* with Linux is to learn how to navigate the **file system**. The file system organizes and tracks data stored on a computer. It keeps track of where files are on a storage device, how big they are, when they were created, which users own them, and many other things.

These days, many people's primary experience with computers is through smartphones. Smartphones running the IOS and Android operating systems obscure and restrict access to the file system. On a smartphone, your entire interaction with the OS is through apps, of which the phone's graphical user interface (GUI) is one. For example, if you want to explore, edit, copy, or delete photos you've taken, you might use Google Photos, or some other specialized app. If you want to listen to music files you've downloaded, you might use Spotify. At no point do you, personally, manipulate any of these files.

GUIs and CLIs on personal computers, on the other hand, often give you more flexibility in access to the filesystem. You might use a photo app to explore or edit photos on your laptop, but if you wished to copy, move, or delete a photo, you could do that yourself by navigating through the filesystem to the place the photo is stored and then directly acting on the file through the operating system, rather than by asking a specialized app to do it for you.

In a GUI, the file system itself is usually represented as a nested series of directories, graphically depicted as windows. You open the window for a directory, and inside you will see files and

subdirectories, possibly in the form of a list, or maybe as a series of icons laid out in a grid. There are also likely to be directories which you can open up in new windows by pointing the mouse at them and double-clicking. You may have multiple windows open at once, but only one of these will be the window in which you are actively working. If you use a keyboard shortcut or menu option to create a new file or directory, that action will occur in the active window.

In a Linux CLI, the file system is also represented by a hierarchical directory structure, where each directory contains files and often sub-directories. There is, however, typically no graphical visualization of that structure available. Though you cannot see that structure in the form of windows, you still have something analogous to the "active window" in a GUI, a *working directory*. If you issue commands that create or modify files or directories, their effects will by default occur in your current working directory.

2.3.1 Paths

When doing any kind of work on a Linux CLI, you'll need to be aware of what the current working directory is. There are two ways to access it. First, there is a simple command, `pwd` (for **p**rint **w**orking **d**rectory). Second, there is a built-in environment variable, `$PWD`.

Try this now: Log in to Xanadu and try entering `pwd` and `echo $PWD` now. Both will return something like the following:

```
/home/FCAM/username
```

This is called a *path*. It gives the full sequence of enveloping directories for the current working directory, with each directory name separated by `/`, and ending in the current working directory with name `username`. The initial `/` refers to the *root directory*, which has no name and no parent directories. `home` is one of the directories found in root, `FCAM` is found inside `home`, and `username` is found inside `FCAM`. If you change your current working directory, the results of `pwd` and `echo $PWD` will change as well.

Assuming you executed the above commands after logging in, and before doing anything else, *the particular path* you saw is pretty important. It is your **home directory**. Xanadu, like most high performance computing clusters, has many users, and each has a home directory where only they have permission to make or modify files and directories. Because of the special status of this directory, it is also assigned a symbolic shortcut: `~`.

Try this now: Type `echo ~`. No matter what your current working directory is, `~` will always be set to your home directory.

Paths that begin with `/`, like `/home/FCAM/username` are referred to as **absolute paths** (or full paths), because they give the complete sequence of directories from the root to the target file or directory. There is no ambiguity about what an absolute path refers to on the file system.

We can also specify paths **relative** to the current working directory. Let's say your current working directory contains a project directory, `killifishGenomes` with two subdirectories `rawdata` and `scripts` that looked like this:

```
killifishGenomes/
   rawdata
   scripts
```

The absolute path to the rawdata directory looks like this:

```
/home/FCAM/username/killifishGenomes/rawdata
```

If your current working directory is `~`, however, you can equally well specify the directory with paths that are *relative* to your working directory like this:

```
killifishGenomes/rawdata
```

or this:

```
./killifishGenomes/rawdata
```

where the `.` stand in for the current working directory.

What if your current working directory is `scripts`, and you want to refer to `rawdata`? You can use `..`, to indicate the relative path goes *up* one level in the hierarchy before descending into another directory:

```
../rawdata
```

Imagine our directory structure was a little more complex, like this:

```
killifishGenomes/
   rawdata
       ont
       pacbio
   scripts
        assembly
        QC
```

Say we are working in the directory `scripts/QC` and we want to point to a file in `pacbio`. The **absolute path** is `/home/FCAM/username/killifishGenomes/rawdata/pacbio` and the path **relative** to our current working directory is `../../rawdata/pacbio`, indicating the path ascends two levels in the directory hierarchy, then descends two levels to `rawdata/pacbio`

> **Note**
>
> Relative paths can be useful in two ways. First, they are often much shorter than full paths, making scripts easier to read. Second, if you are working in a project directory that you need to move, perhaps from `/home/FCAM/username` to `/labs/principalinvestigator`, because you are running out of space in your home directory, scripts containing absolute paths (like `/home/FCAM/username/killifishGenomes/rawdata/pacbio`) pointing inside the project directory will break (i.e. point to locations in the file system that no longer exist), while those with relative paths (like `../../rawdata/pacbio`) will still function.

2.3.2 Changing the working directory

All of the above discussion of paths and working directories raises the question: How do I change my working directory? The answer is to use the command `cd`, which takes a single argument: the directory you wish to move into.

Try this now: Making sure you are logged in to Xanadu, try changing to the root directory, then type `pwd`:

```
cd /
pwd
```

Remember, `/` refers to the root. Now let's navigate to your home directory one step at a time, typing pwd after each step:

```
cd FCAM
pwd
cd username
pwd
```

If you forget where your home directory is, remember the shortcut `~`. `cd ~` will always take you home.

You can use relative and absolute paths to change directories, as we discussed above.

2.3.3 Listing directory contents

A downside of command-line navigation of the filesystem is that you don't immediately see directory contents as you do in a GUI with windows. For that, we use the command `ls`. Since we are just getting started, your home directory on Xanadu is probably empty, so if you go there and type `ls`, you should get no results.

Let's go somewhere more interesting.

Try this now: List the contents of the directory `/isg/shared/apps/`. You can do this one of two ways. Without changing the working directory, and supplying a target directory as an argument to `ls`:

```
ls /isg/shared/apps
```

Or you can move to that directory and type `ls` with no arguments:

```
cd /isg/shared/apps
ls
```

The directory contents should be a very large list. This directory contains a subdirectory for each globally installed (i.e. available to all users) software package on Xanadu. Each subdirectory contains yet more subdirectories for each version of the package installed. Let's look more closely at `iqtree`, a package used for phylogenetic inference.

```
cd /isg/shared/apps/iqtree/
ls
```

At the time of writing, there were four subdirectories, corresponding to 4 package versions:

```
1.5.5  1.6.10  1.6.6  2.1.3  2.2.2
```

Let's visit `2.2.2` and have a closer look:

```
cd 2.2.2
ls
```

You should see:

```
bin  example.cf  example.nex  example.phy  models.nex
```

Ok, great. We see some things in this directory, but can't we learn a little more about them? Yes, by supplying some arguments to `ls`.

The first let's try the argument `ls -l`. This will give you the *long form* directory listing, which looks like this:

```
total 2364
drwxr-xr-x 2 root root     512 Dec 14  2022 bin
-rw-r--r-- 1 root root 2237314 Nov 15  2022 example.cf
-rw-r--r-- 1 root root     174 Nov 15  2022 example.nex
```

```
-rw-r--r-- 1 root root   34178 Nov 15  2022 example.phy
-rw-r--r-- 1 root root  121669 Nov 15  2022 models.nex
```

We won't cover all this in detail just yet, but some key information is

- Column 1: The file permission string. The first letter (`d` or `-`) tells you whether the item is a directory or a regular file.
- Column 5: The file size (in bytes).
- Column 6,7,8: The file modification date.

For easier-to-read file sizes, supply a second argument `ls -l -h` or the more abbreviated combination of arguments `ls -lh`.

```
total 2.4M
drwxr-xr-x 2 root root  512 Dec 14  2022 bin
-rw-r--r-- 1 root root 2.2M Nov 15  2022 example.cf
-rw-r--r-- 1 root root  174 Nov 15  2022 example.nex
-rw-r--r-- 1 root root  34K Nov 15  2022 example.phy
-rw-r--r-- 1 root root 119K Nov 15  2022 models.nex
```

Now the file sizes have a suffix indicating the unit (K = kilobytes, M = megabytes, G = gigabytes)

There are a few more arguments that may come in handy:

- `-t` sorts the listing by modification date
- `-R` lists *recursively*, meaning it will also list the contents of subdirectories.

I want to introduce another concept here, the *glob* : `*`. This symbol serves as a *wildcard* when referring to files and directories. If you want to list only the files in the directory beginning with "example" you can type `ls -lh example*`

```
-rw-r--r-- 1 root root 2.2M Nov 15  2022 example.cf
-rw-r--r-- 1 root root  174 Nov 15  2022 example.nex
-rw-r--r-- 1 root root  34K Nov 15  2022 example.phy
```

You can use multiple globs, if necessary. `ls -lh *l*` will output every element containing the letter "l".

```
-rw-r--r-- 1 root root 2.2M Nov 15  2022 example.cf
-rw-r--r-- 1 root root  174 Nov 15  2022 example.nex
-rw-r--r-- 1 root root  34K Nov 15  2022 example.phy
-rw-r--r-- 1 root root 119K Nov 15  2022 models.nex
```

> **Note**
>
> There is one more way of looking at directory contents, but it's not available on all systems: `tree`. It will print a nicely formatted visualization of directory structures. Try it. You should see:
>
> ```
> .
>    bin
>       iqtree2
>    example.cf
>    example.nex
>    example.phy
>    models.nex
>
> 1 directory, 5 files
> ```
>
> For complex directories you can supply the argument `-L` and a number to specify the number of directories deep the listing should go.

2.3.4 The `find` command

It's often the case that you will want to find files in a large and complicated directory. Linux does not have an indexed search feature like Spotlight in MacOS, so if you're looking for files by name, or content within files you'll need other strategies. Here we'll discuss how to find files by name using `find`.

`find` is an extremely flexible utility included with Linux. It allows searching for files within a specified directory on combinations of attributes such as name, modification date, size, owner and more. Here we'll just cover the simple case of searching by name. The most basic usage of find is `find <path to search> <expression>`. The expression we're going to use is `-name "matchpattern"`. The match pattern can use one or more globs as a wildcard.

Try this now: We can search for all files with the suffix `fastq.gz` in the directory `/core/cbc/tutorials/workshopdirs/RNA-seq-with-reference-genome-and-annotation` like this:

```
find /core/cbc/tutorials/workshopdirs/RNA-seq-with-reference-genome-and-annotation -name "
```

Find needs to traverse all the subdirectories of a target, so in a very large and complicated project, it can be somewhat slow.

2.4 Manipulating files and directories

We have seen some basic ways to navigate the file system. Now we're going to start looking at how to examine and manipulate files and directories.

2.4.1 Creating directories

We're going to start with the basics of creating things and moving them around, and worry about the contents of files later.

Try this now: To start, let's create a series of directories and populate them with (empty) files as a demonstration. Make sure you are logged in to Xanadu and in your home directory (remember `hostname` and `pwd` to check if you're not sure). Enter the following commands at the prompt:

```
mkdir killifishGenomes
mkdir killifishGenomes/scripts
mkdir killifishGenomes/rawdata
mkdir killifishGenomes/results
```

The command `mkdir` creates a directory with the name you provide. This will create a directory structure like this:

```
killifishGenomes/
   rawdata
   results
   scripts
```

You can create multiple directories in a single command by providing multiple arguments. Perhaps the most succinct way of creating all these directories at once is with a **shell expansion**. This is a feature of the bash shell that allows lists or ranges of values to be expanded. Let's first `echo` some shell expansions just to see what it looks like. Enter the following on the command line:

```
echo {a..z}
echo {1..10}
echo {scripts,rawdata,results}
echo killifishGenomes/{scripts,rawdata,results}
```

So, to create all these directories with `mkdir` you could simply have written:

```
mkdir -p killifishGenomes/{scripts,rawdata,results}
```

The `-p` flag means to create any parent directories in the path as needed.

> **Note**
>
> Let's briefly introduce you to some errors.
> Try the following, typed **exactly**:
>
> ```
> mkdir mish/mash
> ```
>
> You got an error, right? Which argument above could have solved this for you?
> Now try this, typed **exactly**
>
> ```
> mkdir -p mish /mash
> ```
>
> You should have gotten another error. The space means `mish` and `/mash` are two separate arguments. `mkdir` created `mish` with no problem (try `ls` and you'll see), but `/mash` couldn't be created because regular users don't have permission to write in the root directory, which, if you remember, is signified by a leading `/` on any path.
> This being the beginning of your journey into bash and Linux, you will encounter many, many errors. We'll talk more about them later.

2.4.2 Creating files

There are several ways that files can be created. A very simple way is by redirecting output into a new file.

Try this now: Let's use the directory structure you created above to do this. Assuming you created the directory `killifishGenomes` in your home, and your home is your current working directory:

```
echo "This is a test directory" >killifishGenomes/README.md
```

This will write the text you `echo`ed to the file `README.md`. The `>` symbol redirects output to a file. If the file already exists, it will be overwritten. We'll talk in more detail about that later.

You can *append* new lines to that file with `>>`:

```
echo "There aren't actually any genomes in here" >>killifishGenomes/README.md
```

We're going to cover more on inspecting files later, but for now, check that your commands were successful by using the command `cat`, which will write the contents of the file to the terminal:

```
cat killifishGenomes/README.md
```

We can also create empty files using the command `touch`. I don't use `touch` often, except in demonstrations like this. You can create many files using shell expansions:

```
touch killifishGenomes/scripts/{QC,assembly}.sh
touch killifishGenomes/rawdata/sample{1..5}.fastq.gz
touch killifishGenomes/results/sample{1..5}.fasta
```

Note that above we used a range of numbers in our shell expansion, `{1..5}`, to create 10 empty files. Shell expansions can also be used with letters, e.g. `{a..z}`. Check to see that the files are where you expect them to be using `ls`.

If you've been following along so far, you can also try typing `tree killifishGenomes`. You should see this structure:

```
killifishGenomes/
   rawdata
       sample1.fastq.gz
       sample2.fastq.gz
       sample3.fastq.gz
       sample4.fastq.gz
       sample5.fastq.gz
   README.md
   results
       sample1.fasta
       sample2.fasta
       sample3.fasta
       sample4.fasta
       sample5.fasta
   scripts
        assembly.sh
        QC.sh
```

2.4.3 Moving and copying

You will often need to move or copy files and directories within a system. There are two key commands we use for these tasks: `mv` for moving, and `cp` for copying.

Let's say you've written some scripts, but you can see that as your project grows, your scripts directory is going to become crowded and feel disorganized. One solution is to moves scripts into subdirectories. `mv` takes two arguments: a path to a source file or directory, and a path to a destination directory.

Try this now: From your home directory type:

```
mkdir -p killifishGenomes/scripts/{assembly,QC}
mv killifishGenomes/scripts/assembly.sh killifishGenomes/scripts/assembly
mv killifishGenomes/scripts/QC.sh killifishGenomes/scripts/QC
```

You can optionally rename things as you move them. If you wanted to rename the `QC` directory, you can simply append a new name to the path. Try it now:

```
mv killifishGenomes/scripts/QC killifishGenomes/scripts/quality_control
```

If you've followed these steps, your directory structure will now look like this:

```
killifishGenomes/
   rawdata
       sample1.fastq.gz
       sample2.fastq.gz
       sample3.fastq.gz
       sample4.fastq.gz
       sample5.fastq.gz
   README.md
   results
       sample1.fasta
       sample2.fasta
       sample3.fasta
       sample4.fasta
       sample5.fasta
   scripts
       assembly
           assembly.sh
       quality_control
           QC.sh

5 directories, 13 files
```

Copying works similarly to moving, but the original copy of the file or directory remains in place. You provide a source file or directory and a destination. You can rename as you copy as well. Let's move to the scripts directory for this step and try copying some things:

```
cd killifishGenomes/scripts
cp assembly/assembly.sh assembly/assemblyV2.sh
cp assembly assembly_flye
```

You should have seen an error when trying to copy the directory `assembly`. To copy an entire directory you need to use the flag `-r`:

```
cp -r assembly assembly_flye
```

Your scripts directory should now look like this:

```
assembly
    assembly.sh
    assemblyV2.sh
assembly_flye
    assembly.sh
    assemblyV2.sh
quality_control
    QC.sh
```

> **Warning**
>
> If you copy, move or rename a file, and a file with that name already exists at the destination, **the pre-existing file at the destination will be overwritten**.

2.4.4 Removing things

Here is where things start to get a bit dangerous. To permanently delete files in a GUI like MacOS or Windows, you usually must take several very explicit, specific actions, and the OS often asks you if you're certain you want to. This is not the case in CLI Linux.

In Linux we have a simple command, `rm`, which takes as its main arguments the items to be deleted. It's a single step, and there are in most cases no warnings issued. The simplest cases are `rm myfile.txt` to remove a single file, or to remove a directory `rm -r mydirectory`. `rm` can accept multiple arguments as `rm -r myfile.txt mydirectory`, and it can accept the glob (`*`) as a wildcard.

Let's try this out now. We'll move to our home directory, copy our `killifishGenomes` directory, and then work on removing some things.

```
cd ~
cp -r killifishGenomes killifishGenomesCopy
rm killifishGenomesCopy/scripts/assembly/assemblyV2.sh
rm -r killifishGenomesCopy/scripts/assembly_flye
rm -r killifishGenomesCopy/results/*fasta
```

After removing these files and one directory the structure should look like this:

```
killifishGenomesCopy/
   rawdata
       sample1.fastq.gz
       sample2.fastq.gz
       sample3.fastq.gz
       sample4.fastq.gz
       sample5.fastq.gz
   README.md
   results
   scripts
       assembly
           assembly.sh
       quality_control
           QC.sh

5 directories, 8 files
```

⚠ SERIOUSLY, READ THIS.

Don't execute any code in this box
Because `rm` can accept multiple arguments and the glob as a wildcard, certain kinds of typos can be **very damaging**. You may want to remove all the files in a given directory like this:

```
rm /path/to/garbage/*
```

But if you mistakenly type this:

```
rm /path/to/garbage/ *
```

`rm` will refuse to remove directory `garbage` and give you an error (because you didn't supply `-r`) and then go on to **remove every single file in your current working directory** because it interprets the wildcard as a second independent argument.

If you accidentally type:

```
rm / path/to/garbage/*
```

then `rm` may or may not be able to remove the files in `path/to/garbage`, depending on if the current working directory is contains the path, but the single `/` means `rm` **will try to delete every file in the root directory**. This would be **very bad**.
Supplying `-r` amplifies the damage done by these typos.
Everyone who has been working at the command line long enough has a horrible story about making a mistake with `rm`, so be very cautious when using it.

2.5 Ownership and Permissions

A pervasive feature of Linux, and one that causes many beginners headaches is the permission system. Most common operating systems that manage multiple users (i.e. Windows and MacOS) manage user access to files and directories invisibly, creating walled off areas of the file system for different users.

Things work a bit differently in Linux. Permissions for files and directories are set explicitly on a case by case basis. Earlier we saw a permission string when we did `ls -l`. When I do this I see

```
drwxr-xr-x  5 nreid cbc          2.0K Dec 18 14:27 killifishGenomes
```

The first field in this output (`drwxr-xr-x`) is the permission string. It is always 10 characters. The first letter, `d` is the file type. It would be `-` for regular files and `d` for directories. The following letters come in groups of three. The groups of three correspond to the permission types for three sets of users: the user owner (`u`) of the file, the user group (`g`) the file is assigned to, and every other user on the system (`o`). The permission types are, in order, "read" (`r`), "write" (`w`), "execute" (`x`). If a given permission is *not granted* to a given set of users, that character will be `-` instead of `r`, `w`, or `x`.

Remember that fields 3 and 4 are the user owner of the file, and the user group the file is assigned to.

So the permission string above indicates that `killifishGenomes` is a directory. The user who owns the directory, `nreid`, has read/write/execute permission (characters 2-4: `rwx`). The user group `cbc` has read and execute permission (characters 5-7: `r-x`) and the rest of the users on the system also have read and execute permission (characters 8-10: `r-x`).

A few notes about this:

1. This is a directory, so execute permission may seem somewhat nonsensical. You can execute a program (or a script), but not a directory. In this case it allows you access to see the list of files in the directory and their metadata. If you don't have execute permission, you can't do anything inside the directory.
2. This directory is inside my home directory, where I am the only user with any permissions at all, therefore, per point one above, even though other users nominally have read and execute access, they effectively do not.

2.5.1 Changing permissions

We use `chmod` to change permissions. There are two ways to change permissions. We're going to learn the easy, more verbose one. To give yourself (`u`), a user group (`g`) or all other users (`o`) read, write, or execute access to a file or directory, you can specify one or more of these sets as a string, and then add `+`, or remove `-` one or more permission types as a string. For example, to give everyone full access, you can do `chmod ugo+rwx filename`. To take away write access from everyone (locking up raw data like this is always a good idea so nobody accidentally deletes it) `chmod ugo-w filename`. To apply a permission string recursively to all files and subdirectories in a directory, you can simply add `-R` as in `chmod -R ugo+rx mydirectory`.

Try this now: Take away all permissions for the group and the rest of the system users on the directory `killifishGenomes` and all its subdirectories.

```
chmod -R go-rwx killifishGenomes
```

If you do `ls -l` you should see:

```
drwx------  5 nreid cbc          2.5K Apr  4 16:51 killifishGenomes
```

And for `ls -l killifishGenomes/*`:

```
-rw------- 1 nreid cbc          67 Dec 18 14:27 killifishGenomes/README.md
-rw------- 1 nreid wegrzynlab    5 Apr  4 16:51 killifishGenomes/test.txt

killifishGenomes/rawdata:
total 20
-rw------- 1 nreid cbc 0 Dec 18 14:27 sample1.fastq.gz
-rw------- 1 nreid cbc 0 Dec 18 14:27 sample2.fastq.gz
-rw------- 1 nreid cbc 0 Dec 18 14:27 sample3.fastq.gz
-rw------- 1 nreid cbc 0 Dec 18 14:27 sample4.fastq.gz
-rw------- 1 nreid cbc 0 Dec 18 14:27 sample5.fastq.gz
```

```
killifishGenomes/results:
total 20
-rw------- 1 nreid cbc 0 Dec 18 14:27 sample1.fasta
-rw------- 1 nreid cbc 0 Dec 18 14:27 sample2.fasta
-rw------- 1 nreid cbc 0 Dec 18 14:27 sample3.fasta
-rw------- 1 nreid cbc 0 Dec 18 14:27 sample4.fasta
-rw------- 1 nreid cbc 0 Dec 18 14:27 sample5.fasta

killifishGenomes/scripts:
total 12
drwx------ 2 nreid cbc 1024 Mar  5 11:06 assembly
drwx------ 2 nreid cbc 1024 Mar  5 11:07 assembly_flye
drwx------ 2 nreid cbc  512 Mar  5 10:59 quality_control
```

2.5.2 Changing groups

For any file you own, you can change the group to any user group you are a member of using `chgrp`. To first see the groups you are a member of type `groups username` (but use your username). For example, I am a member of `reidlab` and `cbc`. I can change this directory (and all its contents recursively) from `cbc` to `reidlab` with `chgrp -R reidlab killifishGenomes`.

2.6 Getting Help

Perhaps you're getting the impression that working at the command line is going to require mastery of many, many small details. To a degree this is true. Fortunately, there are lots of ways to get help.

2.6.1 Built-in help

The most basic way to get help with a program is to try to get the program itself to print its *usage*. That is, you can ask the program to write how it should be used, and a brief description of its command line arguments. One or more of the following approaches will usually work:

1. Entering the command with no options: `<command>`
2. Entering the command with the flag `-h`: `<command> -h`
3. Entering the command with the flag `--help`: `<command> --help`

Most programs will likely print the usage with one or more of these approaches.

Another form of built-in help are manual, or `man` pages. Not every program has a `man` page, but most programs included as part of a Linux distribution will. To access a `man` page, enter

`man <command>`. The documentation will display interactively. To exit the man page and return to the shell prompt, type `q`.

Try these approaches for the commands `find` and `cat`. **Note**: `cat` with no options will drop you into an interactive session with cat. To kill the process and get back to the prompt, try typing `ctrl-.` or `cmnd-x`.

2.6.2 The Internet!

There are **tons** of resources available on the internet for learning Linux and the bash shell, troubleshooting particular commands, and accomplishing specific common tasks. Good places to look are:

1. The bash manual
2. Introduction to Linux
3. stack overflow - a community where users can ask and answer questions on a variety of topics.
4. unix stack exchange

But there are many sites, ranging in quality and focus that can help you figure out answers to questions. Even advanced users regularly consult pages like this and search google for help.

2.6.3 AI/large language models

Since 2022, the emergence of large language models like chatGPT has been transforming how we do work in bioinformatics. These models allow users to ask questions and receive answers in natural language, and they are remarkably adept at producing and interpreting code, at least for relatively straightforward tasks. They can dramatically increase the speed at which experienced users can produce code, and help elevate the abilities of new users.

For a simple example of how you can get help, visit chat.openai.com and try this question: "In bash, how can I use the find command to find all files created since 2022?"

> ⚠ Warning
>
> We encourage the use of LLMs as an aid in this program, but caution students that they can sometimes do inefficient, unpredictable or incorrect things, and that they should understand what code is doing, and verify that it is correct by checking documentation and validating it through testing. Also, be aware that sometimes an LLM will produce a result that is technically correct, but problematic for other reasons.
> For example, if you ask ChatGPT for code to calculate the variance from a sample, it will give you code for a standard statistics textbook equation for estimating variance. This

method is subject to numerical instability, however, and will yield inaccurate estimates under some circumstances. If you know enough to *ask* ChatGPT about the numerical stability issue, it will immediately suggest fixes, but we don't always know what we don't know.

2.7 Asking questions about the system

What is the name of the system you are on now? Try `hostname`. On a computer cluster like Xanadu this is really important. There are many different nodes. Sometimes a job will fail as a result of a problem with the system rather than some kind of user error. It's good to know when troubleshooting which one you were using.

Which Linux distribution and kernel version is your system running? Try `hostnamectl` (or `hostinfo` on a local MacOS machine).

How much memory is available on this system? Try `lsmem` or `cat /proc/meminfo` for a detailed view. You'll note that as you're likely on a login node, the memory available will not be impressive (something like 8 gigabyges).

How many and which CPUs are on this system? Try `lscpu` or `cat /proc/cpuinfo`. From `lscpu`, the total number of CPUs (or cores) on the system is the number of sockets times the number of cores per socket. On the login node that will probably be something paltry like 8.

What processes are currently running on the system? Try `top`. This is a list of running processes. To sort by CPU usage, type `shift-p`. To sort by memory usage type `shift-m`. Since you're most likely on a login node (rather than a heavy duty compute node), hopefully you will not see any processes using a substantial percentage of CPU or memory. Type `q` to quit. You can also use `ps aux`.

How long and how intensively has the system been used? Try `uptime`. Per the man page, `uptime` will tell you how long the system has been running, how many users are currently logged on, and the system load averages for the past 1, 5, and 15 minutes. Note that "Load averages are not normalized for the number of CPUs in a system, so a load average of 1 means a single CPU system is loaded all the time while on a 4 CPU system it means it was idle 75% of the time."

How full are available storage systems? Try `df -h`. Xanadu has several network-attached file systems (NFS). Hopefully none are too close to full…

How much space is a given directory using? We've seen that `ls -l` can tell us how much disk space a file uses, but it won't sum everything up for a whole directory. Try `du -sh`. This will give you an accounting of the space used in your current working directory. To tally

up each directory in the current working directory individually, try `du -sh *` or for a specific directory `du -sh killifishGenomes`.

Which other users are logged in to this system?: Try `who`. Xanadu has many login and compute nodes. This list is only users who are currently logged in (though they may be idle).

Which groups does a user belong to? Try `groups <username>`. Try your own username

Which users belong to a group? Try `getent group <groupname>` (try `cbc` for Computational Biology Core and associated faculty and staff).

2.8 Basic Linux Commands

Command	Description
pwd	print working directory
cd	navigate through directories
ls	list directories
find	search for files
echo	print arguments to standard out.
mkdir	create directory
mv	rename or move files
cp	copy files or directories
touch	create empty file
cat	display file contents or concatenate
rm	delete files or directories
man	get help
less	display paged outputs
chmod	change file permissions
wget	download files from the internet

2.9 Exercises

2.9.1 Practical exercise:

1. Using the commands we covered above, start a fake project directory. It should have exactly this structure:

```
fakeProject
    data
        Sample1.R1.fq.gz
        Sample1.R2.fq.gz
        Sample2.R1.fq.gz
        Sample2.R2.fq.gz
        Sample3.R1.fq.gz
        Sample3.R2.fq.gz
        Sample4.R1.fq.gz
        Sample4.R2.fq.gz
        Sample5.R1.fq.gz
        Sample5.R2.fq.gz
    genome
        mygenome.fa
    results
        alignments
            Sample1.bam
            Sample1.bam.bai
            Sample2.bam
            Sample2.bam.bai
            Sample3.bam
            Sample3.bam.bai
            Sample4.bam
            Sample4.bam.bai
            Sample5.bam
            Sample5.bam.bai
    scripts
        alignment.sh

6 directories, 22 files
```

Names ending in `.bam`, `.bam.bai` or `.fq.gz` are files (these are suffixes corresponding to common data formats, but for now create empty files with these names). Remember, you can use shell expansions, `touch` and `mkdir`. To do this quickly. If you mess up, you should know how to remove it all and start over.

2. Create a directory named after your username here `/core/cbc/gdafiles/ISG5311/2024` (but use the current year if it is no longer 2024!) and move or copy this directory there.
3. Assume you want other users in your user group, but no other users on the cluster to have access to your **data**, but nothing else. Change the permissions accordingly.

2.9.2 Quiz: See Blackboard Ultra for this section's quiz.

3 Working with Files

Learning Objectives:
Move files between computers
Work with file compression
Inspect files
Use regular expressions with `sed` and `grep`
Pipe inputs and outputs

In the previous section, we learned about the shell, how to navigate the file system and how to manipulate files and directories within the file system. Now we're going to get into how we can transfer files on and off the server, edit, inspect and summarize files in the shell. These are essential skills that will give you the flexibility to monitor your analyses and check input and output files against your basic expectations.

3.1 Moving files between computers

You will quite frequently need to move files on and off a remote computer cluster. We will sometimes use specialized pieces of software for retrieving data from particular public databases, but there a number of pieces of widely available software it's good to be familiar with. Here we'll discuss:

1. `wget` and `curl`
2. `scp`
3. `rsync`
4. GUI-based FTP clients and Globus

3.1.1 `wget` and `curl`

These are two commonly used utilities for downloading files via HTTP and FTP protocols. If you've got a URL pointing to a file you want, either of these will do. Both have lots of features and support several file transfer protocols, but for the most common use cases we'll encounter, simply invoking the command and providing the URL as the first argument will work will start the download. For `wget` you don't even need to specify a name for the downloaded file, but

for `curl` you'll need to redirect the output to a file. **Try one of the following** to download the *Fundulus heteroclitus* genome from the ENSEMBL database:

```
wget ftp://ftp.ensembl.org/pub/release-105/fasta/fundulus_heteroclitus/dna/Fundulus_hetero

curl ftp://ftp.ensembl.org/pub/release-105/fasta/fundulus_heteroclitus/dna/Fundulus_hetero
```

3.1.2 `scp`

`scp` is a *secure* version of the `cp` command we learned in the previous chapter. We can use this to securely, i.e. in encrypted fashion, transfer data between computers. The usage is the same as `cp` (requiring the `-r` flag to copy a directory), except that with `scp` you also need to provide an address for a remote host in front of the path like this: `username@remotehostaddress:/path/to/target/file.txt` You can copy data to or from a remote host. You can use `.` if you want the file to simply transfer to the current working directory.

On the Xanadu cluster, you cannot use the normal login nodes (i.e. `xanadu-submit-ext.cam.uchc.edu`) to transfer files. A special transfer host has been set up for this purpose. The address for that host is `transfer.cam.uchc.edu`. **Try this now**: Copy the *F. heteroclitus* genome we just downloaded from Xanadu to your local computer. Open a terminal window and *do not* connect to Xanadu. Enter the following command (be sure to edit the command to reflect your username and the path containing the genome on Xanadu):

```
scp nreid@transfer.cam.uchc.edu:~/Fundulus_heteroclitus.Fundulus_heteroclitus-3.0.2.dna.to
```

Hopefully it is self-evident how to transfer files **to** the remote host with this method. You can feel free to delete the transferred file, as we will not be continuing to use it.

3.1.3 `rsync`

`rsync` is like a much more elaborate version of `scp`. While its usage is generally much more flexible, the key improvement over `scp` is that `rsync` can be used to "synchronize" directories across machines. That is, if you have a copy of some analysis results on your local machine and then you update some part of the analysis on the remote cluster, if you tell rsync to copy the remote results directory to your local computer, it will check the individual files to see if they've changed and skip them if they haven't. This is a pretty useful feature. This also means it can resume failed transfers without having to start from scratch.

The usage of `rsync` is similar to scp, except that when transferring a directory we recommend using the flags `a` `v` and `z`. To move your `killifishGenomes` directory from Xanadu to your local machine you can try: `rsync -avz nreid@transfer.cam.uchc.edu:~/killifishGenomes .`

taking care to edit your username and ensuring the path on Xanadu to your directory is correct. This will transfer the entire directory.

`rsync` has lots of options, use `man rsync` to read up on it.

> **Note**
>
> A subtle but important detail: if you include a trailing `/` on the source directory like this: `rsync -avz nreid@transfer.cam.uchc.edu:~/killifishGenomes/ .` Then the **contents** of the source directory will be transferred to the target directory, but not the enveloping directory.

3.1.4 FTP clients and Globus

The above options for transferring files are all command-line programs available on most, if not all, Linux distributions. Sometimes you may want to use other methods to move data. Some users employ GUI programs that use the file transfer protocol (FTP), such as FileZilla. You can use the transfer node `transfer.cam.uchc.edu` and your Xanadu credentials to move files this way.

When you have very large amounts of data to transfer, a more secure and fault tolerant program can be helpful. On Xanadu we use Globus, a platform available on many institutional HPCs for moving data. We won't cover it right now, but you can see a guide to getting started using it on Xanadu here

3.2 File compression

'omic data files are often very large. A single copy of a human genome is around 3.3 gigabytes. A raw, unassembled sequence of a human genome at 30x coverage might be 180 gigabytes. Data at this scale is cumbersome to transfer and analyze, and expensive to store. Compression eases this burden somewhat, making these files 1/3 to 1/4 the size. So wherever we can, we try to keep data files compressed, and use tools that can read and write data in compressed form. There are a few means of compression that we'll encounter during this course, but here we'll cover `gzip`.

`gzip` refers to both a compression algorithm **and** the most common piece of software used to implement it. Above we download a genome for *F. heteroclitus*. The file name is `Fundulus_heteroclitus.Fundulus_heteroclitus-3.0.2.dna.toplevel.fa.gz`. The `.gz` suffix is commonly applied to indicate that a file has been gzip-compressed. If you list the contents of the directory containing this file with `ls -lh` you should see this compressed genome file is 278 megabytes:

```
-rw-r--r--  1 nreid cbc   278M Mar  6 19:32 Fundulus_heteroclitus.Fundulus_heteroclitus-3.
```

Try this now: Decompressing the file with

```
gunzip Fundulus_heteroclitus.Fundulus_heteroclitus-3.0.2.dna.toplevel.fa.gz
```

You'll see that `gunzip` removes the `.gz` suffix, and `ls -lh` indicates the file is now 992 megabytes, giving a compression ratio of 3.57.

```
-rw-r--r--  1 nreid cbc   992M Mar  6 19:32 Fundulus_heteroclitus.Fundulus_heteroclitus-3.
```

You can compress the file again with:

```
gzip Fundulus_heteroclitus.Fundulus_heteroclitus-3.0.2.dna.toplevel.fa
```

Note that compressing the file takes longer than decompressing.

> **!** Important
>
> Remember, because many, if not most tools that process genomic data can read and write compressed files, unless you know you need uncompressed data, **make sure you keep files compressed**.

You are likely to encounter a particular type of gzipped file known as a **T**ape **AR**chive (suffix `.tar`) or informally as a **tarball**. Tarballs are entire directories of files that have been put into an archive file and then gzipped (suffix `.tar.gz`. You will commonly encounter source code for uncompiled software distributed as tarballs, and sometimes large sets of raw data. To decompress them you can use the utility `tar` like this:

```
tar -xvzf mydir.tar.gz
```

The `-xvzf` flags indicate you want to extract the tarball (`-x -f`) that the tarball is gzipped (`-z`) and that you want it to print out what it's doing as it goes (`-v`, for *verbose*, a common option with command-line software). See the man page for more details.

3.3 Basic file inspection

Linux has a number of simple tools we can use to inspect uncompressed files in plain text. Let's download some files we can do some exploring with. To keep it simple we'll download a catalog of public domain books available through Project Gutenberg, and a plain text copy of

Charles Darwin's "On the Origin of Species By Means of Natural Selection". Let's also rename the file to something more descriptive.

```
# Catalog
wget https://www.gutenberg.org/cache/epub/feeds/pg_catalog.csv
# Origin of Species
wget https://www.gutenberg.org/ebooks/1228.txt.utf-8
mv 1228.txt.utf-8 darwin1859.txt
```

3.3.1 `less`

An easy way to scroll through this file is with the program `less`.

```
less darwin1859.txt
```

Pressing the up or down arrows will scroll through the text. The space bar will move one screen's worth of text at a time. Press `q` to exit. `less` can also view gzip-compressed text files without decompressing them.

3.3.2 `head`, `tail` and `cat`

We can print lines from the file to the screen using three commands.

`head` prints the *beginning* of the file. It prints the first 10 lines by default, but the flag `-n` can modify this:

```
head -n 20 darwin1859.txt
```

`tail` prints the the *end* of the file. It also prints 10 lines by default and accepts the `-n` flag.

```
tail -n 20 darwin1859.txt
```

The `-n` flag in tail can also accept a number in the format `+20` to indicate it should print the end of the file *starting at line 20*. Be careful with this one if the file is large.

```
tail -n +20 darwin1859.txt
```

To cancel the process, remember, try `ctrl-x` or `command-.`.

To print out the *entire file* to the terminal you can use `cat`.

```
cat darwin1859.txt
```

3.3.3 Counting with `wc`

To count the number of lines, words and bytes (a proxy for characters), you can use `wc`.

```
wc darwin1859.txt
```

You can use `wc -l` to output only the number of lines.

3.3.4 `cut`

With tabular data (i.e. our catalog of books) you can cut out columns with `cut`. Try inspecting the file with the above commands first. With cut, you can extract a column (or field) with the flag `-f`. You'll see that each of the fields in this table is separated by a `,`. `cut` expects a tab character by default, so we must also specify a delimiter with `-d`. To extract column 3, the publication date, we would do:

```
cut -f 3 -d "," pg_catalog.csv
```

You'll likely notice some messiness. `cut` is not smart, and doesn't recognize that the commas in a title like "A Medium of Inter-communication for Literary Men, Artists, Antiquaries, Genealogists" are not meant to be field separators.

3.4 Regular expressions and `grep`

We very frequently want to search a text file for occurrences of some pattern, perhaps a word, phrase, or series of numbers. `grep` is a frequently used for this purpose. Provided a file and a matching pattern, it's default behavior is to return all lines in the file containing the matching pattern. It has a lot of flexibility, however. You can have it *exclude* all lines with matches, return lines preceding and succeeding matches, print line numbers, just a count of occurrences, or instead of the whole line, only the matching string. You can check the `man` page for more detail.

How do we specify these matching patterns? with **regular expressions** (sometimes shortened to `regex`. You can think of regular expressions as a very elaborate expansion of the `*` wildcard we have already seen (e.g. `*.txt` to match any file with suffix `.txt`). This is a great page explaining lots of regex features, and is worth skimming through.

To introduce this, we will work through some examples with `grep`. **Follow along in the terminal** using the file `darwin1859.txt` that we downloaded above.

We'll start with the simplest regex, an unambiguous text string, by searching for every occurrence of the word species:

```
grep "species" darwin1859.txt
```

If we want only a count of these occurrences, we can do:

```
grep -c "species" darwin1859.txt
```

In these cases, "species" is our regular expression. What if we want to introduce some ambiguity? In regex syntax `.` is the wildcard character. It matches a single character without respect to identity (including whitespace characters). We could find "species" without regard to capitalization by:

```
grep ".pecies" darwin1859.txt
```

Note that if you actually wanted to match a period character, you need to *escape* it with a \. Escaping tells the regex interpreter to treat an otherwise special character literally. So if you want to match cases where species is the last word in a sentence:

```
grep ".pecies\." darwin1859.txt
```

Of course, this only works when the rest of the word can't have any other unintended matches. Searching for "had" with ".ad", would not only match "Had" and "had", but every other three letter string ending in "ad". Try it now:

```
grep ".ad" darwin1859.txt
```

`grep` has an option `-w` that will match only "words", i.e. only matches that are preceeded or followed by "non-word" characters or line starts/endings.

```
grep -w ".ad" darwin1859.txt
```

But this will also return words like "bad", "mad", etc. In regex syntax we can specify groups of characters, or ranges of characters inside square brackets to be more specific.

```
grep -w "[Hh]ad" darwin1859.txt
```

This will match only the full words "Had" and "had".

We can *exclude* characters by adding a `^`. Had/had are very common. If we only want to match other words we can try:

```
grep -w "[^Hh]ad" darwin1859.txt
```

What if there is some ambiguity in the length of the pattern we wish to match? There are a couple options here. Say we wish to see all words ending in "ing"? We can use the asterisk `*` to say, *match 0 or more of the preceding character* along with a bracketed set of letters. In this case we will quit using `-w` as a crutch and be explicit about the match pattern by excluding any lower-case letter characters following the match.

```
grep "[A-Za-z]*ing[^a-z]" darwin1859.txt
```

There are multiple "flavors" of regular expressions. We've been using the basic version. There are extensions with more features. We can enable extended regexes in grep with `-E`, and perl-style regexes with `-P`. For example, we can specify a number or range of repetitions with `{1,3}` for *match 1-3* of the preceding character:

```
grep -E "[^A-Za-z][A-Za-z]{1,3}ing[^a-z]" darwin1859.txt
```

With perl-style regexes (`-P`) we can also use "lookarounds". These divide a regex into two pieces, a core matching sequence, and an adjacent matching sequence that is required (or excluded), but not returned as part of the total match. We can match words following the word species with a "positive lookbehind" like this:

```
grep -P "(?<=[Ss]pecies )[A-Za-z]+" darwin1859.txt
```

Since in these last couple cases, we don't actually know what our matches are going to be, it can be helpful to return *only* the matches. You can use `-o` for this:

```
grep -o -P "(?<=[Ss]pecies )[A-Za-z]+" darwin1859.txt
```

> **Note**
>
> When you construct regular expressions, you want to test them carefully to see that they match what you want, and **only** what you want. It's pretty easy to come up with something you will work, only to discover later there are lots of unintended matches.
> On this note, large language models like ChatGPT can be pretty effective at writing and interpreting regexes for you, but they can make mistakes, and they can't necessarily anticipate unintended matches in your data any better than you can, so you should still test them before relying on them heavily in code.

3.5 STDIN, STDOUT, STDERR, redirection and the pipe

We have now seen a few different tools we can use to inspect and summarize files. Though each alone is fairly limited, one of the strengths of the Linux command-line environment is the

ability to easily chain simple tools like these together in powerful ways. To understand how this works, we need to cover a few concepts.

Linux programs operate on three main streams of data which users can redirect in various ways. First is the **standard input** (sometimes abbreviated STDIN). Programs can often read data from this stream. The second is the **standard output** (STDOUT) programs often write output to this stream. The third is the **standard error** (STDERR). This stream is often used by programs to send warning or error messages (though scientific software written by scientists, rather than software engineers, will sometimes write output directly to a file, and warnings or errors to stdout, as you will see later in the course). These three streams have numerical file descriptors 0, 1, and 2, for stdin, stdout and stderr.

For programs that write output to stdout and errors to stderr by default, these outputs will be written to the terminal. We saw this above with `grep`. When we searched our text file, the lines we requested streamed across our terminal window. If we made any typos, say referring to a non-existent file such as `darwin1959.txt`, the resulting error message would have come from the stderr stream.

We can **redirect** these streams, however, causing them to be written to files, or if we wish, taking the stdout stream from one program and hooking it up to the stdin stream for another.

Try the following:

To redirect the stdout stream to a file, let's modify one of our commands above with `>`.

```
grep "species" darwin1859.txt >species_lines.txt
```

We have now created a new file with every line containing the string "species" (we can append to an existing file with `>>`).

As above, if we did something that caused an error, such as referring to a file that doesn't exist, we would see an error in the terminal:

```
grep "species" darwin2000.txt >species_lines.txt
```

That error was written to stderr, and we can redirect it to a file like this:

```
grep "species" darwin2000.txt >species_lines.txt 2>species_lines.err
```

`2>` is taking the stderr (which has file descriptor 2) and sending it to a file. The command creates two files. Though the program failed with an error, the redirect still created the file `species_lines.txt`. It's important to know this is the usual behavior. Seeing an output file does not mean a program executed successfully. The second file, `species_lines.err` now contains the output from stderr.

You can even redirect stderr to stdout with:

```
grep "species" darwin2000.txt 2>&1
```

The `&1` refers to the stdout file descriptor. If you just wrote `2>1` stderr would be written to a new file called `1`.

3.5.1 The pipe

The character `|` is usually referred to as the **pipe**. It allows us to redirect the stdout from one program to the stdin for another, thus *piping* together one or more programs, or creating a *pipeline*, though the term pipeline is also used in a much broader sense to refer to any workflow consisting of many sequential steps.

Above we extracted every line containing the word species. We could have used `-c` to count lines We can also *pipe* the output to `wc` to count the number of lines:

```
grep ".pecies" darwin1859.txt | wc -l
```

Here the pipe is sending the stdout from `grep` to the stdin for `wc`.

For something a little less trivial, let's see a really common linux idiom:

```
grep -o -P "(?<=[Ss]pecies )[A-Za-z]+" darwin1859.txt | sort | uniq -c
```

The grep command is extracting every word that comes after "species". We then use two commands piped together, `sort` and `uniq` to tally up their frequencies. `uniq` emits unique lines, but it can only identify duplicates if they are adjacent to each other, so first we must sort them. `-c` tells `uniq` to also write the counts of each element. We can sort this yet again numerically to see the most common words following "species":

```
grep -o -P "(?<=[Ss]pecies )[A-Za-z]+" darwin1859.txt | sort | uniq -c | sort -g
```

Let's try tallying up word frequencies for the entire document. Instead of using `-w`, let's use `\b` to indicate word boundaries, restrict ourselves to words with 4 or more characters (`{4,}`), and let's only look at the 50 most common (`tail -n 50`):

```
grep -o -P "\b[A-Za-z]{4,}\b" darwin1859.txt | sort | uniq -c | sort -g | tail -n 50
```

This isn't totally optimal as it is case-sensitive, but hopefully you can begin to see how these tools will let you rapidly inspect and summarize files in useful ways.

3.5.1.1 File compression redux

Now that we've learned about the pipe, let's see how we can use it to inspect `gzip` compressed files without having to decompress them first.

Let's first use a pipe to create a second, compressed copy of `darwin1859.txt`:

```
gzip -c darwin1859.txt >darwin1859.txt.gz
```

`-c` tells `gzip` to write to the stdout, an we redirect the output to a new file.

Ok, so how can we inspect this compressed file? `zcat`!

```
zcat darwin1859.txt.gz | head
```

Or with `grep`

```
zcat darwin1859.txt.gz | grep "endless forms"
```

There is even a shortcut for the `zcat | grep` idiom:

```
zgrep "endless forms" darwin1859.txt.gz
```

This sort of thing is especially important when it comes to dealing with large compressed files that you want to inspect. You don't always have to create decompressed copies, and in fact you shouldn't if you can avoid it!

3.6 `sed`

Above we've used regexes and `grep` to match text. We can also edit text on the fly using regexes with `sed` (which stands for **S**tream **ED**itor). `sed` has lots and lots of features. See extensive documentation here and a more in-depth tutorial here. We're just going to cover two features here. Replacement operations, and controlling which lines are printed.

3.6.1 Replacements

`sed`'s replacement operator has a relatively simple syntax. `s/matchpattern/replacement/`. The `s` indicates a replacement operation, and the `/`'s delimit the match and replacement strings. Any character can be used as a delimiter, e.g. `s,matchpattern,replacement,`.

The matching patterns use regular expressions as in `grep` and extended regexes can be used with `-r`, but perl-style regexes (i.e. those with lookarounds) are not available.

Try this now: Imagine we would like to make our Project Gutenberg catalog a little less formal. Let's change all occurrences of "Darwin, Charles" to "Darwin, Chuck". First let's use grep to extract only lines containing our target string:

```
sed 's/Darwin, Charles/Darwin, Chuck/g' pg_catalog.csv
```

`sed`'s default behavior is to edit only the first match on each line. In this case, the trailing `g` in the matching expression means tells it to edit all occurrences.

You probably noticed that by default, `sed` writes all output to stdout, rather than editing the original file. If you *want* to edit the original file, you can use `-i` for "edit in place", but destructive editing *en masse* like this is not usually advisable. Let's add a `grep` command to extract only our target lines to see them more clearly:

```
grep "Darwin, Charles" pg_catalog.csv | sed 's/Darwin, Charles/Darwin, Chuck/g'
```

For cases where we have ambiguity in the match that we wish to preserve in the replacement, we can use capturing groups. These are a feature of extended regexes, and require the `-r` flag. Capturing works by putting parentheses around the target string. Captured strings can then be invoked with `\1`, `\2`… for the first, second string, etc. Let's find every name formatted as "<SURNAME>, Alfred" and reformat as "Alfred"The Greatest Author of All Time" <SURNAME>":

```
sed -r 's/([A-Z,a-z,-]*), (Alfred [A-Za-z]*),/\2 "The Greatest Author of All Time" \1,/g'
```

This will substitute names like "Brehm, Alfred Edmund" for ""Alfred Edmund"The Greatest Author of All Time" Brehm".

3.7 Line printing

We can also use `sed` to print specific lines by number. To print line number `24286`, we tell `sed` to print no lines (`-n`) except the ones we specify `24286p`.

```
sed -n '24286p' pg_catalog.csv
```

Alternatively, we could print, say, every 4th line, starting at line 2:

```
sed -n '2~4p' pg_catalog.csv
```

3.8 awk

awk is a program (and a language) that can be used to extract, process, or reformat data. In combination with sed, grep and regular expressions, we have a very powerful set of tools for examining, validating, and/or reformatting data quickly without requiring specialized software (although sometimes we want to use specialized software instead of reinventing our own rickety wheels in custom bash scripts). We'll cover awk superficially here, but see here for the documentation, and here for a shorter tutorial.

Let's use awk to do a little processing of a genome annotation file in GTF format. First download and decompress it:

```
wget https://ftp.ensembl.org/pub/release-111/gtf/fundulus_heteroclitus/Fundulus_heteroclit
gunzip Fundulus_heteroclitus.Fundulus_heteroclitus-3.0.2.111.gtf.gz
```

We'll get into genome annotation files more later, but for now you should know GTF is a tab-delimited format that contains the locations of important genomic features. For each feature, column 1 gives the sequence (often a whole chromosome, sometimes a chromosomal fragment) that the feature is found on, columns 4 and 5 give the start and end positions (1-based, inclusive) of the feature, and column 3 gives the type of feature (exon, CDS, transcript, gene). Features are hierarchical, so coding sequences (CDS) and exons always have a transcript parent feature, and transcripts always have a gene parent feature. In a GTF, all these identifiers are supposed to be provided for each record, though gene annotation file formats are very frequently violated by different pieces of software. The transcript and gene identifiers are found as part of a long semicolon-separated list in column 9.

Let's first use awk in the simplest possible way. We'll print only lines containing gene features. awk automatically parses text files with white space as field separators, so we can access the columns (or fields) with special variables $1, $2, $3... with $0 referring to the entire line. We can match individual fields with a regular expression like this:

```
awk -F "\t" '$3 ~ /transcript/' Fundulus_heteroclitus.Fundulus_heteroclitus-3.0.2.111.gtf
```

Here we're using -F "\t" to tell awk to use only tabs as field separators (to the exclusion of spaces). Piping this whole thing to wc -l we would see this file contains 35,597 transcript records.

We can use logical statements to require multiple matches to pull out transcripts matching a gene identifier:

```
awk  -F "\t" '$3 ~ /transcript/ && $9 ~ /ENSFHEG00000021448/' Fundulus_heteroclitus.Fundul
```

awk has considerably more utility than this, however. I often use it to reformat files.

Another common format used to store information about genomic regions is `BED`. `BED` has only 3 required columns: the sequence identifier, the start position and the end position. The trick here is that whereas GTF is 1-based and fully closed, BED is 0-based and half-open. This means that in a GTF file, an interval of 1-1000 refers to the first 1000 bases, including the start and end point of the interval. In BED format, however, the first base in a sequence is numbered 0, and the end base in an interval is not included as part of the interval, making the BED interval 0-1000. See this blog for a discussion.

If you find this fully closed vs half-open, 0-based vs 1-based stuff irritating, know that it will come up repeatedly, as it has for many decades (see this 40+ year old polemic)

At any rate… we can easily output our GTF intervals in BED format like this:

```
awk -F "\t" '{OFS="\t"}{newstart=$4-1}{print $1,newstart,$5}' Fundulus_heteroclitus.Fundul
```

Here we have grouped commands with `{}` and specified an "output field separator" (`OFS="\t"`) because `awk` will by default write spaces instead of tabs. We told awk to print field 1, our new variable `newstart`, and field 5.

Note that in this case, GTF has a few header lines (beginning with `#`) that we have mangled and included.

We can also summarize our data by doing operations across lines. Let's figure out how much of our genome falls into annotated genes:

```
awk -F "\t" '{if($3 ~ /gene/) n+=$5-$4+1 } END {print n}' Fundulus_heteroclitus.Fundulus_h
```

In this case, we use a conditional `if($3 ~ /gene/)` to say that if field 3 is a gene record, we should add the difference between fields 5 and 4 plus 1 to the variable `n`. `n+=...` is a shorthand for `n=n+...`, so for each subsequent record the value of `n` is incremented rather than replaced by a new value. In `awk`, `END` indicates the following command should be executed when the file has finished processing.

We see here that 474,790,822 bases of our 1 gigabase genome falls inside of gene annotations. To figure out how much is actually exonic is a little more complicated and difficult to achieve with awk alone because each gene may have multiple transcripts, each with their own overlapping exons, though it could be done.

3.9 Linux Commands in This Section

Command	Description
cmnd	desc

3.10 Exercises

3.10.1 Quiz:

This week's quiz is a practical demonstration of the skills from this section. Find it on Xanadu here: `/core/cbc/ISG5311/quizzes/week2quiz.docx`. Transfer it to your local computer, and using Microsoft Word, answer the questions it and upload it to blackboard. <details…>

References

Part III

HPC and SLURM

Part IV

Learning Objectives

Learning Objectives:
Edit files
Use environment variables
Write loops to efficiently do repetitive tasks
Edit files on Xanadu in `nano`
Connect Visual Studio Code to Xanadu to edit files
Connect your desktop to Xanadu's file system to access files
Write and execute scripts
Query SLURM to determine resource availability
Submit work and a resource request to be managed by SLURM
Monitor the progress of work on SLURM
Evaluate the results of work submitted to SLURM
Start an interactive session on a compute node

Resources:

4 Scripts and putting it all together

Learning Objectives:
Edit files
Use environment variables
Write loops to efficiently do repetitive tasks
Edit files on Xanadu in `nano`
Connect Visual Studio Code to Xanadu to edit files
Connect your desktop to Xanadu's file system to access files
Write and execute scripts

We've now gotten some experience navigating the Linux file system, and inspecting and summarizing files. We've seen how we can use pipes to connect simple programs together to quickly get insight into our data. These are key ingredients to building up to analysis of data that you can be confident in. We're now going to introduce some concepts that will help us write **scripts**, which are essentially shell commands organized into a file and executed as a batch, rather than executed interactively.

scripting is not categorically different from *programming*. The terms are generally used for polar ends of of a spectrum of complexity. If you, for example, write bash code to download some data from a database, decompress it, and set permissions, most people would refer to that as a scripting. In contrast, if you write code in c++ to parse thousands of sequence alignments and use complex numerical algorithms to estimate some statistical quantity from them, most people will refer to that as *programming*. In both cases, you are writing instructions for a computer to follow.

In this course we're going to focus mostly on the simpler end of the spectrum: using shell scripts to automate chunks of data analysis performed by other programs written in languages such as **c++**, **python**, or **R**.

4.1 One-Liners

In the previous chapter we introduced the pipe as a way to link the inputs and outputs of multiple shell commands. Short piped commands are sometimes referred to as "one-liners" and you can find lots of useful repositories of them at pages like this and this. A quick skim

through these, especially after we've moved a little further through the course will be super helpful.

4.2 Environment variables

A key feature of BASH is the use of **environment variables**. These are objects used to store text strings. These strings are typically used in 2 ways: to generalize code in shell scripts, or to configure the system.

4.2.1 Invoking variables

Some variables are set automatically (usually those used to configure the system or shell options). To see a list try the command `env`. You should see a long list, but among them is a pretty straightforward variable: `USER`. To invoke it, try any one of the following:

```
echo $USER
echo ${USER}
echo "${USER}"
```

The key thing here is the `$`. The curly brackets and quotes can slightly modify how the variable itself is invoked or understood by the shell (more in a moment), but the `$` is how you let the shell know you're calling out a variable.

Variables can be interpolated into text strings, or commands in a variety of ways. Try this, for example:

```
echo "My username is $USER"
```

4.2.2 Setting variables

Users can also create variables, or modify existing ones. To create a variable holding a path, for example:

```
MYHOMEDIR="/home/FCAM/$USER/"
echo $MYHOMEDIR
```

In this case you can see we've defined a variable using another variable. We can equally well define output files this way:

```
MYOUTFILE="/home/FCAM/$USER/${USER}_testfile.txt"
echo $MYOUTFILE
echo "hello world!" >$MYOUTFILE
cat $MYOUTFILE
```

4.2.3 An aside about quotes in BASH

Now that we're getting a little more into bash, we need to be careful when thinking about quotes and whitespace.

Try to assign the text string "Hello World" without quotes like this:

```
HW=Hello World
echo $HW
```

You should get the error `-bash: World: command not found`. This is because bash tried to assign the string "Hello" to variable "HW", then saw the whitespace and treated "World" as a separate command, which happens not to exist. The entire command failed.

If we wrap the text in double quotes it will work:

```
HW="Hello World"
echo $HW
```

If we wrap the text in single quotes it will work:

```
HW='Hello World'
echo $HW
```

But what's the difference between double and single quotes? Single quotes prevent variable interpolation, leading to *very* different results:

```
HWD="Hello $USER"
echo $HWD
HWS='Hello $USER'
echo $HWS
```

Quoting generally ensures whitespace is maintained as part of a string, which is sometimes very important. Single-quoting prevents variable interpolation (i.e. it treats strings literally) while double-quoting allows it. Tripping over quoting is a common source of errors, even with intermediate level bash users.

4.2.4 The PATH variable

Another key variable you can see in the list produced by `env` is `PATH`. It's a critical variable for configuration. It tells the shell where to look for programs you try to execute. Try

```
echo $PATH
```

You'll see a colon-separated list of paths. Every program you successfully execute will be found in one of those paths. You can add to this list if you want bash to look somewhere new in addition like this:

```
PATH=$PATH:/path/to/my/new/favorite/software
```

That is also generally how you can recursively modify variables, should you need to.

When executing programs it can be really useful to know *which* copy of a program is being executed, as sometimes there will be multiple versions installed on a given system (this is definitely true of Xanadu). You can use the command `which` for this:

```
which ls
which grep
which cat
```

You'll note that each of these is found in a path found in your PATH variable. This will be less trivial and more important to be able to check when it comes to software used for data analysis on Xanadu.

4.2.5 Generalizing code

So far we have been executing commands by explicitly referring to files, programs and their parameters. For example:

```
grep -o -P "(?<=[Ss]pecies )[A-Za-z]+" darwin1859.txt | sort | uniq -c | sort -g | tail
```

In this case, we have specified programs by name, several options, a regular expression, and an input file. We could easily make this line of code more general by replacing important parts of it with variables:

```
REGEX="(?<=[Ss]pecies )[A-Za-z]+"
INFILE=darwin1859.txt

grep -o -P "${REGEX}" $INFILE | sort | uniq -c | sort -g | tail
```

Note that quoting the regular expression variable: `"${REGEX}"` is necessary in this case. If you try without the quotes, some of those special characters will trip bash up, and by extension cause grep to fail.

Using variables like this can be especially useful in two cases:

1. When paths get extremely long and cumbersome and/or file names are not descriptive. Long paths can make code difficult to read. Obscuring them with descriptive variables allows you (and anyone else) to quickly see HOW you ran a program, instead of having to scan over multiple instances of `/the/very/long/and/convoluted/path/to/my/precious/input/data.t`
2. When code must be repeated many times on many different input files. Copying, pasting and editing the same line over and over again is extremely inadvisable as even the most conscientious people will be prone to mistakes. If we get used to using variables like this, we gain the flexibility to use loops and other means of parallelization that make our code less error prone, faster, and prefigure the way workflow languages we will explore in the next semester require us to think about and write code.

4.3 Loops

It's often the case that we want to do something repeatedly. It may be that we want to do some analysis the same way for multiple input files, or do something with every element of a list, or line of a file. Sometimes, though less often in data analysis, we may want to repeat a process until some condition is met.

One common way to approach this is with a **loop**. A loop generally has the following very general syntax:

```
loop condition
  { command1
  command2
  command3 }
```

In BASH we typically use `for` and `while` loops. A `for` loop iterates over a list of fixed length executing the same code each time. A `while` loop executes code repeatedly until some condition is no longer met.

4.3.1 `for` loops.

With `for` loops we can iterate over any sort of list. Let's extract mentions of classic Darwin terms "species", "forms", and "varieties" from the Origin:

```
for WORD in species forms varieties
do
  grep "$WORD" darwin1859.txt >"${WORD}"_mentions.txt
done
```

In this syntax we tell bash we're going to use the variable `WORD`, and for each iteration of the loop, populate it with an element of the list "species forms varieties". The elements are separated by whitespace. The code to be executed for each iteration of the list is sandwiched between the keywords `do` and `done`. We redirect the output to three separate files named "species_mentions.txt", etc.

We can specify the list of elements to be iterated over any number of ways. We can use a range of numbers or letters with a shell expansion:

```
for NUM in {1..10}
do
  echo $NUM
done
```

We can use a glob to iterate over files:

```
for FILE in *mentions.txt
do
  head "$FILE"
done
```

4.3.1.1 A digression about BASH arrays

Sometimes we may wish to construct a list outside of the context of the loop itself. It can be useful to store that list in a special type of variable called a **bash array** (or just "array", but we will deal with other types of arrays later, so we'll try to be specific). bash arrays are lists that require you to use a special syntax to create and access their elements.

bash arrays can be created simply with parentheses like this:

```
DARWIN=(species forms varieties)
```

Or using `ls` or `find`, or `*` to grab files:

```
DARWIN=(*mentions.txt)
```

```
DARWIN=($(ls *mentions.txt))
```

In this last case the syntax `$(command)` is a command substitution. It means run the command and insert the output in place (in this case, inside the `()` used to define the array). Also note that the array elements will be parsed based on whitespace, so if file names contain whitespace (you should *avoid* this always) this approach will break.

bash arrays are zero-indexed, meaning the first element is element `0`. We can access elements with this syntax:

```
echo ${DARWIN[0]}
```

To write out ALL the elements:

```
echo ${DARWIN[@]}
```

To get the length of the array:

```
echo ${#DARWIN[@]}
```

To use this in a loop we can either provide the whole array like this:

```
DARWIN=($(ls *mentions.txt))

for FILE in ${DARWIN[@]}
do
  echo "FIRST TEN LINES OF $FILE --------------------"
  head $FILE
done
```

Or we can iterate over the indexes like this:

```
DARWIN=($(ls *mentions.txt))

for NUM in {0..2}
do
  echo "FIRST TEN LINES OF ${DARWIN[$NUM]} --------------------"
  head  ${DARWIN[$NUM]}
done
```

4.3.2 `while` loops.

`while` loops differ from `for` loops in that they evaluate repeatedly until some condition is no longer satisfied. You can use:

```
COUNTER=1

while [ $COUNTER -le 5 ]
do
    echo "Count: $COUNTER"
    ((COUNTER++))
done
```

Where `[ $COUNTER -le 5 ]` is a conditional construct indicating `COUNTER` must be less than 5. If this evaluates to false, the loop ends. Conditional constructs are a general way to manage when code is executed, and we will cover them more later. `((COUNTER++))` adds one to `COUNTER` every time the loop executes.

You can also use `while` to iterate over lines in a file:

```
FILE=darwin1859.txt
COUNTER=1

while IFS= read -r line ; do
    echo "Processing line $COUNTER: $line"
    ((COUNTER++))
done < $FILE
```

In this case we're using `< $FILE` to direct the standard input to `FILE` and while will read from it. When `FILE` runs out of lines, the loop will break.

4.4 Editing Files

We've covered lots of basic bash features. We're almost ready to start writing scripts. Now that you've been writing lots of commands in the terminal, and you recognize you're connected to a remote computer cluster that your local file system doesn't have access to, you're probably starting to wonder, how do I actually write and save lots of code on the remote server?

In this section we'll deal with that issue and cover some tools you can use to write and edit code, both locally and remotely.

4.4.1 Command-line text editors

The most straightforward approach to this is to use CLI text editors that are already available on Xanadu (and most Linux systems). Since you run them on Xanadu, they can directly create and access files. The most common ones are `nano`, `vim` and `emacs`. `vim` and `emacs` are very powerful editors. They are highly customizable and have large user communities. They have

steep learning curves, however, and there are alternatives to CLI editors, so in this course we are going to focus on `nano`, which is simple to use and suitable for quick edits and copy-paste operations.

If you simply type `nano` on the command line, it will open a new text document. You can immediately begin typing whatever you like. When you're done, you can type `ctrl-x` and it will ask you if you want to save the file. If you do, type `y`, then when prompted, write a file name and hit `enter`. That's it. If you want to edit an existing file, type `nano filename`. Save and exit the same way. `ctrl-c` cancels.

You can navigate around with the arrows on your keyboard (and some keyboard shortcuts we'll talk about in a video). That's it.

4.4.2 Code editors

Another approach suitable for users starting up is to write code using a locally installed, dedicated code editor such as Sublime Text. There are others, but for this course I recommend you download and use Sublime for at least some cases. Unlike `nano`, it has lots of features and user-created plugins. It has **syntax highlighting**, which means if you select the language you are writing in from a dropdown menu in the bottom right corner, it will recognize syntactic features of the language and change the color of the text in ways that greatly help with editing.

A straightforward way to use Sublime to write scripts is to write code locally, and then paste it into documents using `nano`, or use `scp`/`rsync` to upload the documents to Xanadu. These solutions are not exactly ideal, but they work in a pinch. It is possible to connect Sublime through an ssh tunnel to give it access to documents on Xanadu, but we haven't yet shown you the tools to do that, and we'll introduce another, easier approach in a moment.

4.4.3 Integrated development environments.

IDEs are dedicated code editors with *lots* more features. Visual Studio Code is another program we recommend you download and install locally. It also has syntax highlighting, but you'll notice you can open up projects and see the entire directory structure, and if you want you can even open a terminal inside it.

4.4.4 Conveniently accessing files.

Ok, so aside from using `nano` (or learning to be a `vim` power user), how can you conveniently access and edit files on Xanadu? There are two relatively straightforward ways:

1. Visual Studio Code has an extension Remote - SSH. This is **highly** recommended. You can connect VS Code to Xanadu directly via SSH. You can open windows focusing on specific directories, visualize the directory structure, and create and edit files directly on Xanadu. You can also open a Xanadu terminal window so that you can test and execute code, all in VS Code.

> ⚠ Warning
>
> Please don't use language-specific code extensions (python, R, etc) to run code on Xanadu from within VS Code. We'll cover this in the next chapter, but VS Code connects to a **login** node, which has few resources and is shared by many users. To run code on Xanadu, it must be submitted through the batch scheduler, `SLURM`. A VS Code extension that runs code for you can only run it on the login node, which will cause problems for everyone.

2. You can mount the Xanadu file system on your local computer. You can get your operating system to "see" Xanadu's filesystem, and access files as if they were local. You can then edit them using Sublime, VS Code, or anything else. To do this, you **must** be connected to the CAM VPN (instructions).

- From a mac, you can select from your top dropdown menu `Go:Connect to Server` and enter `smb://cfs09.cam.uchc.edu/home/FCAM/<username>`. When prompted, enter your CAM credentials (not your netID/password).

- From Windows, you can map a network filesystem using these directions and the address formatted like this: `\\cfs09.cam.uchc.edu\home\FCAM\<username>`

> ⚠ Warning
>
> You should only use dedicated software for editing code (or CLI editors on Xanadu). Editing using word processors (such as MS Word) will often lead to difficult to diagnose problems. Word processors often insert hidden characters and/or use line break characters incompatible with Linux. These will cause confusing errors when executing code. The first place to start if you think this might be the issue is to try `cat -A myscript.sh`. If `myscript.sh` contains any weird hidden characters, this will print them. Compare to a script you know works. Non-standard line breaks are often the culprit and this will reveal them.

4.5 Scripts

Ok, now we've got all the pieces, we can start writing longer bits of code into **scripts** so that we can run them as batches instead of interactively, line-by-line. Scripts can be as simple as a series of commands that you *could* execute interactively, or as complicated as computer programs that run entire analyses for you.

In this course, we'll aim for the simpler end of the spectrum. We want to write code in chunks that we can pass off to Xanadu's job scheduler and that will serve as the most fundamental documentation of the analyses we do. While keeping notes on what you do (or try) is important, the code itself is the ultimate documentation of your analysis, so it should be complete and well organized into scripts.

4.5.1 The shebang

A script can be written in any language, so typically the way we start one off in a Linux environment is to tell bash what interpreter to use. Right now we're writing bash code (rather than R, python, perl or something else), so we need to tell bash that. To do that, the first line of the script will always be what's referred to as a **shebang**, or `#!` followed by the interpreter we wish to use, in this case `/bin/bash` so:

```
#!/bin/bash
```

If we were writing python code we might write:

```
#!/bin/env python
```

This would tell the shell to use whichever python interpreter was found by searching our `PATH` variable.

After the shebang line, we can start writing code.

4.5.2 Comment lines

Despite our best efforts, code can sometimes be complicated and confusing to read. Because of this, it's really important to document your code with **comments**. In shell scripting, you can include lines prefixed with `#`, that will be ignored by the interpreter. You can and should write notes on these lines explaining what the code does.

```
#!/bin/bash

```

```
# this script will print the 10 most common words found in a text file and their frequenci

# this line specifies the text file
ORIGIN=darwin1859.txt

# this line extracts and prints the word list
grep -o -P "\b[A-Za-z]{4,}\b" $ORIGIN | sort | uniq -c | sort -g | tail -n 10
```

Create this script with the title `commonWords.sh` on Xanadu and change the permission so that you can execute it.

4.5.3 Executing scripts

There are a few ways to execute scripts.

1. `source commonWords.sh` This method *ignores* the shebang (it begins with `#` after all) and executes all the code in the current shell session as if you had typed it on the command line. If you created any variables in the current environment they will be available in the script.
2. `bash commonWords.sh` This method also ignores the shebang and explicitly invokes bash, but it creates a *subshell*. The execution context is mostly isolated from the environment you submitted the script, and environment variables you create are not available unless you export them.
3. `./commonWords.sh` This will execute the script, and looks for the shebang to see which intepreter should be used. It also creates an isolated execution context.
4. Passing the script to a job scheduler (in our case SLURM). This will be covered in then next chapter.

What does it mean to have an isolated environment? Modify the `commonWords.sh` script so that the variable definition is *commented out* and thus ignored by the interpreter:

```
#!/bin/bash

# this script will print the 10 most common words found in a text file and their frequenci

# this line specifies the text file
# ORIGIN=darwin1859.txt

# this line extracts and prints the word list
grep -o -P "\b[A-Za-z]{4,}\b" $ORIGIN | sort | uniq -c | sort -g | tail -n 10
```

If you try to execute it, you will find that it hangs indefinitely because the ORIGIN variable is empty.

If you try to define the ORIGIN variable and then run the script it will work with the `source` method:

```
ORIGIN=darwin1859.txt
source commonWords.sh
```

But for it to work with the other two methods, you need to `export` the variable:

```
export ORIGIN=darwin1859.txt
./commonWords.sh
```

Exporting the variable makes it so that it will be inherited by any processes spawned by the current shell.

Another big difference between `source` and the other two methods is that any variables created by the script will be available in the current shell session after the script has completed.

4.5.3.1 Command-line arguments

Note that we can generalize our script by writing it so that it expects command-line input from the user:

```
#!/bin/bash

# this script will print the 10 most common words found in a text file and their frequenci

# this line specifies the text file
ORIGIN="$1"
REGEX="$2"

# this line extracts and prints the word list
grep -o -P "$REGEX" $ORIGIN | sort | uniq -c | sort -g | tail -n 10
```

Save this script to the file `test.sh` and make it executable. We can now run it like this:

```
./test.sh darwin1859.txt "\b[A-Za-z]{4,}\b"
```

The variables `$1` and `$2` are automatically parsed as whitespace separated arguments provided on the command line when executing the script. There are more complex ways to provide and parse command-line arguments, but we won't cover them here.

4.5.4 Errors and exit codes

Code can fail for many reasons. You will inevitably make mistakes in command line usage for some program, refer to paths that don't exist, or use incorrect syntax, and an error will result. Computer clusters are complex, and they also inevitably have problems. Sometimes a compute node will crash. Sometimes a user will accidentally get around whatever guardrails the system administrators have in place and muck up some important settings. Troubleshooting these errors can be a major challenge for beginners.

Here are a few common errors you are likely to encounter:

"No such file or directory"

```
cat /path/does/not/exist.txt
```

Leads to `ls: cannot access /path/does/not/exist.txt: No such file or directory`.

Check your path/filename.

"Permission denied"

```
echo "I'm going to put a file in the root directory of the cluster!" >/file.txt
```

Leads to `bash: /file.txt: Permission denied`

Check that your path is correct and that you actually *should* have permission. If you own or share the file or the directory, fix the permissions. If someone else owns it, ask them to fix the permissions.

bad quoting

```
grep endless forms darwin1859.txt
```

Leads to search results for "endless" but also `grep: forms: No such file or directory`. This is a quoting issue. `forms` is treated as another file to be searched because of the whitespace. It should be `grep "endless forms" darwin1859.txt`

"Command not found"

```
gep "endless forms" darwin1859.txt
```

Leads to `bash: gep: command not found`. Classic typo. Command not found can also be a sign of incorrect quoting, or the dreaded incorrect line breaks that arise from editing code in non-code-specific editors.

When a command is evaluated, bash produces an **exit code** and stores it in the special variable `$?`. If the command executes without error, the exit code will be 0. If there is an error it

will be greater than 0. With scripts set up as we have above, the exit code will refer to the *last command* in the script. So if there are two commands in a script, and the second one completes successfully, but the first fails, the exit code will still be 0. By default the script wont just quit when it hits an error. There are settings that can change the behavior of bash when it encounters errors, but there is some controversy about whether you should use them or not.

"There is some controversy" is often the case with bash scripting. BASH is a wonky language, and it takes *years* to truly become an expert in it. When learning to do data analysis, we need to do our best, be vigilant about errors, and be open to learning new ways of doing things when we discover problems with our established habits. Pages like this are great for intermediate bash users to consider, but we can't wait to analyze our data until we've assimilated all these best practices.

4.6 Exercises

5 SLURM: the job scheduler

Learning Objectives:
Query SLURM to determine resource availability
Submit work and a resource request to be managed by SLURM
Monitor the progress of work on SLURM
Evaluate the results of work submitted to SLURM
Start an interactive session on a compute node

5.1 High performance computing review

Now that we've covered connecting to a remote server, basic command-line interface usage, and the fundamentals of writing a script, we're ready to explore how to actually use the Xanadu computer cluster. You should be familiar with this already from ISG5301, but Xanadu is not a single computer. It is a large number of computers networked together, most of which are far more powerful than a standard consumer-grade laptop or desktop. These individual computers are often referred to as **nodes**. Each of the nodes is connected to a **network-attached file system** (NFS) which stores user data. Because all the nodes are connected to this file system, no matter which of the nodes you may be working on at any given moment, you will have access to your data.

There are three main categories of nodes.

1. **Compute nodes**: These are the workhorses of the cluster. They typically have many CPUs (24-96) and memory ranging from 256G to 2TB (even a good consumer laptop won't usually have more than 8 CPUs 16G of memory).
2. **Login nodes**: These are much smaller computers, often with only 2 CPUs andn 8G of memory. They may not even be physical machines, but virtual ones. They are meant to serve as portals to more powerful resources. When users connect to Xanandu, they are assigned to one of several login nodes.
3. **The head node**: A head node is typically used only by system administrators to manage the system. It probably runs the workload manager.

Thus far, when we have connected to Xanadu we have connected to a **login** node. Because login nodes have few resources, which are shared among many users, **you should not use them**

for analysis. The kind of light work we have done (navigating the file system, inspecting files) is suitable for the login node, but if you ever run the kind of command that has you thinking, "I'll just check social media for for a moment while this runs" then you have **far exceeded** what you should be doing on a login node.

5.2 SLURM

In order to request suitable resources for data analysis, we need to appeal to software that we will variously refer to as the **workload manager** or **job scheduler**. On Xanadu, we use the software `SLURM`. It is very commonly used on HPCs, but there others such as `PBS` and `LSF`. From a user perspective, these systems have many features in common, and in fact, Xanadu is set up to interpret `PBS` commands if necessary, though this is not recommended.

In this chapter we will cover how to use `SLURM` to ask what resources are available, request resources to do work, monitor the status of running jobs, and evaluate jobs when they have completed.

5.2.1 The general approach

When you have to do some computational work that requires cluster resources (and you will very soon), the process looks something like this:

1. Decide what resources are needed to do the work.

2. Check to see whether the resources are available (whether they exist all, or are currently busy).
3. Submit the work with a resource request to the job scheduler (or simply request resources in the case of interactive work).
4. Monitor job progress until completion or failure (or if interactive work, do the work).
5. Evaluate the work.
6. (Possibly) Go back to (1) for the next step of your analysis.

We'll cover each of these below.

5.2.2 What resources are needed?

We will cover this question on a job-by-job basis when we start analyzing data as it can be somewhat complicated.

5.2.3 What resources exist/are available?

The two primary SLURM commands used here are `sinfo` and `squeue`. We can also look at the file `/etc/slurm/slurm.conf` to see a list of node *features* that we can request if necessary.

5.2.3.1 `sinfo`

`sinfo` prints information about available compute nodes and their status. At the moment of writing, running `sinfo` with no options prints the following output:

```
PARTITION  AVAIL  TIMELIMIT  NODES  STATE NODELIST
general*      up   infinite      1  drain xanadu-05
general*      up   infinite     23    mix xanadu-[01,03-04,08,10,25,39,50-52,57-61,64-66,69-7
general*      up   infinite      4  alloc xanadu-[02,62-63,67]
general*      up   infinite      5   idle xanadu-[46-47,49,53-54]
vcell         up   infinite      4  drain xanadu-[78-81]
vcell         up   infinite      3   idle xanadu-[76-77,82]
vcellpu       up   infinite      1   idle xanadu-32
himem         up   infinite      2    mix xanadu-[40,44]
himem         up   infinite      2   idle xanadu-[06,43]
himem2        up   infinite      2    mix xanadu-[07,75]
xeon          up   infinite      1  drain xanadu-05
xeon          up   infinite     18    mix xanadu-[03-04,08,39,50-52,57-61,64-66,69-70,72]
xeon          up   infinite      4  alloc xanadu-[02,62-63,67]
xeon          up   infinite      5   idle xanadu-[46-47,49,53-54]
amd           up   infinite      2    mix xanadu-[10,25]
mcbstudent    up   infinite      2    mix xanadu-[68,71]
gpu           up   infinite      1  drain xanadu-05
gpu           up   infinite      5    mix xanadu-[01,03-04,07-08]
gpu           up   infinite      1  alloc xanadu-02
gpu           up   infinite      3   idle xanadu-[06,84-85]
crbm          up   infinite      2   idle xanadu-[55-56]
```

Nodes on clusters are sometimes, though not always divided into **partitions**. If they are, you must decide which partition you want to submit to. Partitions need not be mutually exclusive, and may have different behaviors and limitations. Key partitions on Xanadu are `general` and `himem`. In the basic `sinfo` output, you see nodes in each partition, divided into various categories, and a listing of the nodes in each category: **alloc** = allocated, i.e. all resources on these nodes have been assigned; **mix** = some resources on these nodes have been assigned and some remain available; **idle** = these nodes are currently unused; **drain** = these nodes are not accepting new jobs (probably they will be rebooted when current jobs finish). Xanadu

actually has many more nodes than this, at the time of writing many had been removed for maintenance.

You will frequently want to see more detail than this. The following command will print information for each node, formatted according to the obscure syntax found in the `sinfo` man page.

```
sinfo --format="%10P %6t %15O %15C %15F %10m %10e %15n %30E %10u"
```

The first few lines of output:

```
PARTITION  STATE  CPU_LOAD        CPUS(A/I/O/T)   NODES(A/I/O/T)  MEMORY     FREE_MEM   HOSTL
general*   drain  0.01            0/0/36/36       0/0/1/1         225612     16760      xanad
general*   mix    18.37           28/8/0/36       1/0/0/1         257669     9569       xanad
general*   mix    11.49           35/1/0/36       1/0/0/1         257845     45859      xanad
general*   mix    8.34            34/2/0/36       1/0/0/1         257845     1307       xanad
general*   mix    2.41            9/27/0/36       1/0/0/1         257669     50949      xanad
general*   mix    10.76           15/33/0/48      1/0/0/1         386972     52520      xanad
general*   mix    12.61           44/4/0/48       1/0/0/1         257949     7961       xanad
general*   mix    6.03            8/8/0/16        1/0/0/1         128825     26234      xanad
general*   mix    0.06            8/32/0/40       1/0/0/1         257914     119938     xanad
general*   mix    0.08            8/32/0/40       1/0/0/1         192032     13437      xanad
```

Here you get lots of useful information. The column `CPUS(A/I/O/T)` tells you how many cpus are **allocated/idle/other/total** on each node. You can also see the amount of memory and free memory (in megabytes) on each node. This can give you a pretty fine-grained sense of what resources currently exist in terms of CPU and memory, and how many resources are **idle** on the cluster.

5.2.3.2 `squeue`

The `squeue` command lets users see what jobs have been submitted to the job queue, what their status is, and why. This includes pending jobs that are waiting for resources to become available, and currently running jobs. Depending on SLURM configuration, this command may show *only* jobs you have submitted, or it may show *every* job. At the time of writing, Xanadu was configured to show *every* job.

Some example output from `squeue`:

```
  JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
7949897   general nf-MAIN_ ebrannan PD       0:00      1 (QOSMaxMemoryPerUser)
7949952   general nf-MAIN_ vvuruput PD       0:00      1 (Resources)
```

```
7949954   general nf-MAIN_ vvuruput PD       0:00      1 (Priority)
7953478   general slow59.s mhossein PD       0:00      1 (Priority)
7953479   general slow60.s mhossein PD       0:00      1 (Priority)
7946600   general    busco shillima  R 2-20:56:08      1 xanadu-52
7945409   general     bash    shird  R 3-01:21:06      1 xanadu-74
7917445     himem R_divers pmartine  R 6-01:31:02      1 xanadu-40
7953261 mcbstuden     bash meds5420  R    3:46:07      1 xanadu-68
```

The JOBID is a unique numerical identifier assigned by SLURM to *every* job it runs. This is important information we'll discuss later. NAME is the name the user gave SLURM for the job (truncated in this view). USER is user who submitted the job. ST is the status: PD = pending; R = running. NODELIST(REASON) is either the list of compute nodes the job was assigned to (e.g. xanadu-52 for job 7946600) or the reason the job is not yet running: "Priority" essentially means the job will start at any moment; "Resources" means the requested resources are busy; "QOSMaxMemoryPerUser" means the running the job would exceed the user's maximum memory allotment. Individual users have resource limits so that no one person can dominate the system.

You can use the flag `-u <user>` to restrict to a given userid. You can try `squeue -o "%.12i %.9P %.30j %.8u %.2t %.10M %.6D %R"` for a little more spacious formatting.

5.2.3.3 `/etc/slurm/slurm.conf`

SLURM has a configuration file `/etc/slurm/slurm.conf` that lists features of each node. Some of this is listed by `sinfo`, but occasionally you may run into software that has very specific needs. Something may require a particular *instruction set* be present on the CPUs, or you may require a particular type of GPU be present on the node. These **features** are listed at the end of this file, and these features can be requested using **feature constraints**. Try `cat /etc/slurm/slurm.conf`. Look near the end of the file at the section beginning `# COMPUTE NODES`:

```
# COMPUTE NODES
NodeName=xanadu-01 CPUs=36 SocketsPerBoard=2 CoresPerSocket=18 ThreadsPerCore=1 RealMemory=25
NodeName=xanadu-02 CPUs=36 SocketsPerBoard=2 CoresPerSocket=18 ThreadsPerCore=1 RealMemory=25
NodeName=xanadu-03 CPUs=36 SocketsPerBoard=2 CoresPerSocket=18 ThreadsPerCore=1 RealMemory=25
NodeName=xanadu-04 CPUs=36 SocketsPerBoard=2 CoresPerSocket=18 ThreadsPerCore=1 RealMemory=25
```

Here you can see that node xanadu-01 has 36 CPUs, ~256G of memory, a Xeon cpu and an NVIDIA A10 GPU.

5.2.4 Request resources (and submit work)

So far we have shown you ways to ask questions about the cluster and its resources. Now we'll cover how to request them to get actual work done. There are two common use cases for analyzing data using cluster resources: running batch scripts that require no active user intervention, and doing interactive analysis.

5.2.4.1 Running batch jobs with `sbatch`

When you run a batch job, you write a script that will run all the steps of a given analysis for you, hand that script to SLURM, and wait for it to complete (or fail). Using one command (or one script) you tell SLURM which resources you need, and what you want it to do. The command we use for this is `sbatch`.

Running `sbatch` is in essence, just like running a script as we did in the previous chapter, except that instead of the script being run by the current shell, you hand it to SLURM with your resource request, SLURM puts it in the job queue, and when a node (or nodes) with enough resources becomes available, assigns the job there and runs it for you.

Let's look at the most basic usage using our fully defined script from the previous chapter:

```
#!/bin/bash

# this script will print the 10 most common words found in a text file and their frequenci

# this line specifies the text file
ORIGIN=darwin1859.txt

# this line extracts and prints the word list
grep -o -P "\b[A-Za-z]{4,}\b" $ORIGIN | sort | uniq -c | sort -g | tail -n 10
```

Save it to `commonWords.sh`. Note that because we are passing this file to SLURM, not directly to bash, we *definitely* need the shebang at the top.

On Xanadu, you must minimally specify the partition you want SLURM to run the job on, and its associated quality of service (or QOS). Not all SLURM clusters require a QOS to be specified, but Xanadu does.

To submit the script to be run on the general partition you can do:

```
sbatch -p general --qos=general commonWords.sh
```

Our script writes to stdout. Where did the results go? By default, to a file named `slurm-JOBID.out`. In my case at the time of writing `slurm-7953521.out`. You can inspect this file and the results should be there.

This basic approach will request a minimal amount of memory and number of CPUs which will be too little for most real data analysis. We can request more with some options:

```
sbatch -p general --qos=general -c 12 --mem=10G commonWords.sh
```

Now we're asking for 12 CPUs with `-c` and 10 gigabytes of memory with `--mem=10G`. Far more than is needed for this tiny job. What happens if you request more memory than is available on general partition?

```
sbatch -p general --qos=general -c 12 --mem=1000G commonWords.sh
```

An error!

```
sbatch: earror: Memory specification can not be satisfied
sbatch: error: Batch job submission failed: Requested node configuration is not available
```

This is relatively straightforward. But where does stderr go? Can we direct that to a file? Can we name the jobs so we can see them in the queue? Or rename these slurm-JOBID files so that they are a little easier to sort through when we run are running lots of jobs?

The answer is yes, but you can imagine our command-line is going to start getting very long and cumbersome. To solve this, we typically specify SLURM options in a **header** at the top of our script.

Let's put one in our script:

```
#!/bin/bash
#SBATCH --job-name=commonWords
#SBATCH -n 1
#SBATCH -N 1
#SBATCH -c 10
#SBATCH --mem=20G
#SBATCH --partition=general
#SBATCH --qos=general
#SBATCH --mail-user=MY_EMAIL@uconn.edu
#SBATCH --mail-type=ALL
#SBATCH -o %x_%j.out
#SBATCH -e %x_%j.err

hostname
```

```
date

# this script will print the 10 most common words found in a text file and their frequenci

# this line specifies the text file
ORIGIN=darwin1859.txt

# this line extracts and prints the word list
grep -o -P "\b[A-Za-z]{4,}\b" $ORIGIN | sort | uniq -c | sort -g | tail -n 10
```

Any command-line option we can pass to `sbatch` can also be placed in a header that immediately follows the shebang. Each header line begins with `#SBATCH` and a command-line flag follows. If we need to, we can override any of these header lines with new values on the command-line, without altering the script.

Here we name the job `commonWords`, request 10 cpus and 20G of memory. The mail options are optional, but if you provide an e-mail, then SLURM will let you know when your job starts and ends. The last two options specify the file name format for any output written to stdout or stderr. The format is `jobname_jobid{.out,.err}`

We also specify a few other options that aren't necessary and that we rarely change: `-n 1`, saying that we want SLURM to launch 1 task, and `-N 1` saying that we want the resources for the task to be on a single node. These will suit all of our work in this course.

You may also notice we added two extra lines to the top of the script `hostname` and `date`. These will write out which compute node the job ran on, and the date it began to the stdout. This can be helpful information if you need to seek help when troubleshooting. Sometimes individual nodes on the cluster, rather than user errors, can the source of problems.

Update `commonWords.sh` with this header and run it again. Now you can simply enter:

```
sbatch commonWords.sh
```

Now, instead of seeing a single file `slurm-<JOBID>.out`, you should see a pair of files: `commonWords_<JOBID>.out` and `commonWords_<JOBID>.err`. The `.out` file should contain the output from our script, along with the results of `hostname` and `date`, and `.err` should be empty (unless there were any errors).

So, to sum up we request resources and submit work to be run on the cluster by SLURM. Putting SLURM options in the script header is extremely useful, as it keeps a record of *how* we ran our code, not just what code we ran.

5.2.4.2 Starting interactive sessions with `srun`.

It is sometimes the case that we want to do analysis on the cluster that is more intensive than what is permissible on the login nodes, but we are unable to write all the steps into a batch script. This might be an exploratory analysis, where we don't know what the steps are yet, or we might be putting together a complicated set of piped commands and we want to test that the pipe is doing what we expect before we run the entire job.

For this we use an **interactive session**. In an interactive session, we request resources on a compute node, and SLURM drops us into a bash session on that node, rather than a login node. On the compute node we no longer have to worry about gumming things up for everyone else. SLURM will not let us exceed our CPU request, and if we run a program that attempts to exceed the memory request, the session will simply be canceled.

We use the `srun` command for this. The syntax for requesting an interactive session is:

```
srun -p general --qos=general -c 2 --mem=10G --pty bash
```

Try it. To ensure that you have successfully started an interactive session, type `hostname` at the prompt. You should see the name of a compute node that can be found in `sinfo`, something like `xanadu-01`, rather than a login host name like `hpc-ext-1`.

You can exit back to the login node with `exit`

5.2.5 Monitor job progress

There are several strategies for monitoring job progress. We will talk about them here, and practice them later when we start doing longer-running jobs.

1. Check `squeue` to see if your job is pending, currently running, or has completed (or failed) and no longer in the queue.
2. Monitor output files. Check the stdout and stderr log files captured by SLURM. Many programs write progress messages or errors to these files. Check the output files themselves as well.
3. Log in to the compute node and run `top` to see how/if your process is running.

5.2.5.1 `squeue` again

When the cluster is busy and jobs are large enough that they wait in the queue for a while before resources are available, you can use squeue as we did above to see what their status is. If they are running, you'll also be able to see how long they've been running. Jobs that complete too quickly or too slowly are sometimes a sign that something isn't right (or that your expectations aren't right).

5.2.5.2 Monitor output files

You can monitor the `.out` and `.err` files, and output or log files produced by your script or the program you're running. In all of these cases, the first thing you can do is simply use `ls -l` and look at the time stamps. Are any of these files being updated?

Depending on the program or the nature of your script, the `.err`, or `.out` files are going to be key files to check. These are updated in real time. If there are errors, warnings, or progress messages, they are likely to be in one of these two files. Some programs will write helpful, succinct messages, and some will write an endless alphabet soup that makes it hard to distinguish normal progress from problems. It will all depend on the program.

When it comes to program- or script-specific files, you'll have to understand a little about the output you're expecting. Some programs write no output until they're complete, others write out results nearly as fast as they read data in. In the latter case, be sure to check on these files at least once and ensure that they're growing as the program is running.

5.2.5.3 Log in to your compute node

There are lots of cases where you might want to get a more direct look at how your job is going. SLURM doesn't have good tools for this. The solution here is to actually log directly in to the compute node and have a look. First, check which node your job is running on using `squeue`. Then you have two options to get into the node.

First, you can use `srun` and request the node with, e.g. `-w xanadu-01`:

```
srun -w xanadu-01 -p general --qos=general -c 1 --mem=500G --pty bash
```

Here we request minimal resources so that we are more likely to quickly get the session started.

Second, if the node is fully subscribed and you can't get an interactive session, you can still get in from the login node with:

```
ssh xanadu-01
```

If you go this route, you should not do **anything** else other than check on your job. If you couldn't get an interactive session on your node, then all CPU or all memory has been requested, and you are essentially oversubscribing the node by logging in this way. You should check on your job quickly and type `exit` to log out.

With either method, once you're on the compute node, you can use the program `top` to see running processes. You will see ALL running processes on the node in a constantly updating list, including user and system processes. To see your own processes type `u` and then your username followed by `enter`. To sort processes by memory usage, type `shift-m` and by

processor usage `shift-p`. `%CPU` refers to the percentage of a single CPU, so if you requested 10 cpus, you may see up to %1000 CPU usage. `%MEM` refers to the percentage of the total memory on the node. The `S` column indicates whether your process is running (R) or sleeping (S). In an ideal world, whatever you're running would be using all the CPU and all the memory you requested. In reality this is rarely the case. Resource usage often fluctuates. If you see that your processes are often sleeping, that can be a sign they aren't running efficiently and that something may be wrong. If you see that processes you are running never seem to be using as much CPU or memory as you requested, it may be that you requested too much, or that you forgot to tell the actual program you're running how many CPUs to use, for example.

Type `q` to quit `top`.

5.2.6 Evaluate the work

When the job is no longer in the queue, it has completed, failed, or been canceled. You first need to figure out which this is! The two main approaches are to use the slurm commands `seff` and/or `sacct`, and to look at the same output files you looked at when monitoring the job's progress.

5.2.6.1 `seff` and `sacct`

`seff` is the simplest approach. Simply type `seff <JOBID>` at the command line. You will get a report like this from our `commonWords.sh` job:

```
Job ID: 7953646
Cluster: xanadu
User/Group: nreid/cbc
State: COMPLETED (exit code 0)
Nodes: 1
Cores per node: 10
CPU Utilized: 00:00:01
CPU Efficiency: 10.00% of 00:00:10 core-walltime
Job Wall-clock time: 00:00:01
Memory Utilized: 1.56 MB
Memory Efficiency: 0.01% of 20.00 GB
```

Some of this is self-explanatory at this point. `CPU Utilized` is the sum of the CPU-hours used by the job. `Wall-clock time` means the actual time span over which the job ran. `CPU Efficiency` is `CPU Utilized` / (`cores per node` * `wall-clock time`), so the average rate of CPU usage. `Memory Efficiency` is the **peak** memory usage divided by the memory requested. This job was very simple and used virtually none of the resources we requested. This is *not*

what you generally want to see, but because our job only took 1 second, it's not that big of a deal. If every job on the cluster had efficiencies ranging from 1-10%, it would be a pretty big waste.

`sacct` is pretty large and complicated SLURM function that can extract all kinds of job information. We won't get into it too much here except to say this command can provide some detail on a job:

```
sacct -o jobid%-11,jobname%30,nodelist%15,user%12,group%15,partition,state,ReqMem,MaxRSS,R
```

For this job we get this output:

```
      JobID                        JobName        NodeList         User           Group  Part
----------- ------------------------------ --------------- ------------ --------------- -----
7953646                        commonWords       xanadu-25        nreid             cbc    g
```

5.2.6.2 Checking output files

The approach is basically the same as above. Check your log files for errors, warnings and progress messages. Check the program outputs to see that they are what you expect. That last bit requires some understanding of what you're trying to do.

5.2.7 What resources *should* I request?

This is a perennially challenging problem. In much of bioinformatics, software developers do not, or perhaps cannot, give general advice about what resources their program will need. There are so many axes of variation for different datasets: experimental design, species, tissue, sample size and more. For a given algorithm, these may drastically impact resource needs, or not impact them at all.

When analyzing a new dataset, doing a new type of analysis, or using a new piece of software, we generally advise people to expect to have to experiment a bit. We will talk about resource requests for different pieces of software as we move through the course.

That said, we have some general guidelines:

1. Check the software documentation. It's quite possible there is a critical variable that defines how much memory or CPU the program requires, or can utilize, and that the developer has actually explained it for you quite clearly.
2. When you run the job for the first time, check the CPU and memory efficiency. See if the job failed because it ran out of memory usually `oom-kill` is somewhere in the `.err` file. Request more memory if so. Request fewer resources if efficiency was low.

3. Remember that when requesting CPUs, you **almost always** have to tell the actual program you are running how many CPUs are available to it. If you tell SLURM you want 10 CPUs, you usually have to provide an option (sometimes `-p` or `-t` or `-c`) telling the program how many it can use.
4. Try to tune your resource requests to your actual analyses. If you just copy-paste the same SLURM header requesting 24 CPUs and 100G of memory for every job, sometimes it won't be enough, and most of the time it will be way too much. Either way wastes resources for everyone, and your time. Bigger jobs often sit longer in the queue. Rerunning failed jobs is a huge pain.

The UConn Computational Biology Core has a document you can reference about resource requests (much of which is covered here) here.

5.3 Exercises

6 Data Storage and Software on HPC

Learning Objectives:
Distinguish types of data storage available on Xanadu
Compile simple software from source
Install software in an isolated environment using `conda`
Run software inside a Singularity container

We have two more important topics to cover before we move on to actually starting to analyze some real data. How we manage data storage and software on Xanadu.

6.1 Data storage and access

This is a complicated topic, and it's hard to generalize in a way that will apply across organizations, or between HPC and cloud platforms. In general with big 'omic data sets, data storage can grow expensive and unwieldy over time. At many (perhaps most?) organizations, data storage for multi-user systems is regularly in flux, as technologies, cost, and the preferences of users and system administrators evolve. Storage systems may come very close to full before funding and approval come through for expansion, or addition of a new system. Cost models may shift, necessitating changes in user behavior or system policies. These issues can crop up rapidly, even in well run organizations with good communication. Because of this it will pay dividends for users to keep their data organized-to know exactly what they have and where-so they can respond nimbly to requests by system administrators or principal investigators.

What does keeping data organized mean? First we'll talk about storage on Xanadu, then we'll talk about organization. In this section, "data" will refer to both raw data and the products of analysis, which can both be quite large in many cases. The term "large" is quite vague, but here we'll refer to data on the scale of 10s of gigabytes to 1-2 terabytes, the kind of data that would be likely to challenge a typical consumer-grade computer's storage. This is the range that most typical 'omic experiments fall in. There are certainly cases where researchers wish to analyze data from tens or hundreds of experiments, or massive datasets produced by large consortia (e.g. containing genome sequencing data from hundreds of thousands of individuals), but that sort of analysis requires specialized strategies and won't be covered in this course.

6.1.1 Types of storage on Xanadu

Storage space falls into four main categories on Xanadu:

1. Space allocated to specific users/groups.
2. Shared temporary space.
3. Shared storage of active raw data and other resources
4. Archival space

We'll talk about each in turn.

6.1.1.1 User/group-allocated space

`/home/FCAM/<username>`: All new users on receive a home directory with 1-2TB of storage. This is located in `/home/FCAM/<username>` on file system `cfs09`. You can see this size of this file system and how much space remains by `df -h`. This storage can **only** be accessed by the user. On Xanadu, attempts to open home directory permissions to share files with other users are a security risk and will be met with a stern lecture from a system administrator.

`/labs`: Principal investigators or collaborative research groups can also be assigned a space on file system `cfs09` in the directory `/labs`. These directories typically start at 5-10TB and can be expanded if necessary. Each of these directories is assigned to a Linux user group and accessible to that group. Typically this will be members of the PI's lab, or everyone in a collaborative research project.

`cfs09` is a network-attached file system. Anything on it will be accessible from any node on the cluster.

6.1.1.2 Shared temporary space

Many analyses have large short-term storage needs. Sometimes raw data are duplicated 3-4 times during successive processing steps (it's best to avoid doing this where possible though!) but the duplicates can be quickly deleted when the analysis completes. In those cases it doesn't make sense to permanently increase storage allocations to users to accommodate the temporary data. Instead we have shared spaces.

`/scratch`: This directory is on the NFS `cfs08` is the primary shared temporary storage location. Again, with `df -h` you can see how large it is and how much space is available. Any user can create any directory, or series of directories, and store anything they want there. You should definitely use `/scratch` for temporary data storage, but you should never keep anything there you can't afford to lose. Files that haven't been used for 90 days are subject to deletion. That may seem like a lot, but it's really not. Whole chunks of a final project for this course could disappear within a semester if they were stored there.

`/tmp`: This is another shared temporary storage location. `/tmp` is a place lots of software is configured to try to write temporary files to by default. On Xanadu `/tmp` is **local** to nodes. This means whatever is stored there during an analysis is **only** available to that node. `/tmp` is also small on Xanadu and can fill up quickly. Software my quietly try to write there and fail with the error `No space left on device`. If you think this is the problem, you can often either tell the software to use a different directory with a command-line option, or set the environment variable `TMPDIR` and export it like this:

```
export TMPDIR="/scratch/$USER"
mkdir -p $TMPDIR # in case it doesn't already exist
```

6.1.1.3 Other shared storage

`/isg/shared`: This location is where we keep globally installed software packages and commonly used databases. Users have read access to all of it, but for the most part do not have write access. This is on `cfs09`.

`/seqdata`: This directory houses raw data only. Mainly that produced by UConn's sequencing core. It's on the NFS `cfs15`. Only core staff have write access.

6.1.1.4 Archival space

`/archive`: This space is meant for cold storage of raw data and/or finished analyses. Users and groups can offload finished projects or raw data that is no longer needed for active analyses here to free up space for on-going work. It is not meant to be read from or written to with high frequency.

6.1.2 How to manage your data:

We will return to this again when we talk about project organization, but we have some general principles:

1. Keep your raw data separate from analysis outputs. Set the raw data permissions to read-only.
2. Keep your home directory organized. Each project deserves its own directory. You might consider a single directory to house all raw datasets.
3. Storage is limited. Be mindful to structure analyses to minimize the creation of large intermediate products (e.g. using pipes where possible), and delete them when they are no longer useful.
4. Periodically assess how much storage you're using (`du -sh *` in your home directory is a good start).

5. If your home directory is filling up, consider directing intermediate analysis products to `/scratch`. Though be sure that code and final analysis results are not kept there.
6. Keep data compressed where possible. Nearly all raw genomic data and many other types of genomic files can be read and written in gzip compressed format.

6.2 Software

Software is another complicated topic in bioinformatics. There is an ever-increasing variety of 'omic data types and experiments, and software for dealing with them proliferates at an even greater rate. This software is often developed by researchers or statisticians, rather than professional software developers. These folks often do software development as an ancillary part of their role (or on an entirely voluntary basis), and their capacity for polishing a program to perfection, or providing user-support may be limited. Because of this, norms of good software development are not always heeded. This can result in idiosyncratic software that can be difficult to install, confusing to run, and prone to cryptic error messages.

Even in the best case scenario, however, where important software is written and maintained with the support of expert software engineers who are paid for their efforts, the ecosystem of scientific software is becoming incredibly complex, with new software being built in a modular way out of pre-existing pieces. This often leads to a complex web of dependencies that can be challenging to resolve.

In this course we are aiming to teach you some fundamental ways for obtaining and using diverse pieces of software. The strategies we suggest will probably get you to a point where you can access 95-99% of what you need pretty smoothly. The reality, unfortunately, is that last 1-5% can be a hugely frustrating time vacuum, and you will need to get used to going through a process of trying to install something yourself, hitting a wall, and seeking help.

Below, we'll cover some types of software you might use, how to run them, and some strategies for software installation.

6.2.1 Software types

Broadly speaking, when doing analysis we work with two types of software: software with command-line interfaces (CLI), and software with graphical user interfaces (GUI).

Almost all powerful computational analysis is built around software with a CLI. This is because you need to be able to script an analysis and hand it off to be processed by a job scheduler on an HPC or in the cloud. There are, however, cases where a GUI is extremely helpful. These are things like integrated development environments (RStudio, VSCode) or software for data exploration (Integrated Genomics Viewer, a.k.a. IGV, which we'll cover later).

If a piece of software has a GUI, it has usually been professionally developed. Installing it on your local machine will probably be pretty smooth. Getting it to run on the cluster, or access data stored on the cluster, however, is where things get sticky. We talked about this a bit earlier, when we covered connecting VS Code to Xanadu, and mounting the Xanadu file system locally, but we will cover this more later.

CLI software ranges widely in quality and complexity, and managing it is what we'll cover below.

6.2.2 Running software

To review: we have been running CLI software all through this course so far (`grep`, `sed`, `awk`, etc). When we invoke these pieces of software by name, BASH searches directories in our `PATH` variable to see if they exist. Software does not *have* to be in your path to run it though, you can also simply invoke it by name (with a path pointing to it). For example, type this on the command-line:

```
/isg/shared/apps/samtools/1.16.1/bin/samtools
```

This should print the usage for the program `samtools`. If you wanted to be able to just type `samtool` to execute the program, you could append the directory to your path like this:

```
export PATH="$PATH":/isg/shared/apps/samtools/1.16.1/bin/
```

Or if you didn't want to add anything to your `PATH`, you could create a variable and invoke it that way:

```
SAMTOOLS=/isg/shared/apps/samtools/1.16.1/bin/samtools

$SAMTOOLS
```

> **Note**
>
> Running a piece of software with no options, or with `-h` or `--help` will in most cases print helpful information about how to run the software (the "usage"). Even when you're familiar with software, it will probably be difficult to remember the exact flags you need, or how to invoke them for. Checking the usage can be much quicker than referring to the documentation.

6.2.3 The module system

`samtools` is installed for all users of Xanadu. If you have a look at the enveloping directory `/isg/shared/apps/`, you will see more than 700 software packages that have been installed on Xanadu. Within a given software directory, there are subdirectories named by after the software version number, so in reality there are many more than 700 packages. We keep older software versions around so that when users build up their analyses, they don't get the rug pulled out from under them if a piece of software changes its option flags or default behaviors when it is updated (this happen regularly).

While having software on your PATH is super convenient, you can imagine that adding each software/version directory (plus any subdirectories containing essential accessory utilities) to the PATH variable would be extremely cumbersome, not to mention that having `samtools/1.16.1/bin/samtools` and `/samtools/1.9/bin/samtools` means that simply invoking `samtools` is ambiguous.

For this reason, we use a `module` system. The module system allows users to **load** specific software versions they want to use in an analysis. At its simplest, the module system simply adds the software to your PATH variable. If any other environment variables need to be configured, the module system will set those up too.

Try loading `samtools` version 1.16.1 like this:

```
module load samtools/1.16.1
```

Then type `samtools`. You should get the usage. Type `echo $PATH` and you should see that the path to the samtools executable file should eb added to your `PATH` variable.

You can unload the module with

```
module unload samtools/1.16.1
```

Or to get rid of all modules:

```
module purge
```

To see what changes a module makes to your environment you can do:

```
module display samtools/1.16.1
```

Or look directly at the module file:

```
cat /isg/shared/modulefiles/samtools/1.16.1
```

To see a more complex module, look at this old version of the transcriptome assembly software Trinity:

```
module display trinity/2.8.5
```

In this module, many other software dependencies are also `module load`-ed.

You can list all available modules with:

```
module avail
```

You can add any bit of text to narrow the search like this:

```
module avail sam
```

You can load as many modules as you like simultaneously, but be aware that some may have conflicting dependencies or environmental variables, and this may cause problems.

Note that while you *can* invoke modules without version numbers like this:

```
module load samtools
```

you *should not*. This version of the module system sorts the module versions alphanumerically and loads the last one. In reality, version "1.16.1" is *higher* than "1.9", but "1.9" will sort last and thus be loaded. If you do `ls -l /isg/shared/apps/samtools/` you'll note that version 1.9 was installed in *2019*. This document is being written in *2024*. Also, if `samtools/1.91` ever was installed, any script loading samtools without a version number would suddenly and silently switch samtools versions. Lastly, if you use modules in your scripts, writing the version number will help you be explicit and keep track of which versions you want to run.

For Xanadu, you can request that new software modules be globally installed by filling out a form here.

6.2.4 Compiling software

While you can request that software be installed for you on Xanadu (and other clusters as well), there may be a long queue, or you may not even be sure you're going to use the software yet and so installing a module would be premature. In those cases you can try installing it yourself. For some programs, you may be able to download a **binary** executable file that will work on most or all Linux distributions. In such cases, you only have to do something like this (for the variant caller freebayes):

```
wget https://github.com/freebayes/freebayes/releases/download/v1.3.6/freebayes-1.3.6-linux
gunzip freebayes-1.3.6-linux-amd64-static.gz
chmod ugo+x freebayes-1.3.6-linux-amd64-static.gz

./freebayes-1.3.6-linux-amd64-static --help
```

In many cases an executable will not distributed, but the developer will provide instructions for compiling the software from the source code, creating the executable. Compilation simply means to convert the human-readable code as written by the developer into machine-code. Programming languages requiring compilation are typically faster than those where the human-readable code is interpreted on the fly.

Let's look at the case of the demographic history estimation software PSMC. The software is on `github`, a site for hosting version-controlled software repositories (we'll get into lots of detail on this next semester).

We can get the source code, which is written in the language `C`, by visiting the github repository clicking on the green `code` button and copying the URL, then on Xanadu:

```
git clone https://github.com/lh3/psmc
```

Then we have, per the readme, about the simplest instructions you can have:

```
To compile the binaries, you may run

    make; (cd utils; make)
```

So we do:

```
cd psmc
make
cd utils
make
```

We may see some warnings, but if compilation was successful we should have an executable `psmc` in the base directory, and a few more in the `utils` directory. We can do `./psmc` to see the usage.

So what did we do here? `make` is a piece of software for automating things, mostly used for compilation, but also sometimes in data analysis (the concept has been expanded into a python-based workflow language `snakemake`). The author of this software provided a file `Makefile`, containing instructions for the compilation of `psmc`. When we ran `make`, it automatically looked for `Makefile` and followed the instructions.

Compiling software can get more complicated than this sometimes. If you look at `Makefile` you'll see the variable `CC=gcc`. `gcc` is a compiler for `C`. The version installed on Xanadu is *very old*. Some software will require a more recent version (which you can `module load`). Other software will require different build tools (such as `CMake`). Many pieces of software will require dependencies to be installed already, and expect to find them in a specific place.

The takeaway here is that if you look at a code repository and it contains some relatively straightforward instructions for compilation, it's probably worth a shot. Even without any real understanding of computer science, you can often get things working with a minimum of frustration or tinkering.

6.2.5 Conda

Some pieces of software will not be so simple, however. They may require particular dependencies that are not already installed. These dependencies may conflict with other existing dependencies. Webs of dependencies can be incredibly complex, and require very specific software versions. In these cases, manual installation can become a demoralizing grind, and even when successful, these types of installations can be easily broken inadvertently by a software update.

For cases like these, we use the very popular package manager `conda`. `conda` has a two main features. First, it manages the creation of mostly isolated software environments into which you can install one or more pieces of software and their dependencies. This prevents conflicts with existing software installed on the system. If we think back to the `trinity` module we inspected above, all those specific pieces of software required by that version of trinity could be installed inside the conda environment.

The second main feature of conda is thatits dependency solving algorithm. There are multiple repositories, including anaconda, conda-forge, and bioconda that house "recipes" for software installations. These recipes list required software versions. If you ask conda to install one of these recipes in an environment, it will figure out the web of dependencies that are required to make it work on your system, collect them, and install them. This works remarkably well in many cases.

`conda` manages packages at the user level, so each user creates their own conda packages (though these can be shared). Xanadu doesn't have "global" conda environments as we do with modules.

6.2.5.1 Installing miniconda

To use `conda`, we're going to install a version of it called `miniconda` in your home directory. The instructions are here, but we'll also reproduce them below:

Download and run the installer.

```
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
bash Miniconda3-latest-Linux-x86_64.sh
```

Follow the prompts and let it install in your home directory and automatically initialize conda in your shell.

Then, to stop conda from opening a base environment automatically:

```
source .bashrc
conda config --set auto_activate_base false
```

Let's also configure some of those repositories we discussed above:

```
conda config --add channels defaults
conda config --add channels bioconda
conda config --add channels conda-forge
conda config --set channel_priority strict
```

And finally, to install a faster dependency solver than the conda default:

```
conda install -n base conda-libmamba-solver
conda config --set solver libmamba
```

Now exit your bash session / log out of Xanadu and log back in.

Ok, so what have we done here? We've created a few files and directories. To see them type `ls -la`. Files beginning with `.` are hidden by default. The `-a` flag in `ls` shows them.

```
-rw-r--r--  1 usr domain users  577 May  1 15:25 .bashrc
drwxr-xr-x  2 usr domain users  512 May  1 15:19 .conda
-rw-r--r--  1 usr domain users   26 May  1 15:27 .condarc
drwxr-xr-x 19 usr domain users  10K May  1 15:19 miniconda3
```

The directory `miniconda3` will contain all the environments we create. `.conda` and `.condarc` are a conda cache directory and a configuration file respectively. We don't need to edit `.condarc` by hand, just using the command `conda config`. `.bashrc` is a bash configuration file. If you `cat` that out, you'll see the conda initialization code This sets up some environment variables and puts `conda` in your `PATH` variable. Every time you log in `.bashrc` is `source`-ed by the shell. Any changes you wish to be made to your shell environment can be specified here. We'll revisit the `.bashrc` file again in the future.

When you submit scripts via SLURM, you start a non-interactive shell and `.bashrc` is not `source`-ed. For now, you will have to add `source ~/.bashrc` to any scripts you want to use a conda environment in.

6.2.5.2 Using miniconda

Ok, so we've installed it, now how do we actually use conda? Let's install a newer version of samtools than the one we discussed above. If you check the link above in the Files tab you'll get a sense of which versions are available. The syntax is pretty flexible, but let's be as explicit as possible about the steps:

First create a new environment:

```
conda create -n samtools-1.20
```

Then activate the environment:

```
conda activate samtools-1.20
```

You should see your prompt change, indicating the environment is active. Then install samtools:

```
conda install samtools=1.20
```

You can also create an environment and install software in one step:

```
conda create -n samtools-1.20 samtool=1.20
```

If the environment is active, you should be able to type `samtools` and get the usage. Type:

```
which samtools
```

And you should get something like `~/miniconda3/envs/samtools-1.20/bin/samtools`. And in your `PATH` variable you should see something like `/home/FCAM/<username>/miniconda3/envs/samtools-1.20`

You can deactivate the environment with:

```
conda deactivate
```

And now your PATH variable should be returned to its previous state and `which samtools` should give you an error, or point to the module we loaded previously (if you are somehow still in the same session). If you activate a conda environment while another is already activated, it will first be deactivated.

A few more basic conda commands:

To list your environments (when you have a lot you won't remember them all!):

```
conda env list
```

To remove one

```
conda env remove -n <NAME>
```

To create a `.yml` file defining the environment:

```
conda export -n samtools-1.20 >samtools_env.yml
```

If you give that file to someone else, they can recreate your environment with:

```
conda env create -f >samtools_env.yml
```

For more, the conda user guide has very clear and accessible documentation.

6.2.6 Software containers

For our last topic in this section, we're going to introduce **software containers**. We have discussed strategies for installing and using software of increasing complexity (complexity not necessarily of the applications of the software, but of its structure). In extreme cases, package managers installing in isolated environments sometimes still can't quite get you where you want to be. This may be because there are fundamental features of the operating system you're working on that make installing software challenging, or perhaps those features cause slight inconsistencies in the outputs of software across systems, or maybe available conda recipes seem broken (this does happen sometimes). Package managers may also be a challenge when you are running a pipeline that requires dozens of pieces of software in particular versions to be installed. In these cases, you may want to try using a containerized approach.

A software container is very nearly a virtual machine. They differ from VMs in that they still use the core operating system kernel, instead of completely emulating it. This means they are "lightweight" and carry a much lower performance burden than VMs. Because they virtualize the operating system and user interface though, they provide a greater level of isolation and consistency among systems than an environment managed by conda.

I've framed this in terms of easing the burden of software installation, but containers have the added advantage of improving reproducibility and portability of analyses. This development has been critical for the development and distribution of complex software pipelines like those in the nf-core repository. It is very nearly a requirement that those pipelines be run using container systems.

In this section we're going to discuss *using existing software containers*. In the next semester we will cover building them.

There are two main pieces of containerization software: `Docker` and `Singularity`. They both have generally similar features and performance. Docker, however, requires **root access** to

run containers. Root access is access to the most basic functions of a computer system, and it cannot be granted to users on HPC systems, as it represents a massive security risk and an operational vulnerability (a well meaning but inexperienced user doing something wrong as the root user could take down the whole cluster). Because cloud services use virtualized machines, they can offer users root access, so Docker is popular in that context. Singularity, however, does not require that level of access to run a container, making it much more compatible with HPC systems. Fortunately, software containers built using Docker can be converted to Singularity (though not vice versa), so being limited to using Singularity on an HPC is not a huge burden.

6.2.6.1 Obtaining a singularity container

Let's explore what it practically means to use Singularity. Let's start an interactive session (you should *not* use singularity on login nodes):

```
srun -p general --qos=general -c 2 --mem=20G --pty bash
```

And then load a singularity module:

```
module load singularity/vcell-3.10.0
```

Now we'll use the command `singularity pull` to retrieve a container for a very commonly used QC program for high throughput sequence data `fastqc`:

```
singularity pull https://depot.galaxyproject.org/singularity/fastqc:0.12.1--hdfd78af_0
```

If this was successful, you should see a new file `fastqc:0.12.1--hdfd78af_0` in your current working directory.

6.2.6.2 Using a singularity container

There are two main ways we can use the container.

1. We can start up a shell and use it interactively.
2. We can pass singularity a command along with the container and it will execute it inside the container.

A container is *almost* a self-contained computer. To be useful, however, it needs to access the local file system. On Xanadu, by default, some local system paths will be made available inside it. It will bind your home directory, `/labs`, and `/isg/shared` among others. However, local directories like `/usr/bin` which contain lots of commonly used programs, will be masked by the directory with the same name *inside* the container.

To see how this works, let's check a few things on the local system first. Type

```
fastqc --help
```

You should get a `command not found` error.

Type:

```
cat /etc/os-release
```

You should see that Xanadu is running the `CentOS` Linux distribution.

Lastly type:

```
ls /usr/bin
```

You should see a *very* long list of programs.

Now let's try our first method of using the container: starting up a shell (it will usually be BASH):

```
singularity shell fastqc:0.12.1--hdfd78af_0
```

Your prompt should change to `Singularity>`.

Try each of the above commands again. You should see that `fastqc` is now in your PATH and prints its usage. You will also notice that the linux distribution inside the container is `Debian`, and that a different, much shorter list of programs is found in `/usr/bin`. If you type `which fastqc` you'll see that's where it's been installed.

If you `ls` your home directory, you'll see that your home is available inside the container.

Type `exit` to exit the container.

The second way we can use the container is by passing a command with `singularity exec`. To keep it simple, we'll do `fastqc --help`:

```
singularity exec fastqc:0.12.1--hdfd78af_0 fastqc --help
```

It should print the usage. Keep in the back of your mind that things can get a little bit wonky when quoting and shell variables are necessary when passing a command to a container.

6.3 Exercises

Exercises for this section consist of practicing each of these skills. We're going to ask you to compile software, install it using conda, and pull a singularity container, and to write a slurm script to print the usage for each of the programs we direct you to.

Part V

Sequencing Data

Part VI

Learning Objectives

Learning Objectives:
Explain common sequence data formats FASTA and FASTQ
Manipulate sequence data in FASTA and FASTQ format
Distinguish between data produced by the three major sequencing platforms
Perform standard quality control analyses on sequence data

7 High-throughput sequencing data

Learning Objectives:
Explain common sequence data formats FASTA and FASTQ
Manipulate sequence data in fasta and fastq format

7.1 Sequence data

Now that we've covered the basics of Linux, BASH, SLURM and HPC topics in general, we are going to start looking at the data type that will be the main focus of this certificate: nucleotide sequence data (and to a lesser extent, amino acid sequence data). In this chapter we're going to cover the most basic formats for storing it and some ways to manipulate it. In the next chapter we'll go over the basic characteristics of three major platforms for producing it.

7.2 File formats

Sequence data are relatively straightforward. Nucleotide sequence data are typically stored as character strings consisting primarily of A (adenine), G (guanine), C (cytosine), or T (thymine). Even when we sequence RNA, we typically represent the uracil's as T instead of U. There are cases where sequences are stored with ambiguities, in which case IUPAC ambiguity codes are used (e.g. "R" for "G or A", or "N" for any nucleotide). Amino acid sequence data are similarly stored as character strings using single-letter abbreviations, also per IUPAC.

7.2.1 `FASTA`

In the case of both nucleotide and amino acid sequences, the most basic file format is known as `FASTA`. It is pronounced variously as "fast-ay", "fast-uh", or if maybe if you're from the UK "fahstuh".

FASTA format is very simple. A sequence record consists of a header line, beginning with a `>` and immediately followed by a sequence name, and one or more following lines containing a sequence. This can be as simple as:

```
>Sequence1
aaccctaaaccctaaaccctaaaccctaaaccctaaaccctaaaccctaaaccctaaaccctaaaccctaaaccctaaaaccctaa
```

A sequence can also be broken up across multiple lines, which can be helpful for very long sequences like chromosomes.

```
>Sequence1
aaccctaaaccctaaaccctaaaccctaaaccctaaaccctaaaccctaa
accctaaaccctaaaccctaaaccctaaaccctaaaaccctaaaccctaa
```

Sequence header lines can contain information besides the name:

```
>CM042378.1 Thymallus thymallus isolate DD20220222a ecotype Europe chromosome 1A, whole genom
cctaaccctaaccctaaccctaaccctaaccctaaccctaaccctaaccctaaccctaaccctaaccctaaccctaaccc
taaccctaaccctaaccctaaccctaaccctaaccctaaccctaaccctaaccctaaccctaaccctaaccctaacccta
accctaaccctaaccctaaccctaaccctaaccctaaccctaaccctaaccctaaccctaaccctaaccctaaccctaac
cctaaccctaaccctaaccctaaccctaaccctaaccctaaccctaaccctaaccctaaccctaaccctaaccctaaccc
```

In this case, the name `CM042378.1` is an accession number for the sequence at NCBI. Everything starting with the first space is extra information and is **usually ignored** by programs that accept FASTA sequences. Only the text after `>` and before the first space should be considered the sequence name.

FASTA files can contain multiple sequences like so:

```
>101020_0:000000
MALLRLPYLFSSSTPNQSSLITCQTHPFSSVRTKKREVSRLYASSSGGKSDPYLMDSEERREWRDKIRQVLDKNPDLEEELDP
>101020_0:000001
MCPRNADINILKLKAFPFSLEDRAKSWLIDLPTGHIDSWEKMMSEFLTKYFPASKITVMRKQITGIQQGPEETFCAYYERFKALVATCPSHGL
>101020_0:000002
MEACNCIEPQWPADELLIKYQYISDFFIAIAYFSIPLELIYFVKKSAVFPYRWVL
```

You may notice that in above sequences we have a mix of upper and lower-case letters. These do not change the meaning of the sequence itself, but are a form of sequence **annotation**. In nucleotide sequences, we typically use lower-case letters to indicate that sequences have been identified by some algorithm as **repetitive** or **low-complexity**. This is referred to as "soft-masking" because some algorithms will take this as a signal to ignore the bases, without our having to change them to `N` (referred to as "hard-masking").

Because telomeric sequences at chromosome ends are typically highly repetitive, when a whole chromosome has been assembled and annotated, a large fraction of the first few kilobases is often soft-masked.

FASTA format contains only sequences. Because of this, it's only really suitable for sequences we are pretty confident in. That is to say, we believe that each of the bases or amino acids in the sequence has been correctly determined. In the case of high-throughput sequencing data, however, it's not always the case that each base is known with a high degree of confidence. That leads us to our second file format…

7.2.2 FASTQ

FASTQ format is essentially FASTA with base qualities. The base qualities carry information about the certainty that a given base has been determined correctly. This makes FASTQ a more suitable format for raw sequence data from high-throughput sequencers, which have error rates high enough that they need to be accounted for in some analyses. A sequence record in FASTQ has 4 lines:

1. A header line starting with @ (instead of >) immediately followed by a sequence identifier.
2. A sequence (usually a nucleotide sequence). Unlike FASTA, all on a single line.
3. A line beginning with +, typically followed either by nothing, or the sequence name repeated.
4. A line containing ASCII-character-encoded, PHRED-scaled base qualities.

A single FASTQ record looks like this, but as with FASTA, many records can be put in a single file:

```
@A01010:43:H2YNYDSXY:1:1101:1108:1000 1:N:0:GCGTATTAAT+TGCGGTGTTG
NGTCACTCTATGAGTTTTTACTTCCTCCTTATCAACAGTGCCGGCCACGTTTGTTAAAAATGTTTATTTCTTAACTTGGGATTGTTTCTCCA
+
#FFF:FFFFFFFFFF::FFFFFFFFFFFFFFFF:FFFFFFFFFFFFFFFFFFFFFFFFFFFF:FFFFFFFF:FFFFFFFFFFFFFFFFFFF
```

This sequence was generated by an Illumina sequencer (more on that later). The sequence identifier contains lots of information about the sequencing run, but we rarely, if ever, need to parse that ourselves, so we won't break it down here.

The next two lines are pretty straightforward. The final line requires some explanation. To repeat, it contains *ASCII-character-encoded, Phred-scaled base qualities.*

ASCII-encoded refers to the fact that each ASCII character in the string codes a quality score. The standard translation for data produced from the early 2010s on is referred to as Sanger encoding, which means that quality scores from 0-93 are encoded by characters 33-126 in the ASCII table. For an easy translation see this website and enter the quality string above.

The quality scores themselves are Phred-scaled. They have a log-scaled relationship with the probability that a base has been called in error. The quality score **Q** is related to the error probability **P** by the following equations:

$$Q = -10log_{10}P$$
$$P = 10^{-Q/10}$$

So:

Phred Quality Score	Probability of incorrect base call	Base call accuracy
10	1 in 10	90%
20	1 in 100	99%
30	1 in 1000	99.9%
40	1 in 10,000	99.99%
50	1 in 100,000	99.999%
60	1 in 1,000,000	99.9999%

Phred-scaled error probabilities crop up repeatedly in genomics, so it's a good idea to get a handle on them now.

7.3 Manipulating `FASTA` and `FASTQ`

In this section we're going to introduce some approaches for manipulating fasta and fastq files. We'll start simple, with tools we have already seen, and then introduce some pieces of software that will be faster and suitable for more complex tasks.

Let's start by downloading two versions of the human genome: GRCh38, which has been in use for many years (with occasional patches and updates), but has some unfinished gaps, and CHM13, a telomere-to-telomere assembly of a human genome. We're going to get them from NCBI, a US government organization that houses a staggering amount of sequence data, both raw output from sequencers, and finished genome and gene sequences. We're going to use a tool developed by NCBI to download data called `datasets`. If you visit this page and this page you'll see that they provide you with a datasets command to run that will download several files for each genome (we'll modify it slightly to get a gtf annotation in addition to gff2):

```
module load datasets/16.13.0
# T2T
datasets download genome accession GCF_009914755.1 --include gtf,gff3,rna,cds,protein,geno
unzip -d T2T T2T.zip
# GRCh38
datasets download genome accession GCF_000001405.40 --include gtf,gff3,rna,cds,protein,gen
unzip -d GRCh38 GRCh38.zip
```

```
rm GRCh38.zip T2T.zip
```

This is a bit of a mess, so let's move things around a little:

```
mkdir genomes
mv T2T/ncbi_dataset/data/GCF_009914755.1 genomes/
mv GRCh38/ncbi_dataset/data/GCF_000001405.40 genomes/
```

Try

```
ls -lh genomes/*
```

You should see that each directory contains a number of files:

- `*genomic.fna`: the genome sequence in fasta format.
- `genomic.gff`: a gff3 formatted annotation file
- `genomic.gtf`: a gtf formatted annotation file
- `cd_from_genomic.fna`: coding sequences in fasta format as inferred from the gff file
- `rna.fna`: RNA sequences in fasta format
- `protein.faa`: protein sequences in fasta format

7.3.1 Basic Linux tools

Let's explore the genome sequences a bit with some commands we already know:

How many sequences are in each genome fasta? The regular expression `^>" matches only lines that begin with`>', which is every header line, and no sequence lines.

```
grep -c "^>" genomes/GCF_000001405.40/GCF_000001405.40_GRCh38.p14_genomic.fna
grep -c "^>" genomes/GCF_009914755.1/GCF_009914755.1_T2T-CHM13v2.0_genomic.fna
```

There are so many sequences GRCh38 because it has some alternate haplotypes and unplaced sequences. T2T does not have those. Why are there 24 sequences in the T2T genome when there are only 23 pairs of chromosomes in the human genome?

We can't count fastq records this way because "@" and "+" can both be present in the quality string.

7.3.2 bioawk

Using standard Linux command-line tools to manipulate files in this way is fairly limited (though we did do it a little more in depth with our RADseq exercise).

We can take things a little further with `bioawk`, a modification of `awk` to deal with genomic data formats that adds a few nice functions. `bioawk` is suitable for quick data exploration and validation tasks.

Let's look at the length and GC composition of sequences in our genomes:

```
module load bioawk/1.0

bioawk -c fastx '{print $name, gc($seq), length($seq), $comment}' genomes/GCF_000001405.40
bioawk -c fastx '{print $name, gc($seq),length($seq), $comment}' genomes/GCF_009914755.1/G
```

`bioawk` has built in functions, like `gc()` and built-in variables like `$seq`, `$name`, and `$comment` that allow you to access different parts of a sequence record. `-c fastx` indicates that we're giving it a fasta or fastq file.

GRCh38 has some alternate haplotypes and unplaced sequences. Why are there 24 sequences in the T2T genome when there are only 23 pairs of chromosomes in the human genome?

As in `awk` you can use `BEGIN` and `END` blocks to execute code before and after processing a file. To get the mean GC content in the entire T2T genome:

```
bioawk -c fastx '{tl+=length($seq); gcl+=length($seq) * gc($seq)}END{print gcl / tl}' geno
```

With fastq files we can do something similar (here we pass the stdout to `head` because there are millions of sequences in that fastq file):

```
FQ=/core/cbc/tutorials/workshopdirs/RNA-seq-with-reference-genome-and-annotation/01_raw_da

bioawk -c fastx '{print $name, gc($seq),length($seq), meanqual($qual)}' $FQ | head -n 100
```

Note that for fastq files, bioawk has a `$qual` variable for the quality scores.

In principle, you could use bioawk and Linux tools piped together to do all kinds of manipulation and filtering. We usually just use them for quick inspection and summarization though, as we have faster purpose-built tools we can use for large datasets.

7.3.3 seqkit

seqkit is one such fast, purpose-built tool for manipulating fastx files. It has tons of features, and skimming through the documentation is worth your time. It may take a minute, because at the time of writing, `seqkit` had 38 subcommands to accomplish various tasks. We'll look at just a couple here.

> **Note**
>
> Note that the model of developing a "toolkit" containing many individual tools (invoked commonly with the syntax `<toolkit> <tool> --options` is a common one in bioinformatics (see also samtools, bcftools, vcftools, bedtools, gatk, kat, and others). In this course we'll typically only be working through a handful of tools in any given toolkit, but the above advice usually holds, that it can be well worth your time to acquaint yourself with other features by skimming the documentation.

7.3.3.1 Summarizing files

We can start out by calculating some basic stats on all of our fasta files:

```
module load seqkit/2.8.1

seqkit stats genomes/GCF_00*/*f{n,a}a
```

Which will print this output table:

```
file                                                                format  type     num_seqs
genomes/GCF_000001405.40/cds_from_genomic.fna                       FASTA   DNA       145,346
genomes/GCF_000001405.40/GCF_000001405.40_GRCh38.p14_genomic.fna    FASTA   DNA           705
genomes/GCF_000001405.40/rna.fna                                    FASTA   DNA       185,121
genomes/GCF_009914755.1/cds_from_genomic.fna                        FASTA   DNA       130,323
genomes/GCF_009914755.1/GCF_009914755.1_T2T-CHM13v2.0_genomic.fna   FASTA   DNA            24
genomes/GCF_009914755.1/rna.fna                                     FASTA   DNA       178,809
genomes/GCF_000001405.40/protein.faa                                FASTA   Protein   136,194
genomes/GCF_009914755.1/protein.faa                                 FASTA   Protein   129,662
```

7.3.3.2 Extracting sequences

Let's extract every exon from the T2T genome using the GTF file:

```
GENOME=genomes/GCF_009914755.1/GCF_009914755.1_T2T-CHM13v2.0_genomic.fna
GTF=genomes/GCF_009914755.1/genomic.gtf
EXONS=genomes/GCF_009914755.1/exons.fna

seqkit subseq --gtf ${GTF} --feature exon ${GENOME} >${EXONS}
```

In this case we're using the GTF file, restricted only to exon features (column 3), to extract all exonic sequence from the genome into a fasta file.

The first two records:

```
>NC_060931.1_59220-59296:- LOC124906657
agatttttataaaatgaaagctgCCTCTGAAGCACTGCAGACTCAGCTGAGCACCGatac
aaagaaagacaaacatc
>NC_060931.1_42869-42973:- LOC124906657
TAACAATTTGGACAAGACAGCAAATGCTATTGTCCAAGTTTTCTAAAGAAGAATCTGAAG
TGAAATGACATCAAGAGACCTATCAAGACCTGTATCCAGGAAAAG
```

In this case, `seqkit` has given a sequence identifier corresponding to the location and strand of the feature in the format `SEQUENCE_START-STOP:STRAND`, and added the contents of the `gene_id` tag from the feature following a space. In this case an NCBI gene identifier `LOC124906657`.

We can also extract arbitrary sequence ranges from fastx files. First we need to **index** the file we want to extract sequence from. Indexes play a big role in 'omics. Just like in a book, an index allows us to rapidly skip to a particular region of interest without scanning the entire volume. Indexes can be content based (as in books), as we'll see when we get to sequencing mapping, or they can be positional (more like a table of contents). Here we'll make a positional index.

```
GENOME=genomes/GCF_009914755.1/GCF_009914755.1_T2T-CHM13v2.0_genomic.fna

seqkit faidx $GENOME
```

This creates the file `GCF_009914755.1_T2T-CHM13v2.0_genomic.fna.fai`. It has a pretty simple format. Five tab-separated columns giving the sequence ID, how long it is, where it starts in the file, how many bases are on each line, and the length of each line in bytes.

```
NC_060925.1 248387328   87  80  81
NC_060926.1 242696752   251492344   80  81
NC_060927.1 201105948   497222893   80  81
...
```

To extract after indexing:

```
GENOME=genomes/GCF_009914755.1/GCF_009914755.1_T2T-CHM13v2.0_genomic.fna

seqkit faidx $GENOME NC_060927.1:10000000-10000200
```

Produces:

```
>NC_060927.1:10000000-10000200
GAGCTGCTGTCCTTTAAATCCACATTGGGGATGAAGGTCCCTCAGGTGTCAATACTGGAT
GTCCCTCCACCCAATGCCAGTTTGTACCCCCAGAAAGAGGGCCCCTGTGGGTTCTGCTTG
AGCTCCCCAAGGGCCCATCAGAGTCTGTCCCTCCACCTTGCTGCTGTGCCCCCTCTGGAA
GTGCTGTCCTGTGAGAGGGCA
```

You can also extract sequences using a list of sequence identifiers, or by matching identifiers (or by matching the sequences themselves) with a regular expression. Our RNA file has gene names in the comment section of the header. We can search those like this:

```
RNA=genomes/GCF_009914755.1/rna.fna

seqkit grep -n -r -p ".ryl .ydrocarbon" $RNA
```

This will return sequences with headers like:

```
>NM_001033054.3 Homo sapiens aryl hydrocarbon receptor interacting protein like 1 (AIPL1), t
>XM_054336619.1 PREDICTED: Homo sapiens aryl hydrocarbon receptor nuclear translocator (ARNT
```

There are many, many more things you can do with seqkit, but we're going to move on to another tool.

7.3.4 vsearch

What is the gc content of exons (rounded to the nearest 10th of a percent)

8 Sequencing platforms

Learning Objectives:
Distinguish between data produced by the three major sequencing platforms

In this chapter we cover three major sources of high-throughput sequencing data. Each has different characteristics and hence different applications. We will cover these basic characteristics before moving on to quality control analyses in the next chapter. There is innovation and competition in this space, so things may change.

A few terms:

- **sequence read** or **sequence** or **read**: a nucleotide sequence produced by a DNA sequencer.
- **paired-end sequencing**: the practice of determining short nucleotide sequences from each end of a single DNA fragment.
- **sequence library**: a set of DNA fragments that have been prepared for sequencing.
- **flow cell**: for most platforms, a single consumable sequencing unit.

8.1 Illumina

Illumina is a company whose name is virtually synonymous with its sequencing format. Data produced by Illumina has the following characteristics:

- Reads are short. Typically 100-300bp.
- Reads are usually paired end, though single-end reads are possible.
- Reads are accurate. Mean raw sequence quality of around Q35 (0.03% error rate).
- Throughput is high. The smaller MiSeq instrument can produce 15Gb of data per flow cell. The NovaSeq X can produce 8Tb (these stats subject to constant change).
- Sequencing is done by synthesis. This means many mostly identical copies of a single molecular are created to generate the signal that is read by the instrument.
- Per-base cost is low.

Illumina is the workhorse of high-throughput sequencing. Any application that requires lots of sequence data will use Illumina. Expression profiling (as we are about to do) and small variant genotyping are classic applications of Illumina. Repetitive and low-complexity genomic regions common to many organisms make it difficult to use Illumina data for genome or transcriptome assembly, to detect structural variants accurately, or to detect small variants in such regions.

Illumina sequence is typically paired end, which means any given fragment of DNA is sequenced from either end, yielding two sequence reads. These "forward" and "reverse" reads are typically delivered in two separate files `sample_R1.fastq.gz` and `sample_R2.fastq.gz`. Mate pairs are in the same order in each file, so you should never do anything to disrupt that order. All commonly used tools for filtering or trimming Illumina data are aware of this.

DNA fragments put on the sequencer vary in length, but unless you are sequencing short RNAs, they are typically 200-600bp long. Many fragments are longer than the sum of the two read lengths, so for many fragments, a segment in the middle remains unsequenced, while for others, there is a region of overlap between the two reads. For some short fragments, the read length is longer than the fragment, so sequence reads will pass through the template fragment and into sequencing adapter. This is referred to as "read-through" adapter contamination and it should be trimmed (see the next section).

8.2 Pacbio

PacBio is another important platform. Their most successful product is "HiFi" sequencing. HiFi data has the following characteristics:

- Reads are long. Typically 10-20kb.
- Reads are accurate, though not quite as good as Illumina at around Q30.
- Reads are NOT paired.
- Single molecules are sequenced (sort of!). For HiFi, reads are the consensus of many copies of a single molecule (you will learn more details in ISG5301).
- Throughput is moderately high, with around 90Gb of data per flow cell on the Revio instrument.
- Per-base cost is high compared to Illumina. On par with long-read competitor Oxford Nanopore.

PacBio's technology is incredibly useful for genome assembly. The long reads and high accuracy mean that many repetitive regions can be assembled, leading to high contiguity, high completeness, and high accuracy in the final product (rather than the fragmented and incomplete genomes produced early-on using Illumina data). For heterozygous organisms, phased diploid (or polyploid) assemblies are possible. The long reads are also excellent for resolving transcript isoforms, a big challenge when assembling transcriptomes with Illumina data, though the high cost per base is prohibitive for differential expression studies requiring lots of replicates.

8.3 Oxford Nanopore Technologies

ONT is the last platform we'll cover. They have a range of products with diverse uses, but the key characteristics are:

- Reads can be **very** long. Up to 4 megabases with an ultra-long library prep. Standard preps are more like 15-40 kilobases with a fat tail of much longer reads into the 100's of kb.

- Reads have variable accuracy. Current chemistry and base callers can produce around Q20 reads with the standard approach. "Duplex" sequencing can yield higher accuracy, but lower throughput. Older data is more like Q13.
- Raw signal data is produced, which can be reinterpreted when new base-calling software is released, improving accuracy.
- Single molecules are sequenced.
- Methylation can be detected as part of the base-calling process.
- Throughput can be highly variable depending on organism, instrument, and library prep. The extremely tiny, portable MinION: 48Gb. 60-200Gb per flow cell on the production instrument PromethION.
- Per-base cost on par with PacBio.

ONT produces by far the longest reads. At the moment, these are required to produce genuinely complete, unfragmented chromosome-scale assemblies of larger eukaryotic genomes. The very long reads are necessary to resolve messy arrays of tandem repeats, though PacBio is often used in these assemblies as well, because the lower error rate yields a lower error rate in the final assembly, and allows assembly of phased diploid genomes. ONT is also good for resolving transcript isoforms. Detecting modified (i.e. methylated) bases is also an important application of ONT data.

Raw signal data is in fast5 (now deprecated) or pod5 formats. These files are needed if re-basecalling or calling modified bases is desired.

9 Sequence QC

Learning Objectives:
Perform standard quality control analyses on sequence data

9.1 Quality control of sequence data

In this section we're going to work on doing some standard quality checks for sequencing data. There are three basic topics to cover here:

1. Collecting basic summaries
2. Trimming
3. Contamination detection

The only one that is absolutely required across all data types and experiments is the collection of basic summaries. Your sequencing center may be lovely and your lab scientists ultra-competent, but you can't just trust that you've got the data you expect, you have to actually look at it. There is no worse feeling than troubleshooting a pipeline step-by-step backward from the final products, only to discover if you had just taken a few minutes to scrutinize the data before you got started, the last many hours (days? weeks?) of your work would not have been wasted. Once you look at summary data, you can decide whether you can proceed, or if further action is required.

9.1.1 Collecting basic summaries

9.1.1.1 Illumina data

We typically use `fastqc` to characterize Illumina data. It's been around a long time, and it does a fine job of covering summaries we want to look at for most applications. It flags statistics when they are worrisome, but this is less useful, because these flags are tuned to apply to whole genome sequencing data (WGS), so if you have a sequencing library that has different characteristics (e.g. RNA-seq), fastqc will often flag it as problematic though it may not be.

Paired-end sequence is typically delivered in two separate files, which we analyze separately with fastqc.

The usage for fastqc is very simple:

```
module load fastqc/0.11.7

fastqc -o output_dir input_dir/mysequences.fastq.gz
```

You can glob the command like this and it will churn through all the files matching the glob pattern:

```
fastqc -o output_dir input_dir/*.fastq.gz
```

The main output from fastqc is an HTML file which is viewed in a web browser. You can also aggregate many of these HTML files into a single report using multiqc (see below).

For an explanation of fastqc output, please watch this video created by the developers, look at their example reports here and read the brief descriptions of the analysis modules here.

9.1.1.2 Oxford Nanopore Technologies data

ONT data straight from the sequencer is a bit more complex than Illumina data. The raw signal data is often delivered alongside fastq files (or sometimes BAM alignment files - more on that later). Furthermore, the data are often chunked up into many smaller files that you may need to concatenate together for further analysis.

The signal data is delivered in fast5 (deprecated) or pod5 format. You will need the signal data if you want to re-basecall, or do modified basecalling (i.e identify methylated bases). We won't cover that this semester.

When basecalling is complete, a file, usually named `sequencing_summary.txt` is produced. This file can be used to quickly generate summaries of the quality and length distribution of the reads. This information can be generated directly from the reads themselves, but it's slower. While there can't be much length variation in Illumina data (even if you quality trim it, every read is going to be <= 300bp), for Nanopore data the length distribution can be very wide and highly variable among runs and so it's a key variable for understanding your data.

Worth noting is that after basecalling a `report.html` file is also generated. This is worth looking at.

If you download Nanopore sequence data from a public archive, the raw signal data, sequencing_summary.txt, and report.html are unlikely to have been archived alongside the basecalls.

For Nanopore data, we typically use either pycoQC or NanoPlot to summarize.

To run NanoPlot on fastq data (`-t 4` for 4 cpus):

```
module load NanoPlot/1.41.6

NanoPlot --fastq input_dir/mysample.fastq.gz -o output_dir -p mysample -t 4
```

Or on `sequencing_summary.txt`:

```
NanoPlot --summary input_dir/sequencing_summary.txt -o output_dir -p mysample -t 4
```

pycoQC can only accept the summary file like this:

```
module load pycoQC/2.5.2

pycoQC -f sequencing_summary.txt -o pycoQC_output.html
```

Both NanoPlot and pycoQC produce HTML files as their primary outputs (but also some useful text summaries). Both of these programs can produce additional summaries if you provide an alignment file (we have not covered alignment yet!).

For NanoPlot, please read through the author's gallery of example plots. There is not a ton of documentation for pycoQC's output, so you'll have to puzzle it out yourself. It's fairly self-explanatory.

> **Note**
>
> ONT's technologies have been changing more rapidly than other major platforms over the last several years. On both the instrument side (nanopores, chemistry) and the processing side (basecalling from raw signal) there have been substantial improvements in recent years. Because of this it is very important to keep track of the instrument, nanopore version, kit, basecalling software version, and the basecalling model selected in the software when dealing with nanopore data. Some downstream applications will want this information, as the implied error profiles can be different. Unfortunately it is not always provided by researchers who upload data to public repositories.

9.1.1.3 PacBio

PacBio HiFi data is nice and simple. It's not paired-end, and since we don't usually look at the "subreads" that make up the HiFi consensus sequences, we are usually only concerned with a single fastq file.

To summarize, you can use PacBio's proprietary smrtlink software and feed it an XML formatted file produced by the sequencer. Like the helper files from ONT, though, these are not

typically available in public databases, so we recommend just using NanoPlot on the fastq file, as above.

9.1.2 Trimming

Trimming is something we typically do only with Illumina data. There are three aspects to it:

1. Removing adapter contamination ("read-through" contamination is the most common type by far).
2. Trimming low quality ends from sequences.
3. Removing sequences that have been trimmed too short to be useful in steps 1 and 2.

As you will note in fastqc outputs, it is not uncommon to get adapter read-through in Illumina data. It's good to remove this. In many applications a small amount can be tolerated, however. Reference mapping programs mostly do "local" rather than "global" alignment these days, so they are capable of leaving some bases at the end of as sequence unmapped if they don't seem to fit.

To trim adapter contamination, software needs to know what adapters to look for. For some software you will need to provide a fasta file with the specific adapters used in your study. Other software comes pre-loaded with common adapter sequences and you only need to provide them if you're using something unusual.

Sequence qualities often decline towards the 3' ends of sequence fragments in Illumina data. The longer the read, the worse the decline. For most common applications of Illumina data, we don't need to be extremely stringent with the quality trimming. The Phred-scaled base qualities capture the uncertainty in the base calls reasonably well and are used by many downstream applications. For this reason, unless we have a specific need to tightly control potential base calling errors, we just try to trim out the worst bits.

Finally, if after trimming adapters and low quality sequence, we wind up with very short sequences (perhaps < 50bp), we typically remove them. This often leaves "orphaned" mate sequences that are long enough and high enough quality to be retained, which are directed to a separate file. Since there usually not that many of these, we often ignore them downstream as they introduce more complexity in handling data for not much benefit.

Here we'll use `Trimmomatic`, a venerable and widely used piece of software. One downside of using it is that it requires that you know what adapter sequences were used to generate the library and provide them. When that information is not available (sometimes published data do not state it) FastQC usually infers it for you (when there is detectable adapter contamination that is). Also, Trimmomatic comes packaged with fasta files for the most common adapters, so this is not really a large barrier.

There are many other similar useful tools for trimming, such as `fastp`, `sickle` and `cutadapt`.

For bioinformatics workflows, we do not typically modify our raw data files in any way (rather, we set permissions to read-only). This means that some steps, like trimming, more or less double the size of our data during the course of analysis. Once you're fairly certain you're done with intermediate analysis products like trimmed reads, it's a good idea to go back and remove them to save storage space.

To run `Trimmomatic`:

```
module load Trimmomatic/0.39

ADAPTERS=/isg/shared/apps/Trimmomatic/0.39/adapters/NexteraPE-PE.fa

java -jar $Trimmomatic PE -threads 4 \
        indir/mysample_R1.fastq.gz \
        indir/mysample_R2.fastq.gz \
        trimdir/mysample_trimmed_R1.fastq.gz trimdir/mysample_trimmed_orphans_R1.fastq.gz
        trimdir/mysample_trimmed_R2.fastq.gz trimdir/mysample_trimmed_orphans_R2.fastq.gz
        ILLUMINACLIP:"${ADAPTERS}":2:30:10 \
        SLIDINGWINDOW:4:25 MINLEN:45
```

A couple of notes: - `\` at the end of each line tells the bash interpreter to ignore the line break so that all 7 lines above are treated as a single continuous line. This simply allows more clarity with long command lines. - `java -jar program.jar <options>` is a common way to run programs written in Java. In the module system we usually have the module create an environment variable upon loading that contains the path to the jarfile. Try loading Trimmomatic and doing `echo $Trimmomatic` now. You can see how it is set by doing `module display Trimmomatic/0.39` and looking at the `setenv` line. - `ILLUMINACLIP` provides the parameters needed to search for adapters. Read through the documentation to understand it better. - `SLIDINGWINDOW` gives parameters for sliding window quality trimming (if any 4 bases drop below a mean Q of 25, the rest of the read is trimmed) - `MINLEN` should be self-explanatory.

We may do other quality filtering for ONT and PacBio data, depending on application, but that is typically length and/or mean quality filtering, which can be easily accomplished using `seqkit`, which we demonstrated previously, or taxonomy-based filtering which we will cover in the next section. For more in depth filtering of long reads by quality and length you can also try `filtlong`, but we won't cover it here.

9.1.3 Detecting mixtures and contamination

Sometimes our sequence data is a mixture of DNA or RNA from different organisms. This may be a result of biology. That is, for some organisms, tissues, or data types, we may be unable

to completely separate our focal organism's DNA or RNA from one or more other organisms. In this case, non-target DNA is usually derived from a taxonomically distant organism.

Mixtures may also be a result of inadvertent contamination in the field or laboratory. Depending on the context, mixtures may be of distant or closely related organisms.

When we are concerned this may be the case, there are QC steps we can take. When it is detected, what we decide to do depends on our downstream application. We may want to remove non-target reads at this stage, or we may want to retain them and manage their influence on our analysis later.

In this section we'll talk about how to use metagenomic classification software to try to identify the extent of mixing of more distantly related species (note that software such as `CalculateContamination` exists for the cross-contamination case).

Metagenomic classification essentially takes "query" sequences of interest and maps them to a database of "target" sequences. When matches are found, they are reported. In the software we use, both detailed match information for each sequence (which can be quite a large file) and summary information about the taxonomic distribution of matches is produced. The key element in all classification analyses is the database. A match can only be identified if it is present in the database. There are publicly available standard databases for the software we recommend, but users can also build their own composed of potential contaminants.

For Illumina short-read data we recommend using `kraken2`. Databases can be obtained here, but the standard database is already available on the Xanadu cluster here: `/isg/shared/databases/kraken2/Standard`. This database contains bacterial, viral, archaeal and human sequences.

```
module load kraken/2.1.2

kraken2 -db /isg/shared/databases/kraken2/Standard \
        --paired indir/mysample_R1.fq.gz indir/mysample_R2.fq.gz \
        --use-names \
        --threads 16 \
        --output outdir/kraken_general.out \
        --unclassified-out outdir/unclassified#.fastq \
        --classified-out outdir/classified#.fastq      \
        --report outdir/kraken_report.txt \
        --use-mpa-style
```

In this case the output file format `unclassified#.fastq` will cause kraken to produce R1 and R2 fastq files without having to specify them both by name. `kraken_general.out` will give the classification status of ALL reads (or read pairs in this case). `kraken_report.txt` will give a taxonomic summary.

We recommend centrifuge for long reads from PacBio and ONT. You can run it like this:

Outputs are

```
module load centrifuge/1.0.4

centrifuge -q \
        -x /isg/shared/databases/centrifuge/b+a+v+h/p_compressed+h+v \
        --report-file outdir/report.tsv \
        --quiet \
        --min-hitlen 50 \
        -U indir/mysequences.fastq.gz >outdir/centrifuge_results.txt
```

Note that it uses a different database format, but is still composed of the same sequence set. It is available at the link above, but also on Xanadu at the path given in the example command. This will run with unpaired reads (`-U`) and require database hits to be at least 50bp.

The stdout captured by `outdir/centrifuge_results.txt` will give the classification of every read. `outdir/report.tsv` will give the taxonomic summary.

9.2 MultiQC

MultiQC is a fantastic tool for aggregating quality reports across samples and pieces of software. It can read in reports from most things we'll use and digest them into a readable format. This is particularly useful when you have many samples. You can imagine that checking fastqc html files individually for 20 or 30 (or 800) samples can range from tedious to virtually impossible. MultiQC gives you a quick overview of everything that can key you into problems when they exist.

MultiQC has very simple usage:

```
module load MultiQC/1.15

multiqc -f -o multiqc_out input_dir
```

`-o` specifies an output directory. `-f` means "force-overwrite" if the directory already exists and contains results (handy when re-running or testing scripts). MultiQC will search `input_dir` and subdirectories for summary files it recognizes.

9.3 Exercises:

In the directory `/core/cbc/gdafiles/ISG5311/data/sequence_examples/` we have 6 sequencing data from our three focal platforms. You will run various QC steps on each.

In the `Illumina` subdirectory, we have three datasets:

1. Paired-end WGS data for a fish, *Fundulus grandis* (`BU18FLFB002`).
2. Paired-end RAD-seq data for an ash tree in the genus *Fraxinus* (`Plate1_pool1`).
3. Paired-end RNA-seq data for a deciduous conifer *Larix laricina* (`K31`).

Important note about the RAD-seq data: R1s begin with a variable length sample barcode sequence (5-8 bases, 16 samples in the pool) followed by 6 bases of the SbfI restriction enzyme recognition site. R2s begin with a 10 base unique molecular identifier ("UMI"; the first 5 bases are completely random) followed by 3 bases of the MseI restriction enzyme recognition site.

In the `pacbio` subdirectory, we have only one dataset.

1. `SRR15654800`, whole genome HiFi data from a caddisfly.

In the `ONT` subdirectory we have two datasets (information about basecalling is provided in the README.md file):

1. A fastq file (`SRR28295757`). This is newer ONT duplex data from a human sample, HG002. A sample widely used in benchmarking studies for sequencing technologies.
2. We also have a nanopore `sequencing_summary.txt` file from a fish, again *Fundulus grandis*. These data are older and will have a much higher error rate.

Analysis to be done:

1. **FastQC** - On all fastq files.
2. **NanoPlot** fastq mode - On ONT human data (`SRR28295757`) and on PacBio caddisfly data (`SRR15654800`).
3. **NanoPlot** sequence summary mode - On *Fundulus grandis* (`sequence_summary.txt`)
4. **pycoQC** - On *Fundulus grandis* (`sequence_summary.txt`)
5. **Trimmomatic** - On only one Illumina dataset: *Fundulus grandis* (`BU18FLFB002`).
    - Run `FastQC` again on the trimmed sequences.
6. **Kraken2** - On only *Larix laricina* (`K31`).
7. **Centrifuge** - On ONT human data (`SRR28295757`).
8. **MultiQC** - On the directory containing your results for the Illumina data only. This should have fastqc reports for both before and after trimming. Running it for *all* the data will result in some wonky figures because the ONT and PacBio data are so different in length.

This is a lot of analysis! The idea is to help you see some of the differences among different types of datasets and how they turn up in standard quality control measures. Put each analysis in a separate script. Use the scripts to organize your output into a single results directory with a subdirectory for data from each platform.

These are real datasets, and running these programs will take more than a few minutes, some will take several hours. You will need to submit this work via SLURM and monitor the jobs to ensure they complete without error. You should give yourself plenty of time to do this. For all programs, `module load` them and run `<program> --help` to look at the usage and basic options.

Some pointers:

- Kraken2 needs to load the entire database into memory to run, so you will need a large enough memory request (plus a few gigabytes extra) for that. Check the size of the directory containing the database to figure this out.
- For the ONT data in fastqc you will need to change add the flag `--memory 2000`. This is the amount of memory in megabytes. The default is 512Mb, but the very long reads require more.
- remember to check the stderr and stdout files produced by SLURM for potentially useful summaries.
- This is going to require many scripts (or many lines in a single script). Think about how you're going to keep scripts, log files and outputs organized for these diverse programs. This may feel like a struggle. I have deliberately not provided a lot of guidance here. We'll cover project organization in the next module.

Questions:

1. Which of the Illumina datasets has the highest adapter contamination?
2. Which of the Illumina datasets has the highest duplication rate?
3. Which of the 3 long-read datasets (2 x ONT, 1 x pacbio) has the longest median read length?
4. For every 10,000 sequenced bases, you would expect _____ errors in the older killifish ONT data, _____ errors in the newer human ONT duplex data, and _____ errors in the pacbio caddisfly data. This is calculated by multiplying the mean probability (from NanoPlot, *not* fastqc) by 10,000.
5. How many fragments of the Larix dataset were classified as bacteria?
6. What percentage of bases were removed from the killifish Illumina data by Trimmomatic?
7. Which of the Illumina data sets has the worst (untrimmed) median quality score at the 3' end of the sequence (use fastqc or multiqc output)
8. The R2 sequences for the RAD-seq data `Plate1_pool1` begin with a *unique molecular identifier*, or *UMI*. This is a random nucleotide sequence that can be used to determine when two sequences are clones of the same original biological molecule (for a pair of sequences, if the UMIs are the same, and the sequences are the same for high quality

bases, then the sequences derive from the same molecule). Judging from the fastqc output, how many fully random bases (we usually term these *4-fold degenerate* bases) does the UMI sequence start with?

Part VII

Measuring Gene Expression I

Part VIII

Learning Objectives

Learning Objectives:
Organize a computational project
Retrieve data from public databases
Perform initial QC
Align data against a reference genome
Quantify gene expression using an existing gene annotation

10 Overview of differential expression with RNA-seq

Learning Objectives:
blah blah

10.1 A model workflow

Beginning with this section of the course we're going to walk through the steps of a model workflow: **differential expression with RNA-seq**. This is a common application of high-throughput sequencing data, and will introduce a number of concepts and techniques applicable across diverse workflows.

10.2 Workflow steps

A differential expression analysis has these basic steps:

1. Retrieve and QC data.
2. Quantify gene (or transcript) expression.
3. Test for differences in expression among treatment groups.
4. Interpret differential expression results via ranking, gene set enrichment, and/or pathway analysis.

Steps 1 and 2 are typically done using Linux tools on the HPC. Steps 3 and 4 will require us to introduce a new language, R. Specialized software has been written for differential expression analysis in R, but R is very widely used in statistical analysis, data cleaning, and generating visualizations.

10.3 Focal data

We're going to walk through the workflow using a real dataset from this paper:

> Reid, Noah M., et al. "The genomic landscape of rapid repeated evolutionary adaptation to toxic pollution in wild fish." Science 354.6317 (2016): 1305-1308.

The Atlantic killifish (also known as the mummichog, *Fundulus heteroclitus*) is an abundant inhabitant of estuaries along the Atlantic coast of North America. Some of these estuaries were intensely polluted by heavy industry in the middle of the 20th century. Pollutants are diverse, but include highly toxic polychlorinated biphenyls (PCBs), polycyclic aromatic hydrocarbons (PAHs) and dioxins. These pollutants cause adverse effects in nearly all vertebrates, but exposure is particularly damaging during early development.

In several distinct heavily polluted estuaries, killifish persist. These fish show extremely high resistance to these pollutants compared to fish from nearby sites with minimal pollution. The resistance is heritable across many generations in fish bred in the lab, indicating it has a genetic basis.

The authors of this study wanted to determine the genomic basis of adaptation to toxic pollution in these fish, and learn whether it differed among populations at different polluted sites. They collected population genomic and gene expression data from 4 pairs of populations (one sensitive, one tolerant) to address this question. The population genomic data were generated from wild-caught fish. The expression data were the result of an experiment conducted with lab-bred fish from each population. See the paper for details, but in brief: embryos were exposed to either a DMSO control, or PCB-126. RNA from whole embryos was extracted to measure gene expression, with 4-6 replicates per treatment group.

The authors measured gene expression with RNA-seq. RNA-seq will be covered in more detail in ISG5301, but briefly: RNA-seq is a catchall term that refers to RNA that has been extracted from a tissue or organism, reverse-transcribed into cDNA, and then fragmented and sequenced on the Illumina platform. We frequently use RNA-seq to quantify transcript or gene expression. To do this we make the assumption that the number of fragments of a given transcript or gene found in our sequencing library is roughly proportional to the frequency of that transcript in the cell population we extracted the RNA from.

There are *lots* of caveats here, which we will cover in due time. At any rate, our goal in the first half of this workflow is to generate counts of RNA fragments attributable to a given gene or transcript to use as a measurement of expression.

The RNA-seq data is what we'll use here. In the chapters that follow, we'll use data from the Elizabeth River/King's Creek pair of populations, and exercises will require you to expand the analysis to include more populations.

10.4 Excercises

Read the paper linked above and answer the following questions:

11 Project Organization

Learning Objectives:
blah blah

Before we can get started, we need to cover a really basic topic: project organization. Lots of people will have preferences about how to do this, but mainly they will boil down to a few key points:

1. Stay organized.
2. Document everything.
3. Use file names that are easy to parse with code.

11.1 Stay organized

This is a very vague directive. In part this is because projects are diverse and have diverse needs. We are, however, going to recommend (more like require) that you follow some very specific guidelines in this course regarding organization.

First, each project should have **its own directory**. Computational projects, especially when you're working with new data or a new workflow can grow organically, so deciding when something is a *new* project requiring its own space, or just an extension of the current project is somewhat subjective. What you definitely *should not* do, is start writing random bits of code and downloading data willy nilly into your home directory. You *should not* take delivery on a dataset and start your analysis in the shiny new raw data directory.

Even if you are "just trying something out, just for a few minutes, it definitely won't turn into some big involved thing", create a new directory.

In general, your projects for this course should be organized like this:

```
myProject/
   data
   README.md
   results
```

```
  scripts

3 directories, 1 file
```

You can initialize a new project very simply like this:

```
mkdir -p myProject/{scripts,results,data}
touch myProject/README.md
```

You may eventually create other directories. Maybe you want to install specific pieces of software in a project directory called `bin`. Maybe you are generating figures and you want them to have their own `figures` directory. Maybe you are writing out detailed descriptions of your methods and you would like to keep them in `docs`. That's fine. The key here is that your *data* goes in `data`, your *code* goes in `scripts` and your *results* go in `results`. We'll cover how to make this happen as we move forward.

An organizational structure like this, or some version of it, will make your life easier in so many ways. It will be easy for you to move between projects (you are likely to have several projects active at any given time), to pick up projects that you have left idle for some weeks (or possibly months), and for collaborators to look in on your work if you need help or feedback at this granular level. You won't have to remember where you put everything for every given project, or just exactly how far along it was when you put it down, you'll have a logical layout that quickly explains itself.

Within these basic directories, you can use subdirectories to keep things organized. The most optimal arrangement of subdirectories can be hard to predict in advance, but if you start with one layer of subdirectories in each main directory, then breaking things up or reorganizing them without breaking relative paths is not hard. Using adequately descriptive names, and possibly even numbering the directories to indicate what order the steps were run in is extremely helpful.

For the scripts directory, we could start with something like:

```
mkdir -p myProject/scripts/01_getdata myProject/scripts/02_qc
```

To the greatest extent possible, you want to have everything you need to do the project inside this directory. The more things outside the project directory you point to, the more opportunities there are for scripts to break when things are moved or deleted.

11.1.1 Where and how to keep data

Contrary to the last section, for large sequencing datasets that you've generated, it's probably not a good idea to keep the data inside a specific project directory. You may want to use

the data for multiple projects, and copying it is a waste of storage space. On the other hand, it's very useful to design self-contained project directories. A solution to this is to keep raw data in one place (and change permissions to read-only), and then **symlink** the data to one or more other places. When you symlink a file from one directory to another, you're creating a marker in the target directory that acts as if it's a file (or directory), but is actually just a pointer to the original. We'll cover how to do this in the next chapter.

11.2 Document everything

There are many levels of documentation, but the most basic is **the code itself**. Write scripts for **literally every piece of your project**. Do you need to move a file? Put it in a script. Create a directory? Put it in a script. You don't need separate scripts for all of this. We'll see more as we move along, but for each step in your analysis, you can do things like begin the script by creating the directory in which the outputs will be stored. If you record every command required to recreate your results in a script, you will never find yourself wondering "how the heck did I create this intermediate file, anyway?", let alone worrying about whether you created it correctly, because you can go back and check the code. This is absolutely essential to have confidence in your work.

You are almost certainly going to produce wrong or problematic results at some point, it's an occupational hazard in bioinformatics. You never want to be in a position where you're wrong and you don't have the documentation to figure out how it happened.

The next layer of documentation is **code comments**. You may have all your code written down in nice, clean scripts, but sometimes interpreting that code without running it can be challenging (both for your collaborators and future you). At key moments in your scripts, you should add comment lines to explain what the code is doing in plain language.

A final layer of documentation is to create one or more README files that explain clearly what the project is trying to accomplish, what the steps are and where to find code and results. It may also contain notes about various abandoned pieces of software, parameter combinations, etc that may explain the final form of the analysis.

11.2.1 Documenting data

The preceding recommendations do not just apply to your code, but data as well. When you download data from a any kind of public repository, you want to keep a record of exactly what you downloaded. Ideally, the primary place you do this is in a script maintained inside your project. In a previous chapter we used NCBI's `datasets` software to download genomes and annotation files. We used an **accession number** to get the data. This is a stable identifier that refers to a specific, unchanging item in an NCBI database. For an NCBI-housed genome,

this is what you want to track. NCBI provides accession numbers for many other pieces of data it houses, as do many other public databases.

11.2.2 Documenting software

Software in bioinformatics changes, in some fields quite rapidly. It's important to keep track of which software versions you're using for a given task, and ideally, to set your code up so that software versions will not change without your knowledge when re-running code. It may seem like updating software to the latest version is always good, but sometimes flags or default behavior can change. This can cause outright errors, which can be frustrating, but at least you know about them, or it can quietly alter your results, a much worse state of affairs.

Some tips:

- If you're using the module system, always specify the software module.
- If you've installed your own software, put the version number in the readme.
- If you're using a conda environment, export the environment yml file and keep it somewhere in your project directory.
- If you're using a public software container, be explicit about the version you're pulling, and use a script to pull it.

11.3 File naming

This may seem like a silly topic, but it's worth covering briefly. If you're a user of GUI operating systems, you may be accustomed to naming things using arbitrary character strings, sometimes including whitespace, e.g.: `my resume - 2023.docx`. You may have multiple versions of files with descriptive, but inconsistently formatted names. In the same directory you may also have `my resume - April 2021.docx` and/or `CV - 2020 FINAL VERSION.docx`, `CV - 2020 DRAFT.docx`.

Names like this can be problematic in command-line environments. To start with, while whitespace is technically allowed in file names on Linux systems, whitespace is also the delimiter for elements on the command line. So if you tried to copy that first resume into the directory `resumes/` like this:

```
cp head my resume - 2023.docx resumes/
```

You would get

```
cp: my: No such file or directory
cp: resume: No such file or directory
```

```
cp: -: No such file or directory
cp: 2023.docx: No such file or directory
```

You would actually have to use slashes to *escape* the spaces, or quote the entire string:

```
# like this
cp my\ resume\ -\ 2023.docx resumes/
# or this
cp "my resume - 2023.docx" resumes/
```

A further problem, which maybe doesn't matter too much for a bunch of resume versions, but becomes an issue with data files in particular, is that these files don't have a consistent naming convention. It would be hard to programmatically manage them. It's useful to think of file names (particularly when there will be many similar files) as being almost like rows in a table, with pieces of the names being columns.

It's better to aim for something more like this:

```
2020_01_CV_draft.docx
2020_02_CV_final.docx
2021_04_resume_final.docx
2023_06_resume_final.docx
```

In this case the dates are coded consistently and in a way that the usual lexicographic sorting in `ls` will sort them by date. There are 5 pieces of information in each name (year, month, document type, status, file suffix) each separated by a `_`. If you wanted to manage these files using code, it would be straightforward: loop over each file, extract information from the name, and taking some action based on that information.

So, to summarize: name files with numbers, letters, underscores and dashes, and try to stick with a consistent naming convention.

12 Retrieving data

Learning Objectives:
blah blah

12.1 Getting set up

As we walk through this RNA-seq workflow you will do the following at each step:

1. Go over the details of a script that has already been written.
2. Run the script.
3. Check that the script completed successfully.
4. Examine the output.
5. As an exercise, modify the script in one or more ways.

12.1.1 Getting the scripts

The scripts you'll run and modify are in a code repository on GitHub. GitHub is a web platform that hosts code repositories. It's built on the version control software `git`. We will cover git extensively next semester.

You can see our scripts here. Note that this repository already has the skeletal project organization we discussed in the previous chapter. To download the scripts to your home directory on Xanadu, we're going to use `git`, specifically the subcommand `clone`, to create a local copy of this repository. Git is available by default on Xanadu nodes, including the login node, so we don't need load any modules.

To clone any git repository you have access to, you can go to the repository's GitHub site, click on the big green button in the upper right that says `<> Code`, select "HTTPS", and copy the URL. Then, in general:

```
git clone <URL> <directory_name>
```

You can omit `<directory_name>` and the name of the repository will be used.

Let's name our first local copy of the repository `rnaseq_tutorial`:

```
git clone https://github.com/isg-certificate/rnaseq.git rnaseq_tutorial
```

The directory tree should start like this:

```
rnaseq_tutorial/
  metadata
      SraRunTable.txt
  README.md
  scripts
      01_rawdata
          01_downloadSRA.sh
          02_downloadGenome.sh
      02_QC
          01_fastqc.sh
          02_multiqc.sh
          ...
      03_...
```

You can modify these scripts in any way you like without impacting the remote repository on GitHub (you don't have permission to push changes there). You can also create as many clones of the repository as you like as long as you give them different names.

12.2 Downloading data from the SRA

12.2.1 Where do we get the data?

Our first step is to download RNA-seq data. As we mentioned in the overview chapter, we're going to walk through the workflow using real expression data from this paper:

> Reid, Noah M., et al. "The genomic landscape of rapid repeated evolutionary adaptation to toxic pollution in wild fish." Science 354.6317 (2016): 1305-1308.

The data have been deposited in a database at NCBI. There are actually many interconnecting databases at NCBI, and this dataset touches on several of them. To start with, the data have been organized into a **BioProject**. BioProjects accessions are umbrellas that can encompass a wide variety of data, including assembled genomes, resequencing data, expression data and more. Our BioProject has accession number PRJNA658029. If you follow the link, you'll find information about the experiment, including links to other databases. 75 individual fish embryos were selected for sequencing, so there are 75 accessions in the **BioSample** database.

The key link here is to SRA or the Sequence Read Archive. SRA houses raw sequencing datasets, and holds an enormous amount of data. Many scientific journals require sequence data to be deposited in the SRA as a condition of publication. If you click on the SRA experiment link, you should get to a page that lists SRA accessions for each of our samples. At the top, there should be a link, `Send results to run selector`. If you click there, you should be able to view a table containing the metadata for all of our samples, including their population of origin and their experimental PCB-126 exposure. Downloading this table will give you the same file that is found in the `metadata` directory of the git repository.

Note also that the BioProject page has a link to **GEO**, or the Gene Expression Omnibus. That database also houses the count data generated from these samples that was used in the original publication.

To download the data, we're going to use `sratools`, a toolkit created by NCBI to access the SRA.

The expression data are **unstranded, paired-end sequenced RNA-seq**. You should now know what paired-end refers to, but *unstranded* means that the cDNA has been sequenced without respect to strand. In a stranded RNA-seq library, reads will all be derived from one strand or the other.

12.2.2 Running tasks in parallel

We're going to download the data using the script `scripts/01_rawdata/01_downloadSRA.sh`. But before we do that, we need to talk about running tasks in **parallel**. Part of the reason we use computer clusters for bioinformatics is that consumer laptops and desktops don't have the computational power required to accomplish some tasks. In truth, however, individual CPU cores on a consumer machine might not be much slower, if at all, than the individual CPU cores on Xanadu. Xanadu, however, is composed of many nodes, and on each of these nodes are 32-96 CPU cores, compared with 8-12 on a high end consumer computer. The increased computing speed we get from using a cluster primarily comes from the ability to divide work up into pieces and use many CPU cores simultaneously, or *in parallel*.

There are two general types of parallel computing applications:

1. **Coupled applications**, in which parallel processes need to communicate with each other, at least sometimes.
2. **"Embarrassingly parallel"** applications, in which parallel processes are independent.

Many programs that require parallel operations to be coupled are written so that they can easily use many CPU cores on a single computer. As we have seen in some past applications, you may provide the flag `-t` or `-p` to specify the number of CPU cores to be used. Because the CPU cores are all in a single machine, they can communicate easily.

This type of within-machine parallelism is **very common** in bioinformatics software. The main thing to know about this is that though many programs can be sped up by allowing them to use more processors, but the speedup is seldom linear. Adding more processors comes with increasing overhead for managing them, and for most applications, how quickly returns diminish depends on the program. We rarely use more than 8-12 CPUs for a single program.

In some coupled applications, however, even the 96 CPUs on Xanadu's are insufficient. In those cases, coupled parallel processes need to be able to communicate *across nodes*. That adds an extra layer of complexity, which is often handled by MPI, or "Message Passing Interface". Applications requiring this large scale parallelism are often written to use MPI to manage it. Between-node coupled parallelism is fairly uncommon in bioinformatics, and we won't cover it in this course.

There are some cases where extremely high coupled parallelism is required, and the overhead of MPI is inefficient. For those cases, we often use graphical processing units (`GPUs`). These days, the most common use cases involve machine learning models. Basecalling raw Nanopore signal data essentially requires a GPU to complete in a reasonable amount of time. Multiple nodes on Xanadu have GPUs available. Next semester we will use GPU resources in some of our workflows.

Embarrassingly parallel problems are **extremely common** in bioinformatics. It is very often the case that a large analysis can be broken up into many small, independent pieces. Most commonly, we break these problems up by sample (think of running `fastqc` on, say, 75 RNA-seq samples) or by genomic region (think of searching for genetic variants in 10,000 genomic windows).

There are two typical ways to manage embarrassingly parallel jobs on an HPC:

1. Use a program like `GNU parallel`
2. Use a SLURM array

We're going to cover `GNU parallel` in this section, and SLURM arrays just a few sections down.

12.2.2.1 GNU parallel

Usually just called `parallel`, this program is one of the most commonly used Linux utilities. It's a very powerful and flexible program and we'll cover only a couple of approaches to using it here, but there is an extensive tutorial.

The basic idea is that given a list of inputs, parallel can be used to operate on each element of the list simultaneously.

Let's create a test file:

```
echo -e "A\nB\nC" >abc-file.txt
```

It should read:

```
A
B
C
```

You can run parallel on this file like this:

```
parallel -a abc-file.txt echo {}
```

The `{}` is used as a placeholder for the input value and you can use it to construct command lines. In this simple case you can leave it out. There are lots of options for modifying it (see the docs) but we're going to keep it simple here.

Alternatively, you can have it read from stdin like this:

```
cat abc-file.txt | parallel echo {}
```

Or like this:

```
parallel echo {} < abc-file.txt
```

Oor directly from the command line, using whitespace as a separator:

```
parallel echo {} ::: A B C
```

Importantly, you can limit the number of jobs that run simultaneously with `-j`

```
parallel -j 2 echo {} ::: A B C
```

And you can get it to "dry run" by simply printing the commands instead of running them. This is critical to check when building complex command lines:

```
parallel --dry-run -j 2 echo {} ::: A B C
```

If you're having trouble formatting the command line you want, you can resort to constructing it in bash and just sending it to parallel to be read from stdin:

```
for i in $(cat abc-file.txt)
do echo "echo $i"
```

```
done |
parallel -j 2
```

12.2.3 Finally, the download script

Ok, so you're logged into Xanadu, you've cloned the git repository, you get the gist of `parallel`. You're ready to walk through our script and run it. The script is found in `scripts/01_rawdata/01_downloadSRA.sh`. It's written to be run from that directory, so `cd` there to have a look.

We'll go through the script in chunks.

First, the SLURM header. We're requesting 12 cpus and 15G. We're going to run two downloads simultaneously in this script, with one CPU per execution of the download tool, but we'll compress 12 samples at a time after they've finished downloading, so we request 12 CPUs. We're submitting to general partition with general QOS.

```
#!/bin/bash
#SBATCH --job-name=fasterq_dump_xanadu
#SBATCH -n 1
#SBATCH -N 1
#SBATCH -c 12
#SBATCH --mem=15G
#SBATCH --partition=general
#SBATCH --qos=general
#SBATCH --mail-type=ALL
#SBATCH --mail-user=first.last@uconn.edu
#SBATCH -o %x_%j.out
#SBATCH -e %x_%j.err
```

This is not required, but we often start scripts with these commands to make it simpler to keep track of when and where (on xanadu) they ran.

```
hostname
date
```

We're loading `parallel` and `sratoolkit` here. `sratoolkit` has the tool we'll use to download the data.

```
# load software
module load parallel/20180122
module load sratoolkit/3.0.1
```

Oooh look! Comments that help us keep track of what data exactly we're going to download.

```
# The data are from this study:
    # https://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE156460
    # https://www.ncbi.nlm.nih.gov/bioproject/PRJNA658029
```

Below we're specifying variables that point to things we'll use further down in the script. We find that obscuring paths and filenames with descriptive variables can help make them much more readable and organized. Note that we're using relative paths here. This helps keep our project directory self-contained and minimizes problems if things get moved around on the cluster. It also means the script needs to be run from this location.

```
OUTDIR=../../data/fastq
    mkdir -p ${OUTDIR}
METADATA=../../metadata/SraRunTable.txt
```

Note that `OUTDIR` does not exist yet. We create it in this script so that we can write our fastq files there. This a small example of writing everything you do into a script. In this case, a key project directory is created only when it is needed.

`METADATA` is a file that is a part of the repository and was downloaded when you cloned it. It's the SRA metadata mentioned in a section above. You can do `less` on the file to see what it looks like. The metadata file contains all 75 samples from all 8 populations. We're just going to work with 2 populations here, so we're going to extract the SRA accession numbers for those samples and put them into a text file:

```
# extract rows matching our population names, pull out the SRA accession number (the first
ACCLIST=../../metadata/accessionlist.txt
grep -E "Elizabeth River|King's Creek" $METADATA | cut -f 1 -d "," >$ACCLIST
```

Make sure you know why we use these options with `cut`: `-f 1 -d ","`, and then have a look at the accession list file. We will return to this file over and over again to construct file names in future scripts. The accession list should have 20 elements in it.

Finally, we run the program `fasterq-dump`:

```
# use parallel to download 2 accessions at a time.
cat $ACCLIST | parallel -j 2 "fasterq-dump -O ${OUTDIR} {}"
```

Try manually creating all these variables in an interactive session and running this command line with the `--dry-run` option. As I mentioned, when writing your own scripts this is a critical step for checking for mistakes.

`fasterq-dump` does not automatically compress the fastq files, so to be efficient with disk space we will compress them:

```
ls ${OUTDIR}/*fastq | parallel -j 12 gzip
```

Submit this job with `sbatch 01_downloadSRA.sh`. Monitor it periodically for progress or errors. It should take under an hour. Look back at the chapter on monitoring SLURM jobs if you need to refresh. Make sure you look at `.out` and `.err`.

This is going to sound silly, *but check that you have the correct number of files in your output directory.* This is a basic sanity check and people often do not do it. SRA network connections can be problematic, and downloads are occasionally interrupted, so it's important not to just assume this worked.

Notes:

- There are THREE files per sample. This is because these data were uploaded after an initial QC step, and orphaned reads were included. We're going to ignore these.
- From the moment you get your hands on data, you should start treating it skeptically. Ask if it meets your expectations. Look at the file sizes for our data. `SRR12475446` has comparatively *very* little data. Ultimately we will have to drop this sample.

12.3 Downloading genomes from ENSEMBL

Our analysis here is going to be reference-genome based. This means we're going to align our RNA-seq data to a reference genome and use the gene annotation to attribute RNA fragments to genes. We need to download the genome and annotation in order to do this.

While NCBI and Europe's equivalent, ENA, are probably the two biggest concentrations of public raw sequence data, genome, and particularly genome annotations, are housed in more dispersed databases. NCBI has an annotation pipeline it runs on many genomes. When researchers generate an annotation as part of a project, they will often deposit it in generic data repository. There are some taxon-specific databases as well, such as flybase and phytozome.

We're going to get our genome and annotation from ENSEMBL. ENSEMBL is a database for vertebrate genomes. They generate their own annotations, and resources for comparing genes and genomes among species. Using ENSEMBL will allow us to easily extract functional information for *F. heteroclitus* genes that we would otherwise have to infer ourselves with a separate pipeline.

ENSEMBLE releases new versions periodically. Annotated genes have accession numbers that are tied to ENSEMBLE release versions. Automated gene annotation is a constantly improving field, so gene models can change a lot, even if the underlying genome assembly does not. It's

important when getting data from ENSEMBL to get it from a specific version of the database, and to keep track of which version you're using.

Note also that ENSEMBL doesn't sequence genomes. They pull genomes in from other organizations. Our genome is identical to an NCBI accessioned genome, for which they provide an NCBI accession number (GCA_000826765.1).

Getting the data is pretty straightforward. We visit our organism's page and follow the appropriate links to get URLs which we use with `wget`. The FASTA file for the genome we want has the suffix `dna_sm.toplevel.fa.gz`. `sm` means repetitive elements have been softmasked. We're also going to grab the annotation in `GTF` format and the inferred transcript sequences (suffix `cdna.all.fa.gz`).

The script is straightforward. It requires few resources.

```
#!/bin/bash
#SBATCH --job-name=get_genome
#SBATCH -n 1
#SBATCH -N 1
#SBATCH -c 4
#SBATCH --mem=2G
#SBATCH --partition=general
#SBATCH --qos=general
#SBATCH --mail-type=ALL
#SBATCH --mail-user=first.last@uconn.edu
#SBATCH -o %x_%j.out
#SBATCH -e %x_%j.err
```

We're going to index the genome with samtools (more in a moment).

```
# load software
module load samtools/1.16.1
```

As before, we're going to create our output directory as part of the script.

```
# output directory
GENOMEDIR=../../genome
mkdir -p $GENOMEDIR
```

More useful comments!

```
# we're using Fundulus heteroclitus from ensembl v112
    # we'll download the genome, GTF annotation and transcript fasta
    # https://useast.ensembl.org/Fundulus_heteroclitus/Info/Index
```

```
# note that in these URLs we are downloading v112 specifically.
```

Download the files.

```
# download the genome
wget -P ${GENOMEDIR} http://ftp.ensembl.org/pub/release-112/fasta/fundulus_heteroclitus/dn

# download the GTF annotation
wget -P ${GENOMEDIR} http://ftp.ensembl.org/pub/release-112/gtf/fundulus_heteroclitus/Fund

# download the transcript fasta
wget -P ${GENOMEDIR} http://ftp.ensembl.org/pub/release-112/fasta/fundulus_heteroclitus/cd
```

Decompress the files. Unfortunately, reference genome/transcriptome fasta files need to be decompressed for lots of applications.

```
# decompress files
gunzip ${GENOMEDIR}/*gz
```

Finally, we're going to create simple indexes for the fasta files with `samtools faidx`.

```
# generate simple samtools fai indexes
samtools faidx ${GENOMEDIR}/Fundulus_heteroclitus.Fundulus_heteroclitus-3.0.2.dna_sm.tople
samtools faidx ${GENOMEDIR}/Fundulus_heteroclitus.Fundulus_heteroclitus-3.0.2.cdna.all.fa
```

All the outputs from this script should be pretty self-explanatory, except the index files (suffix `.fai`), which we covered in the chapter on sequence data formats. Remember to check the `.err` and `.out` files for errors or warnings, and to verify that your files downloaded.

12.4 Sequence data QC

In the previous module we discussed doing quality control on Illumina data using FastQC, MultiQC and Trimmomatic. Here we will do each of these steps on our newly downloaded data.

12.4.1 FastQC on raw data

The script for FastQC is `scripts/02_QC/01_fastqc.sh`. The full script looks like this:

```
#!/bin/bash
#SBATCH --job-name=fastqc
#SBATCH -n 1
#SBATCH -N 1
#SBATCH -c 10                    #<1>
#SBATCH --mem=20G                #<1>
#SBATCH --partition=general
#SBATCH --qos=general
#SBATCH --mail-type=ALL
#SBATCH --mail-user=first.last@uconn.edu
#SBATCH -o %x_%j.out
#SBATCH -e %x_%j.err

echo `hostname`

########################################################################
# FastQC
########################################################################
module load fastqc/0.12.1        #<2>
module load parallel/20180122    #<2>

# set input/output directory variables
INDIR=../../data/fastq/                          #<3>
REPORTDIR=../../results/02_qc/fastqc_reports
mkdir -p $REPORTDIR

ACCLIST=../../metadata/accessionlist.txt         #<4>
# run fastp in parallel, 10 samples at a time
cat $ACCLIST | parallel -j 10 \                  #<5>
    "fastqc --outdir $REPORTDIR $INDIR/{}_{1..2}.fastq.gz"
```

① We'll use 10 CPUs and 20G of memory.
② We will again use parallel to run fastqc in parallel on multiple files.
③ Specify the input and output directories as variables, and create the output directory.
④ Re-use the accession list we created previously.
⑤ Run fastqc in parallel. Note here we are *also* using a shell expansion `{1..2}` to simplify things. This runs fastqc on both files in a pair.

Run this script with `sbatch 01_fastqc.sh`. Monitor it and ensure that it produces the output you expect with no errors. Run `seff <jobid>` to see how efficiently we used our resource request.

Download (or connect to Xanadu's file system) and look at a few of the HTML reports. You

don't need to check them all because...

12.4.2 MultiQC on raw data

FastQC is going to produce 40 HTML reports - too many to comfortably look through one by one. We'll use MultiQC to aggregate the reports.

The script is `scripts/02_QC/02_multiqc.sh`. It looks like this:

```
#!/bin/bash
#SBATCH --job-name=multiqc
#SBATCH -n 1
#SBATCH -N 1
#SBATCH -c 2
#SBATCH --mem=10G
#SBATCH --partition=general
#SBATCH --qos=general
#SBATCH --mail-type=ALL
#SBATCH --mail-user=first.last@uconn.edu
#SBATCH -o %x_%j.out
#SBATCH -e %x_%j.err

hostname
date

#########################################################################
# Aggregate reports using MultiQC
#########################################################################

module load MultiQC/1.15

INDIR=../../results/02_qc/fastqc_reports/
OUTDIR=../../results/02_qc/multiqc

# run on fastqc output
multiqc -f -o ${OUTDIR} ${INDIR}
```

It should require little explanation at this point. Get the HTML file and look at it in a browser. You should see some minor adapter contamination consistent with read-through. This is unlikely to be a problem at this very small level, but we're going to trim it out anyway.

FastQC will label adapter contamination as "Illumina universal adapter". If you try to search for this term, you'll find it to be ambiguous. You'll remember that you need to supply adapters to Trimmomatic to be trimmed. So how can you figure this out? FastQC's adapter list is found here `/isg/shared/apps/fastqc/0.12.1/Configuration/adapter_list.txt`. You can see our adapter sequence is `AGATCGGAAGAG`. Trimmomatic is pre-packaged with common adapters here `/isg/shared/apps/Trimmomatic/0.39/adapters/`. Let's see if this sequence matches any of them:

```
grep "AGATCGGAAGAG" /isg/shared/apps/Trimmomatic/0.39/adapters/*
```

We get results:

```
/isg/shared/apps/Trimmomatic/0.39/adapters/NexteraPE-PE.fa:AGATCGGAAGAG
/isg/shared/apps/Trimmomatic/0.39/adapters/TruSeq2-PE.fa:AGATCGGAAGAGCGTCGTGTAGGGAAAGAGTGTAG
/isg/shared/apps/Trimmomatic/0.39/adapters/TruSeq2-PE.fa:AGATCGGAAGAGCGGTTCAGCAGGAATGCCGAGAC
/isg/shared/apps/Trimmomatic/0.39/adapters/TruSeq2-SE.fa:AGATCGGAAGAGCTCGTATGCCGTCTTCTGCTTG
/isg/shared/apps/Trimmomatic/0.39/adapters/TruSeq2-SE.fa:AGATCGGAAGAGCGTCGTGTAGGGAAAGAGTGT
/isg/shared/apps/Trimmomatic/0.39/adapters/TruSeq2-SE.fa:AGATCGGAAGAGCGGTTCAGCAGGAATGCCGAG
/isg/shared/apps/Trimmomatic/0.39/adapters/TruSeq3-PE-2.fa:AGATCGGAAGAGCGTCGTGTAGGGAAAGAGTGT
/isg/shared/apps/Trimmomatic/0.39/adapters/TruSeq3-PE-2.fa:AGATCGGAAGAGCACACGTCTGAACTCCAGTCA
/isg/shared/apps/Trimmomatic/0.39/adapters/TruSeq3-SE.fa:AGATCGGAAGAGCACACGTCTGAACTCCAGTCAC
/isg/shared/apps/Trimmomatic/0.39/adapters/TruSeq3-SE.fa:AGATCGGAAGAGCGTCGTGTAGGGAAAGAGTGTA
```

Let's use `/isg/shared/apps/Trimmomatic/0.39/adapters/TruSeq3-PE-2.fa` in Trimmomatic.

12.4.3 Running Trimmomatic in a SLURM array

Quality trimming requires a bit more resources for each job than than FastQC, and it takes longer to do. Here we'll use a SLURM array to run each trimming job independently. SLURM arrays are a feature of SLURM that allows you to essentially write a single script and have it run repeatedly. There is a guide to running SLURM arrays on Xanadu. Please read through it, try out the simple examples there, and then come back here.

So the idea is that, much like a bash loop, or running GNU parallel, in an array job script, we write generic code, and then figure out how to substitute in the specific files or parameters for each instance of the array. The advantage of arrays over using parallel, is that each instance of the array (or "array task") will be scheduled independently, receive its own resource allocation, and can run on any node where there is space for that job. This makes arrays better suited than parallel for repetitive jobs that require substantial resources. For example, if you had 30 tasks that each required 1 cpu and 10G of RAM, running that job with GNU parallel would require the majority of resources on a single node (30 CPUs and 300G of RAM), and there

might be a wait for those resources to become available. If you run the job as an array, however, each task can be slotted in on any node where resources are available, so your tasks will fan out and soak up smaller chunks of unused resources. If you have 1000 jobs, each requiring 20G of RAM and 10 CPUs, restricting yourself to a single node with GNU parallel will take *much* longer than allowing those jobs to spread out over the cluster. Only 4-6 could run at a time, depending on the node, whereas 40 could run simultaneously with an array (user max CPU limit on general partition is 400).

So, to sum up, SLURM arrays are a key means to execute parallel jobs on Xanadu.

For a SLURM array, every instance (or task) has a unique numerical value assigned to the environment variable `SLURM_ARRAY_TASK_ID`. We're going to use that variable to make substitutions into our generic code. Our script is `scripts/02_QC/03_trimmomatic.sh`. It looks like this:

```
#!/bin/bash
#SBATCH --job-name=trimmomatic
#SBATCH -n 1
#SBATCH -N 1
#SBATCH -c 1                    #<1>
#SBATCH --mem=15G
#SBATCH --partition=general
#SBATCH --qos=general
#SBATCH --mail-type=ALL
#SBATCH --mail-user=first.last@uconn.edu
#SBATCH -o %x_%A_%a.out       #<2>
#SBATCH -e %x_%A_%a.err       #<2>
#SBATCH --array=[1-20]%10     #<3>

hostname
date

#########################################################################
# Trimmomatic
#########################################################################

module load Trimmomatic/0.39
module load parallel/20180122

# set input/output directory variables
INDIR=../../data/fastq/
TRIMDIR=../../results/02_qc/trimmed_fastq
mkdir -p $TRIMDIR
```

```
# adapters to trim out
ADAPTERS=/isg/shared/apps/Trimmomatic/0.39/adapters/TruSeq3-SE.fa #<4>

# accession list
ACCLIST=../../metadata/accessionlist.txt

# run trimmomatic

SAMPLE=$( sed -n ${SLURM_ARRAY_TASK_ID}p ${ACCLIST} ) #<5>

java -jar $Trimmomatic PE -threads 4 \ #<6>
        ${INDIR}/${SAMPLE}_1.fastq.gz \
        ${INDIR}/${SAMPLE}_2.fastq.gz \
        ${TRIMDIR}/${SAMPLE}_trim_1.fastq.gz ${TRIMDIR}/${SAMPLE}_trim_orphans_1.fastq.gz
        ${TRIMDIR}/${SAMPLE}_trim_2.fastq.gz ${TRIMDIR}/${SAMPLE}_trim_orphans_2.fastq.gz
        ILLUMINACLIP:"${ADAPTERS}":2:30:10 \
        SLIDINGWINDOW:4:25 MINLEN:45
```

① 15G is probably overkill for Trimmomatic. We'll check at the end.
② These options specify the array job `.err` and `.out` file name formats. The file names will be `<jobname>_<jobid>_<SLURM_ARRAY_TASK_ID>.{err,.out}`
③ This specifies the array task IDs, in this case a range: `[1-20]` and the number of jobs to run at once: `%10`. If you leave out the number of simultaneous jobs to run, the array will run as many as your resource allocation (and available resources) allow. If the array is large enough, this will prevent you from running any other jobs.
④ The adapter sequences we determined we should supply.
⑤ In contrast to GNU parallel or a loop, we need a way to pull out a single sample for each array task. Here we use `sed` to extract a sample ID by its line number in our list of accessions.
⑥ This is our Trimmomatic command line, which we covered in the section on sequence QC.

With array jobs, as with loops and with GNU parallel, you may want to test that you have written the generic code correctly, and that the substitutions will occur as you expect. You could modify your code to simply echo important file paths and check the `.out` files to ensure things are correct:

```
#!/bin/bash
#SBATCH --job-name=trimmomatic
#SBATCH -n 1
#SBATCH -N 1
#SBATCH -c 1
```

```
#SBATCH --mem=15G
#SBATCH --partition=general
#SBATCH --qos=general
#SBATCH --mail-type=ALL
#SBATCH --mail-user=first.last@uconn.edu
#SBATCH -o %x_%A_%a.out
#SBATCH -e %x_%A_%a.err
#SBATCH --array=[1-20]%10

hostname
date

#########################################################################
# Trimmomatic
#########################################################################

module load Trimmomatic/0.39
module load parallel/20180122

# set input/output directory variables
INDIR=../../data/fastq/
TRIMDIR=../../results/02_qc/trimmed_fastq
mkdir -p $TRIMDIR

# adapters to trim out
ADAPTERS=/isg/shared/apps/Trimmomatic/0.39/adapters/TruSeq3-SE.fa

# accession list
ACCLIST=../../metadata/accessionlist.txt

# run trimmomatic

SAMPLE=$( sed -n ${SLURM_ARRAY_TASK_ID}p ${ACCLIST} )

echo $SAMPLE
echo ${INDIR}/${SAMPLE}_1.fastq.gz

# java -jar $Trimmomatic PE -threads 4 \
#         ${INDIR}/${SAMPLE}_1.fastq.gz \
#         ${INDIR}/${SAMPLE}_2.fastq.gz \
#         ${TRIMDIR}/${SAMPLE}_trim_1.fastq.gz ${TRIMDIR}/${SAMPLE}_trim_orphans_1.fastq.g
```

```
#         ${TRIMDIR}/${SAMPLE}_trim_2.fastq.gz ${TRIMDIR}/${SAMPLE}_trim_orphans_2.fastq.g
#         ILLUMINACLIP:"${ADAPTERS}":2:30:10 \
#         SLIDINGWINDOW:4:25 MINLEN:45
```

You can also simply set some of these variables in an interactive session and check if they get constructed correctly:

```
INDIR=../../data/fastq/
ACCLIST=../../metadata/accessionlist.txt
SLURM_ARRAY_TASK_ID=5
SAMPLE=$( sed -n ${SLURM_ARRAY_TASK_ID}p ${ACCLIST} )
echo $SAMPLE
echo ${INDIR}/${SAMPLE}_1.fastq.gz
```

It's advisable to try these methods of testing before launching a new array script.

Run the script now with `sbatch 03_trimmomatic.sh`.

You should see using `squeue` that up to 10 jobs are spawned simultaneously, and that new `fastq.gz` files are growing in the results directory. When the jobs have all completed, check that they were successful, and examine the resource efficiency for a few of the array tasks (`seff <jobid>_<taskid>`)

Trimmomatic will not print much in the way of progress reports. It will give a short summary in the `.err` file about how many reads were retained. To grab those, do

```
grep "Surviving" trimmomatic_<JOBID>_*err | sort -V
```

That will pull out the summary for each file and sort the summaries by the array task ID.

To get a better sense of how our data were impacted, we will run FastQC and MultiQC again below.

12.4.4 FastQC and MultiQC

We will run these programs just as we did previously. We have new scripts: `scripts/02_QC/04_fastqc_trim.sh` and `scripts/02_QC/05_multiqc_trim.sh`. Because we've covered this material already, we can use this as an opportunity to check out another feature of SLURM: dependencies.

You can set SLURM jobs to start only after some condition is satisfied. The option is `-d` or `--dependency` and you specify them as `--dependency=type:jobid`. The most type we'll use here is `afterok`, which means the dependent job will begin after the specified job completes without error. See here for more details.

So if we do:

```
sbatch 05_fastqc_trim.sh
```

And get the message

```
Submitted batch job 8017345
```

We can submit MultiQC, which requires the fastqc output as:

```
sbatch --dependency=afterok:8017345 05_multiqc.sh
```

You can automatically grab the job ID like this and submit both scripts:

```
JOBID=$(sbatch --parsable 04_fastqc_trim.sh)
sbatch --dependency=afterok:${JOBID} 05_multiqc.sh
```

When these are complete, download the FastQC and MultiQC results and compare them in terms of the number of sequences, the quality scores and the adapter content.

12.5 Exercises

For this section, your assignment is to clone a new copy of the github repository and modify the scripts we have done so far to run with **all 75 samples** in the experiment.

Answer the following questions:

13 Reference Alignment

Learning Objectives:
blah blah

Now that we've got our genome, our annotation, and we've downloaded our sequence data and QC'ed it, we're ready to begin the process of quantifying gene expression. Remember that the measurement we are making is **a count of fragments of RNA derived from a given gene**. In this chapter we will quantify gene expression by mapping our RNA fragments against a reference genome. We'll use the annotation of the reference genome to determine which fragments originated from which genes. We'll cover each of the steps needed for this analysis, and talk about the relevant file formats in depth.

13.1 Indexing the reference genome

The process of alignment to a reference genome requires identifying possible mapping locations for each read pair, scoring them, and then choosing the one with the best score (or possibly randomly selecting from among ties).

It goes without saying that even small genomes are pretty large (at least among eurkaryotes), so the process of finding initial candidate matches would be quite time-consuming if you had to check every location in the genome for every read. To get around this, all commonly used reference alignment software creates an **index**. Indexes function much like they do in large texts: interesting topics (in this case short genomic subsequences) and their page numbers (genomic locations) are stored. When you want to find locations of interest, you find them quickly through the index, rather than scanning through the entire text.

Variations on the genome index and the way it is searched have been a main target for innovation among different read aligners, so each one will require you to create a particular index.

When aligning RNA-seq data to a reference genome, we have a particular challenge: most (but typically not all) reads have been derived from mature mRNAs with introns spliced out. That means the aligner needs to be able to identify when reads (or read pairs) span splice junctions and accurately align across them. When aligning RNA-seq data, we typically use **spliced aligners** designed to deal with this issue.

Here we are going to use HISAT2. Creating an index with hisat2 is pretty simple. We'll use script `scripts/03_align/01_index.sh`. It looks like this:

```
#!/bin/bash
#SBATCH --job-name=hisat2_index
#SBATCH -n 1
#SBATCH -N 1
#SBATCH -c 16
#SBATCH --mem=10G
#SBATCH --partition=general
#SBATCH --qos=general
#SBATCH --mail-type=ALL
#SBATCH --mail-user=first.last@uconn.edu
#SBATCH -o %x_%j.out
#SBATCH -e %x_%j.err

hostname
date

######################################################################
# Index the Genome
######################################################################

# load software
module load hisat2/2.2.1

# input/output directories
OUTDIR=../../genome/hisat2_index
mkdir -p $OUTDIR

GENOME=../../genome/Fundulus_heteroclitus.Fundulus_heteroclitus-3.0.2.dna_sm.toplevel.fa

hisat2-build -p 16 $GENOME $OUTDIR/Fhet
```

We should be getting pretty familiar with the general organization of our scripts now, so go ahead and run this with `sbatch 01_index.sh`. When it completes, it should have created an index, which is actually a series of files, with the path/prefix `$OUTDIR/Fhet`. When aligning reads, you'll specify that path/prefix, not the reference genome itself. We don't need to be concerned with what's in these index files, as we will never need to parse or modify them.

13.2 Aligning to the reference

Now we have our index ready to go, we're ready to align reads to the reference genome (again using `HISAT2`). Producing a usable alignment file requires a few steps:

1. Alignment. Read pairs are passed to the aligner, and alignments information produced. Reads come out in the order they went in, but as a single file this time (more on the format below).
2. Position-sorting. For the alignment file to be useful, reads need to be sorted by their mapping position in the genome. This allows us to index the file and quickly access all the reads from a given genomic region.
3. Compressing. Alignment records contain all the same information as a fastq file PLUS the alignment information, so we want to keep these files compressed (more on the format below, we don't use gzip for this one).
4. Indexing. Just like with the genome, we index the alignment files for rapid access to data from particular genomic regions.

You can do steps 1-3 one by one, producing intermediate alignment files each time, but this will cause your data to balloon and require you to clean up intermediate files. We're going to take care of it in a pipe.

Our script is another array script: `scripts/03_align/02_align.sh`. It looks like this:

```
#!/bin/bash
#SBATCH --job-name=align
#SBATCH -n 1
#SBATCH -N 1
#SBATCH -c 3              #<1>
#SBATCH --mem=15G         #<1>
#SBATCH --partition=xeon
#SBATCH --qos=general
#SBATCH --mail-type=ALL
#SBATCH --mail-user=first.last@uconn.edu
#SBATCH -o %x_%A_%a.out
#SBATCH -e %x_%A_%a.err
#SBATCH --array=[1-20]%10

hostname
date

######################################################################
# Align reads to genome
######################################################################
```

```
module load hisat2/2.2.1
module load samtools/1.16.1

INDIR=../../results/02_qc/trimmed_fastq
OUTDIR=../../results/03_align/alignments
mkdir -p ${OUTDIR}

# this is an array job.
    # one task will be spawned for each sample
    # for each task, we specify the sample as below
    # use the task ID to pull a single line, containing a single accession number from the
    # then construct the file names in the call to hisat2 as below

INDEX=../../genome/hisat2_index/Fhet                                    #<2>

ACCLIST=../../metadata/accessionlist.txt

SAMPLE=$(sed -n ${SLURM_ARRAY_TASK_ID}p ${ACCLIST})

# run hisat2
hisat2 \                                                                #<3>
    -p 4 \
    -x ${INDEX} \
    -1 ${INDIR}/${SAMPLE}_trim_1.fastq.gz \
    -2 ${INDIR}/${SAMPLE}_trim_2.fastq.gz | \
samtools sort -@ 1 -T ${OUTDIR}/${SAMPLE} - >${OUTDIR}/${SAMPLE}.bam    #<4>

# index bam files
samtools index ${OUTDIR}/${SAMPLE}.bam                                  #<5>
```

① The amount of memory required for aligners depends on the size of the genome being analyzed. The *F. heteroclitus* genome is about 1GB, and our index takes about 1.5G of disk space. We requested 15G of memory above, but probably could have gotten by with more like 5G. We request 3 cpus (2 for the aligner and 1 for the sorter). The aligner could probably go faster with a few more, but this doesn't take too long as it is.

② Note that we provide only the path and prefix to our index files.

③ Our alignment command, broken up across a few lines with \. It's fairly straightforward. It writes to the standard out. We capture it with a pipe and send it to be sorted and compressed.

④ Here we are using `samtools sort` to sort the file. `-@ 1` means use a single CPU. `-T ${OUTDIR}/${SAMPLE}` is specifying the location and prefix of any temporary files (sorting can create many, many temporary alignment files and then merge them). `-` tells

samtools to read from the standard input (the pipe). And finally we write to a file with the suffix `.bam`, which is the compressed alignment format, written by default.

⑤ For the last step, we create the index file with `samtools index`. This will produce a file with the suffix `.bam.bai` in the same directory as the bam.

Submit this script with `sbatch 02_align.sh` and monitor it. Ensure that all alignments complete successfully, and that you wind up with 20 `.bam` files and 20 `.bam.bai` files in the output directory. Check the `.err` file. hisat2 produces a helpful (but annoying to parse) summary of the alignment rate. Not all aligners do this. An example:

```
13732517 reads; of these:
  13732517 (100.00%) were paired; of these:
    4263933 (31.05%) aligned concordantly 0 times
    9249450 (67.35%) aligned concordantly exactly 1 time
    219134 (1.60%) aligned concordantly >1 times
    ----
    4263933 pairs aligned concordantly 0 times; of these:
      445817 (10.46%) aligned discordantly 1 time
    ----
    3818116 pairs aligned 0 times concordantly or discordantly; of these:
      7636232 mates make up the pairs; of these:
        4634197 (60.69%) aligned 0 times
        2823388 (36.97%) aligned exactly 1 time
        178647 (2.34%) aligned >1 times
83.13% overall alignment rate
```

Ideally, reads will map exactly one time. When reads map zero times, there isn't a good match in the reference genome. If they map more than once, a very similar sequence occurs multiple times in the reference.

Concordant alignment means the pairs of sequences have an orientation and distance between them consistent with expectations. The expected orientation is for pairs to be on opposite strands, (they were generated from opposite ends of a single fragment of DNA) and "inward" facing. The expected distance is often calculated from the library itself (the fragment size distribution is library specific), but generally read pairs ought not be mapped to separate contigs or chromosomes. When any of these expectations are violated, pairs are marked as discordantly mapping.

Ideally we would see a concordant alignment rate higher than 67%, but this genome is an older assembly. It is highly fragmented and probably missing some sequences. That would contribute to cases where reads map discordantly (across contigs, particularly) or one or both reads in a pair fails to map. An overall alignment rate of 83% is not stellar, but not terrible either. For a good library on a highly contiguous and complete genome the overall mapping percentage could be expected to be in the high 80s to 90s.

13.3 SAM/BAM format

Now we have our alignments in `.bam` files. `BAM` stands for **binary alignment/map** It is (barring some minor differences) the compressed, binary version of `SAM`, or **sequence alignment/map** format. Even though we always store our files in BAM, we are going to cover `SAM` format here in some detail, as it is the plain-text, human-readable format, and as you will see, it is trivial to convert between them as necessary.

You can find the official specification with lots of detail and definitions here. We're going to go over the basics here. As with many formats in bioinformatics,

13.3.1 The header

`SAM` files begin with a **header**. All header lines start with `@` and a tag indicating the contents. The first line gives the SAM version and the sort order, e.g.:

```
@HD VN:1.0  SO:coordinate
```

Next comes a sequence dictionary (`@SQ`), which lists the names and lengths of sequences in the reference genome:

```
@SQ SN:KN805525.1   LN:6356336
@SQ SN:KN805526.1   LN:6250690
@SQ SN:KN811289.1   LN:6115441
...
```

There may be some others for some types of alignment, but the last lines in our files contain program calls (`@PG`) used to produce the file:

```
@PG ID:hisat2   PN:hisat2   VN:2.2.1    CL:"/isg/shared/apps/hisat2/2.2.1/hisat2-align-s --wr
@PG ID:samtools.1   PN:samtools PP:samtools VN:1.16.1   CL:samtools sort -@ 1 -T ../../result
```

Note that the hisat2 call is different from our actual command line. The `hisat2` program we used is a wrapper script that called a separate program to do the alignments.

13.3.2 The alignments

After the header comes alignment data in tabular format. The tabular data has 11 mandatory columns, though many aligners add more custom columns after. The mandatory columns are:

Column Number	Column Name	Description
1	QNAME	Query template name. Unique identifier for each read or read pair.
2	FLAG	Bitwise FLAG. Describes properties of the read, such as paired, mapped, etc.
3	RNAME	Reference sequence name.
4	POS	1-based leftmost mapping position.
5	MAPQ	Mapping quality. A Phred-scaled quality score.
6	CIGAR	CIGAR string. Base-level alignment of the read to the reference.
7	RNEXT	Reference sequence the mate is mapped to.
8	PNEXT	Position of the mate/next read. 1-based, leftmost.
9	TLEN	Template length. Length of fragment inferred from mapped read pair
10	SEQ	Nucleotide sequence.
11	QUAL	Base qualities. ASCII-encoded base quality scores for the read sequence.

A few example alignments:

```
SRR12475447.2760    163 KN805525.1  223713  60  13M2350N88M =   226185  223 GCTCAAGATGAACATTC
SRR12475447.2735    147 KN805525.1  223716  60  10M2350N43M =   221834  -1935   CAAGATGAACATT
SRR12475447.2726    83  KN805525.1  223729  60  101M    =   223510  -335    GGAAACACACTGATTAT
SRR12475447.2727    83  KN805525.1  223743  60  101M    =   223469  -375    TATGGTCCCTGTGCATT
```

Some of the columns are straightforward, but others bear some explanation.

`FLAG` contains a fair bit of information encoded in a single integer. We have four different values of FLAG in the above example: 164,147 and 83 (x2). The easiest way to understand what these flags mean is to visit Decoding SAM flags. If you go there and enter `147`, you'll see boxes checked for "read paired", "read mapped in proper pair", "read reverse strand", and "second in pair". This indicates this read is R2, mapped in a proper pair and on the reverse strand. You can infer that its mate will be on the forward strand. `163` only differs in that the *mate* is on the reverse strand. `83` indicates the read is properly paired, on the reverse strand and *first* in the pair. You can filter on these flags, or any combination of them, as we will see below.

`MAPQ` contains a Phred-scaled probability that the read mapping is in error. This is a function of whether or not there are other locations in the genome with competitive alignment scores. If there are multiple equally plausible mapping locations, this number will be 0. The lowest probability of error hisat2 will assign is 60 (or 10^{-6}). Unmapped reads will have `*`.

`CIGAR` gives the base-level alignment of the read to the reference. It's formatted as , where "number" is a number of bases and "letter" is indicates if the bases are aligned 1-1 to the reference, or something else is going on (an insertion, deletion, splice, or unaligned bases). The specification gives all the details, but in our examples `101M` indicates 101 bases of this read are aligned to the genome. They don't have to have identical nucleotides, just be aligned one-to-one. `10M2350N43M` indicates 10 bases are aligned (`10M`), followed by a 2,350 "skipped bases" (`2350N`) then 43 more mapped bases (`43M`). The skipped bases are how hisat2 indicates a splice junction.

`TLEN` gives the template length implied by genomic span of the mapped reads (the spec says "observed template length", but I don't like this term because it does not incorporate insertions/deletions, some of which may occur in the gap between reads when they don't overlap). This number is positive for the left-most mapping fragment and negative for the rightmost mapping fragment.

Files that are taken straight from the aligner, or deliberately sorted by query name will have mate pairs on adjacent lines. This is not true of coordinate-sorted reads, for which mates may be quite some distance away in the file.

A pair of reads will look like this:

```
SRR12475447.2735    99  KN805525.1  221834  60  101M    =   223716  1935    GGGTCATCTTTGTAAG
SRR12475447.2735    147 KN805525.1  223716  60  10M2350N43M =   221834  -1935   CAAGATGAACAT
```

Note the opposite-signed and very long TLEN. Presumably the length is due to the gap between the reads spanning an exon (in addition to the splice in one of the reads). Most sequenced fragments from Illumina will actually be between 200-600bp.

13.3.3 Manipulating alignments

13.3.4 Visualizing alignments

check contig names and number!

13.4 Alignment QC

14 Expression Quantification

Learning Objectives:
blah blah

Part IX

Measuring Gene Expression II

Part X

Learning Objectives

Learning Objectives:
Generate a reference transcriptome with StringTie
Generate a reference transcriptome de novo with Trinity
Annotate a reference transcriptome
Quantify gene expression using a transcriptome reference

15 Untitled

16

Part XI

Statistical Analysis I

Part XII

Learning Objectives

Learning Objectives:
blah blah

17 Untitled

18

Part XIII

Statistical Analysis II

Part XIV

Learning Objectives

Learning Objectives:
blah blah

19 Untitled

20

Part XV

Writing Reports

Part XVI

Learning Objectives

Learning Objectives:
blah blah

21 Untitled

22

Part XVII

Presentations

Part XVIII

Learning Objectives

Learning Objectives:
blah blah

23 Untitled

24